BPSC TRE 4.0 · STET · DSSSB TGT · KVS/NVS

Computer Science — the complete course

The whole BPSC TRE 4.0 / STET / DSSSB Computer Science syllabus taught in short lessons: one big idea, an everyday picture, solved examples, exam traps, memory tricks and a quick quiz in every lesson.

12 modules · 47 lessons · 146 quiz questions

Contents

1. Number systems & digital logic
1.1 How binary works (binary → decimal) · 1.2 Decimal → binary, octal & hexadecimal · 1.3 Binary addition, negative numbers & codes · 1.4 Logic gates & Boolean algebra · 1.5 K-map: simplifying with a picture · 1.6 Circuits that calculate & circuits that remember

2. Programming in C & Python
2.1 What is a program? Algorithms, flowcharts & translators · 2.2 C basics: variables, data types & operators · 2.3 Decisions & loops: if, switch, for, while, do-while · 2.4 Functions & recursion · 2.5 Arrays, strings & pointers in C · 2.6 Python basics: variables, types & operators · 2.7 Python strings, lists, tuples, dictionaries & errors

3. Data structures
3.1 What is a data structure? Arrays vs linked lists · 3.2 Stack & queue (and converting expressions) · 3.3 Trees, binary search trees & heaps · 3.4 Graphs & hashing

4. Algorithms & complexity
4.1 Time complexity (Big-O) from zero · 4.2 Searching & sorting — traced step by step · 4.3 Problem-solving strategies & graph algorithms

5. Operating systems
5.1 What an OS does; processes & threads · 5.2 CPU scheduling — Gantt charts step by step · 5.3 Process synchronisation & semaphores · 5.4 Deadlock & Banker's algorithm · 5.5 Memory management, virtual memory & disk scheduling

6. DBMS & SQL
6.1 What is a database? Tables, keys & the ER model · 6.2 SQL from zero: creating tables & asking questions · 6.3 Joins, GROUP BY, HAVING & subqueries · 6.4 Functional dependencies & normalisation (1NF → BCNF) · 6.5 Transactions, ACID, concurrency & indexing

7. Computer networks & security
7.1 Network basics: types, shapes, devices & cables · 7.2 The OSI & TCP/IP models (the 7 layers) · 7.3 IP addresses & subnetting — fully worked · 7.4 Protocols, port numbers, ARP & DNS · 7.5 Error control, who-may-talk rules, flow control & network security

8. Computer organisation & architecture
8.1 Inside the CPU: parts, registers & the instruction cycle · 8.2 Instructions & addressing modes · 8.3 Memory hierarchy, cache, pipelining & I/O

9. OOP with C++
9.1 Classes, objects & the four pillars of OOP · 9.2 Constructors, destructors, overloading & overriding · 9.3 Virtual functions, abstract classes & virtual destructors

10. Software engineering
10.1 SDLC & software development models · 10.2 Testing, cohesion & coupling, and metrics

11. Theory of computation & compilers
11.1 Automata & the Chomsky hierarchy · 11.2 How a compiler works: the six phases

12. Web, emerging trends & cyber law
12.1 How the web works: HTML, CSS, JavaScript & HTTP · 12.2 Emerging technologies, cyber security & the IT Act

MODULE 1

Number systems & digital logic

How computers count, add and decide — with 0s and 1s.

LESSON 1.1 · 12 MIN

How binary works (binary → decimal)

In one line: **In binary each place is worth double the place to its right: 1, 2, 4, 8, 16… A 1 means "take it", a 0 means "skip it".**

Step 1 — start with something you already know: our normal (decimal) numbers. We use ten digits: 0 1 2 3 4 5 6 7 8 9. That is why decimal is called **base 10**.

In a number like **345**, every digit sits in a "place", and every place has a value. Right-most is the **ones** place (1), next is **tens** (10), next is **hundreds** (100). Notice each place is **10 times** the place on its right.

	Hundreds	Tens	Ones
Digit	3	4	5
Place value	100	10	1
Digit × place value	3 × 100 = 300	4 × 10 = 40	5 × 1 = 5

300 + 40 + 5 = 345 — you do this in your head every day without noticing.

Step 2 — why do computers use binary? Inside a computer everything is an electric switch that is either **OFF** or **ON**. Two states → two digits: **0 (off)** and **1 (on)**. A number system with only two digits is called **binary**, or **base 2**. One binary digit is called a **bit**.

Step 3 — place values in binary. Decimal places grow ×10 because decimal has 10 digits. Binary has 2 digits, so its places grow **×2**. Start with **1** on the right and keep **doubling** as you move left:

Place (from right)	8th	7th	6th	5th	4th	3rd	2nd	1st
Place value	128	64	32	16	8	4	2	1
As a power of 2	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

1 → 2 → 4 → 8 → 16 → 32 → 64 → 128: each one is double the previous.

💡 **THINK OF IT LIKE THIS**

Imagine a row of light bulbs labelled 8, 4, 2, 1. A binary number tells you which bulbs are ON (1) and which are OFF (0). The number's value is simply the total of the labels of the bulbs that are ON.

SOLVED EXAMPLE

Convert $(1011)_2$ to decimal — slowly

1. Write the binary number with space between digits: 1 0 1 1. It has 4 digits, so we need 4 place values.
2. Under the RIGHT-most digit write 1. Moving left, double each time: 1 → 2 → 4 → 8. So the place values (left to right) are 8 4 2 1.
3. Now pair each digit with its place value: the 1 sits on 8, the 0 sits on 4, the 1 sits on 2, the 1 sits on 1.
4. A digit 1 means TAKE that place value; a digit 0 means SKIP it. So we take 8, skip 4, take 2, take 1.
5. Add only what you took: 8 + 2 + 1 = 11.

✅ $(1011)_2$ = 11 in decimal

	Left-most			Right-most
Binary digit	1	0	1	1
Place value	8	4	2	1
Take or skip?	take 8	skip	take 2	take 1

Same example as a table: 8 + 2 + 1 = 11

SOLVED EXAMPLE

One more: $(110010)_2$

1. 6 digits → place values from the right: 1, 2, 4, 8, 16, 32 → written left to right: 32 16 8 4 2 1
2. Line them up: 1→32, 1→16, 0→8, 0→4, 1→2, 0→1
3. Take the places that have a 1: 32, 16 and 2
4. Add: 32 + 16 + 2

✅ $(110010)_2$ = 50

What does the little $_2$ mean? It tells you the base. $(1011)_2$ is binary; $(1011)_{10}$ would be one thousand and eleven in decimal. Always check the small number before you start.

MEMORY TRICK

Learn the doubling chain by heart: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024. Once you know it, every binary → decimal question is just “add the places that have a 1”.

⚠ EXAM TRAP

Place values always start from the RIGHT (1 is the right-most place). A common mistake is to start 1, 2, 4, 8 from the left — that gives the wrong answer.

- 1 bit = one 0 or 1; 4 bits = 1 nibble; 8 bits = 1 byte
- With n bits you can make 2^n different patterns. 3 bits → 2^3 = 8 patterns (000 to 111, i.e. 0 to 7)
- The biggest number with n bits is $2^n - 1$ (all ones). 4 bits: 1111 = 8 + 4 + 2 + 1 = 15 = $2^4 - 1$
- Memory units grow by 1024 (= 2^{10}): 1 KB = 1024 bytes, 1 MB = 1024 KB, 1 GB = 1024 MB

⏱ RECAP IN 30 SECONDS

✓ Binary has two digits, 0 and 1 (OFF and ON)

✓ Place values from the right: 1, 2, 4, 8, 16, 32, 64, 128 — each double the last

✓ Binary → decimal: add the place values that have a 1

✓ n bits → 2^n patterns; the largest n-bit number is $2^n - 1$

✍ **Quick quiz**

Q1. $(1101)_2$ in decimal is:

(A) 11 (B) 14
(C) 12 (D) 13

Q2. $(10000)_2$ in decimal is:

(A) 16 (B) 32
(C) 5 (D) 10

Q3. How many different values can 8 bits represent?

(A) 256 (B) 128
(C) 255 (D) 8

Answers: Q1 D — Places 8 4 2 1 → take 8, take 4, skip 2, take 1 → 8 + 4 + 1 = 13. · Q2 A — 5 digits → left-most place is 16; only that place has a 1. · Q3 A — 2^8 = 256 patterns (0 to 255).

LESSON 1.2 · 14 MIN

Decimal → binary, octal & hexadecimal

In one line: **To write a number in binary, break it into powers of 2 (16 + 8 + 1); octal and hex are just short ways of writing groups of 3 or 4 bits.**

Method 1 (easiest for beginners): use the doubling chain. Write the place values 1, 2, 4, 8, 16, 32… and keep picking the biggest one that still fits into your number.

➖ SOLVED EXAMPLE

Convert 25 to binary — the "biggest fits" method

1. Doubling chain: 1, 2, 4, 8, 16, 32. 32 is bigger than 25, so the biggest that fits is 16. Put a 1 under 16. Left over: 25 − 16 = 9
2. Next place 8: does 8 fit into 9? Yes → 1. Left over: 9 − 8 = 1
3. Next place 4: does 4 fit into 1? No → 0
4. Next place 2: does 2 fit into 1? No → 0
5. Next place 1: does 1 fit into 1? Yes → 1. Left over: 0 — done
6. Read the 1s and 0s from 16 down to 1: 1 1 0 0 1

✅ $25 = (11001)_2$ — check: 16 + 8 + 1 = 25 ✓

Method 2 (what textbooks use): keep dividing by 2. Divide the number by 2, write down the remainder (always 0 or 1), then divide the answer again — until you reach 0. The remainders, read from the **bottom up**, are the binary number.

Divide	Answer (quotient)	Remainder
25 ÷ 2	12	1 ← last digit
12 ÷ 2	6	0
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0	1 ← first digit

Read the remainders from bottom to top: 1 1 0 0 1 → $25 = (11001)_2$

⚠ EXAM TRAP

Read the remainders from the BOTTOM to the TOP. Reading top-down gives 10011 (= 19), which is a classic wrong option in exams.

Octal (base 8) uses eight digits 0–7, so its places are 1, 8, 64, 512… (each ×8). **Hexadecimal (base 16)** uses sixteen symbols: 0–9 and then letters for 10 to 15.

Hex symbol	A	B	C	D	E	F
Value	10	11	12	13	14	15

Hex places are 1, 16, 256 … (each ×16)

SOLVED EXAMPLE

Convert $(2F)_{16}$ to decimal

1. Places from the right: 1, 16
2. F is on the 1-place: F = 15 → 15 × 1 = 15
3. 2 is on the 16-place: 2 × 16 = 32
4. Add: 32 + 15

✅ $(2F)_{16} = 47$

The shortcut between binary, octal and hex. 8 = 2^3, so one octal digit is exactly **3 bits**. 16 = 2^4, so one hex digit is exactly **4 bits**. That means you can convert by simply grouping bits — no division needed.

3 bits	000	001	010	011	100	101	110	111
Octal digit	0	1	2	3	4	5	6	7

SOLVED EXAMPLE

Convert $(110101)_2$ to octal and to hex

1. Octal: cut into groups of 3 starting from the RIGHT → 110 | 101
2. 110 = 4 + 2 = 6, 101 = 4 + 1 = 5 → octal 65
3. Hex: cut into groups of 4 from the RIGHT → 11 | 0101; pad the left group with zeros → 0011 | 0101
4. 0011 = 3, 0101 = 5 → hex 35

✅ $(110101)_2 = (65)_8 = (35)_{16} = 53$ in decimal

MEMORY TRICK

To go between octal and hex, always pass through binary: octal → write each digit as 3 bits → regroup into 4 bits → hex.

⚠ EXAM TRAP

"10" is a different number in every base: two in binary, eight in octal, ten in decimal, sixteen in hex. Always read the base first.

⏱ **RECAP IN 30 SECONDS**

- ✓ Decimal → binary: subtract the biggest power of 2 that fits, again and again (or divide by 2 and read remainders bottom-up)
- ✓ Octal digits 0–7, places ×8; hex digits 0–9 and A–F (10–15), places ×16
- ✓ 1 octal digit = 3 bits; 1 hex digit = 4 bits — group from the right
- ✓ Octal ↔ hex: always go through binary

✍ **Quick quiz**

Q1. 13 in binary is:

(A) 1011 (B) 1110
(C) 1001 (D) 1101

Q2. $(2F)_{16}$ in decimal is:

(A) 45 (B) 52
(C) 47 (D) 32

Q3. $(101110)_2$ in octal is:

(A) 65 (B) 46
(C) 56 (D) 2E

Answers: Q1 D — 13 = 8 + 4 + 1 → places 8 4 2 1 → 1 1 0 1. · Q2 C — 2 × 16 + 15 (F) = 47. · Q3 C — Groups of 3 from the right: 101 | 110 → 5 | 6.

LESSON 1.3 · 16 MIN

Binary addition, negative numbers & codes

In one line: **Binary addition works like school addition — only you carry at 2 instead of 10. Negative numbers are stored with the 2's complement trick.**

Part 1 — adding in binary. Remember school addition: 7 + 5 = 12 → you write 2 and **carry 1** to the next column, because one column can only hold up to 9. In binary one column can only hold up to **1**, so as soon as a column reaches 2 you write 0 and carry 1.

Add these bits	Total (in decimal)	Write down	Carry to the left
0 + 0	0	0	0
0 + 1	1	1	0
1 + 0	1	1	0
1 + 1	2	0	1
1 + 1 + 1 (with a carry)	3	1	1

Only these five cases exist. Learn them and you can add any binary numbers.

➖ SOLVED EXAMPLE

Add 0110 (6) + 0011 (3)

1. Write them one under the other and start from the RIGHT column, exactly like school addition.
2. Column 1 (right-most): 0 + 1 = 1 → write 1, no carry.
3. Column 2: 1 + 1 = 2 → write 0, carry 1 to column 3.
4. Column 3: 1 + 0 + carry 1 = 2 → write 0, carry 1 to column 4.
5. Column 4: 0 + 0 + carry 1 = 1 → write 1.
6. Read the answer: 1001.

✅ 0110 + 0011 = 1001 → 8 + 1 = 9 ✓ (6 + 3 = 9)

Part 2 — how can a computer store a negative number? It only has 0s and 1s — there is no "−" sign. The answer used by every modern computer is called **2's complement**.

First, the **1's complement**: simply **flip every bit** (0 becomes 1, 1 becomes 0). Then the **2's complement** = 1's complement **+ 1**.

➖ SOLVED EXAMPLE

Find the 2's complement of 0101 (which is 5)

1. Flip every bit: 0101 → 1010. This is the 1's complement.
2. Add 1: 1010 + 1 = 1011. (Right column: 0 + 1 = 1, nothing else changes.)
3. So in 4-bit 2's complement, 1011 means −5.

✅ 2's complement of 0101 = 1011 (this represents −5)

🧠 MEMORY TRICK

Fast way: start from the RIGHT, copy every bit up to and including the FIRST 1 you meet, then flip all the remaining bits. 0101 → copy "1", flip "010" to "101" → 1011. Same answer, no addition needed.

Why is this useful? Because subtraction becomes addition. To do A − B, the computer adds A + (2's complement of B) and simply throws away any extra carry on the far left. The computer needs only an adder circuit — no separate subtractor.

➖ SOLVED EXAMPLE

Calculate 7 − 5 using only addition (4 bits)

1. Write 7 in binary: 0111. Write 5 in binary: 0101.
2. Make −5: 2's complement of 0101 = 1011.
3. Now ADD 7 and −5: 0111 + 1011.
4. Right column 1+1 = 2 → write 0 carry 1; next 1+1+1 = 3 → write 1 carry 1; next 1+0+1 = 2 → write 0 carry 1; last 0+1+1 = 2 → write 0 carry 1.
5. The result is 1 0010 — five bits. We only keep 4 bits, so the extra 1 on the far left is dropped → 0010.

✅ 0010 = 2 ✓ (7 − 5 = 2)

How do you know a 2's-complement number is negative? Look at the **left-most bit** (the sign bit): **0 = positive, 1 = negative**. With 4 bits the numbers go from 1000 (−8) up to 0111 (+7).

Bits	Smallest number	Largest number
4 bits	−8	+7
8 bits	−128	+127
n bits	-2^{n-1}	$2^{n-1} - 1$

There is one more negative number than positive, because zero takes one of the "positive" patterns.

⚠ EXAM TRAP

The range is NOT symmetric: 8 bits go down to −128 but only up to +127. Exams love asking "the minimum value of an 8-bit signed integer" — the answer is −128, not −127.

Why not use 1's complement? In 1's complement 0000 is +0 and 1111 is −0 — two different patterns for zero. That wastes a pattern and complicates circuits. 2's complement has only one zero, so computers use it.

Part 3 — codes you must know by name.

Code	What it is	Example
BCD (8421)	Each DECIMAL digit written separately in 4 bits	45 → 0100 0101 (4 → 0100, 5 → 0101)
Gray code	Each next number changes only ONE bit	0, 1, 3, 2 → 00, 01, 11, 10 (used in K-maps)
Excess-3	BCD + 3	5 → 5 + 3 = 8 → 1000
ASCII	7-bit code for 128 characters	'A' = 65, 'a' = 97, '0' = 48, space = 32
Unicode (UTF-8, UTF-16)	Covers all world scripts, including Hindi	UTF-8 uses 1–4 bytes per character

🧠 **MEMORY TRICK**

ASCII: capital letters start at 65, small letters at 97 — the gap is always 32. So 'a' = 'A' + 32.

⏱ **RECAP IN 30 SECONDS**

- ✓ Binary addition: 1 + 1 = 0 carry 1; 1 + 1 + 1 = 1 carry 1
- ✓ 1's complement = flip all bits; 2's complement = flip all bits + 1
- ✓ Subtraction = add the 2's complement and drop the extra carry
- ✓ Left-most bit 1 means negative; n-bit range is -2^{n-1} to $2^{n-1} - 1$
- ✓ BCD codes each decimal digit in 4 bits; Gray code changes one bit at a time

✍ **Quick quiz**

Q1. The 2's complement of 1100 (4 bits) is:

(A) 1011 (B) 0011
(C) 0100 (D) 0101

Q2. Range of an 8-bit signed number in 2's complement:

(A) −127 to 127 (B) 0 to 255
(C) −128 to 128 (D) −128 to 127

Q3. 1011 + 0110 in binary is:

(A) 11001 (B) 10010
(C) 1111 (D) 10001

Q4. The BCD code of 39 is:

(A) 100111 (B) 0011 1001
(C) 0010 0111 (D) 0011 1000

Answers: Q1 C — Flip → 0011, add 1 → 0100. (Shortcut: copy "100", flip "1" → 0100.) · Q2 D — -2^7 to $2^7 - 1$. · Q3 D — 11 + 6 = 17 = 16 + 1 → 10001. · Q4 B — 3 → 0011, 9 → 1001, written separately.

LESSON 1.4 · 16 MIN

Logic gates & Boolean algebra

In one line: **A logic gate is a tiny decision-maker: it looks at its inputs (0 or 1) and gives one output by a fixed rule.**

What is a logic gate? Think of it as a tiny machine with one or more inputs and exactly one output. Every input and output is either **1 (true / ON)** or **0 (false / OFF)**. The gate follows one fixed rule. Every calculation a computer does is built from millions of these gates.

A **truth table** is simply a list of every possible input combination and what the gate outputs for each. With 2 inputs there are 4 combinations (00, 01, 10, 11).

1. AND gate — output is 1 only when **ALL** inputs are 1. Written A · B (or AB).

> 💡 **THINK OF IT LIKE THIS**
> AND is a room with two locks on one door. You get in (output 1) only if you have key A AND key B. One key alone is not enough.

2. OR gate — output is 1 when **ANY** input is 1 (one or both). Written A + B.

> 💡 **THINK OF IT LIKE THIS**
> OR is a room with two doors. If door A OR door B (or both) is open, you can get in.

3. NOT gate — only one input; it gives the **opposite**. NOT 1 = 0, NOT 0 = 1. Written A′ or Ā (A-bar).

4. NAND = NOT of AND (AND, then flip the answer). **5. NOR** = NOT of OR (OR, then flip). **6. XOR** (exclusive OR) = 1 when the inputs are **different**. **7. XNOR** = 1 when the inputs are the **same**.

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

Read row by row: e.g. A = 1, B = 0 → AND gives 0, OR gives 1, XOR gives 1.

> 🧠 **MEMORY TRICK**
> Shortcuts: AND → “any 0 makes it 0”. OR → “any 1 makes it 1”. NAND and NOR are just the AND and OR columns flipped. XOR → “are they different?”. XNOR → “are they equal?”.

Why are NAND and NOR called universal gates? Because you can build ANY other gate (NOT, AND, OR, XOR…) using only NAND gates — or only NOR gates. Factories like this because they can

make chips from one kind of gate.

SOLVED EXAMPLE

Make a NOT gate from one NAND gate

1. Connect the same signal A to BOTH inputs of a NAND gate.
2. NAND gives $(A \cdot A)'$. But $A \cdot A$ is just A ($1 \cdot 1 = 1$, $0 \cdot 0 = 0$).
3. So the output is A' — the opposite of A. That is exactly a NOT gate.

✅ NOT from NAND = 1 gate. (AND needs 2 NANDs, OR needs 3, XOR needs 4.)

Boolean algebra = rules for simplifying these expressions, like normal algebra but with only 0 and 1. The most important rules:

FORMULAS

Identity	$A + 0 = A,\ A \cdot 1 = A$
Null	$A + 1 = 1,\ A \cdot 0 = 0$
Idempotent	$A + A = A,\ A \cdot A = A$
Complement	$A + A' = 1,\ A \cdot A' = 0$
Double NOT	$(A')' = A$
Absorption	$A + A \cdot B = A,\ A \cdot (A + B) = A$
De Morgan 1	$(A \cdot B)' = A' + B'$
De Morgan 2	$(A + B)' = A' \cdot B'$

De Morgan's theorem in plain words: to remove a bar that covers a whole expression, put a bar on each variable separately AND swap the sign (· becomes +, + becomes ·).

MEMORY TRICK

De Morgan in four words: “break the bar, change the sign”.

SOLVED EXAMPLE

Check De Morgan with A = 1, B = 0

1. Left side $(A \cdot B)'$: $A \cdot B = 1 \cdot 0 = 0$, then NOT → 1.
2. Right side $A' + B'$: $A' = 0$, $B' = 1$, so $0 + 1 = 1$.
3. Both sides give 1 — the rule works (try the other three combinations yourself).

✅ $(A \cdot B)' = A' + B'$ ✓

➖ SOLVED EXAMPLE

Simplify A·B + A·B′

1. Both terms contain A, so take A out (like normal algebra): A · (B + B′).
2. B + B′ means "B or not-B" — one of them is always 1, so B + B′ = 1 (complement rule).
3. A · 1 = A (identity rule).

✅ A·B + A·B′ = A

⚠ EXAM TRAP

In Boolean algebra 1 + 1 = 1 (it means "true OR true = true"), not 2. Don't mix it with binary addition, where 1 + 1 = 10.

⏱ RECAP IN 30 SECONDS

- ✓ AND = all 1s needed; OR = any 1 is enough; NOT = opposite
- ✓ NAND/NOR = AND/OR flipped; both are universal gates
- ✓ XOR = different → 1; XNOR = same → 1
- ✓ De Morgan: break the bar, change the sign

✍ Quick quiz

Q1. (A + B)′ equals:

(A) A · B
(B) A′ + B′
(C) A′ · B′
(D) A + B

Q2. Which gate gives 1 only when its inputs differ?

(A) XOR
(B) XNOR
(C) NOR
(D) AND

Q3. A + A·B simplifies to:

(A) B
(B) A
(C) 1
(D) A·B

Q4. A NAND gate with inputs 1 and 1 outputs:

(A) Depends on the clock
(B) 0
(C) 1
(D) Undefined

Answers: Q1 C — Break the bar, change + to ·. · Q2 A — XOR = "are they different?". · Q3 B — Absorption: A(1 + B) = A · 1 = A. · Q4 B — AND gives 1, NAND flips it to 0.

LESSON 1.5 · 15 MIN

K-map: simplifying with a picture

In one line: **A K-map lays out a truth table as a grid where neighbouring boxes differ by only one variable — grouping 1s lets you cancel that variable.**

Why do we need it? Boolean expressions often come out long, and a long expression means more gates, more cost and more power. A **Karnaugh map (K-map)** is a picture method that finds the shortest expression without doing algebra.

Two words first. A **minterm** is one row of the truth table where the output is 1. "F = Σm(0, 1, 4, 5)" simply means "the output is 1 in rows 0, 1, 4 and 5" (row numbers are the inputs read as a binary number).

Step 1 — draw the grid. A 3-variable function (A, B, C) has 8 rows, so the grid has 8 boxes: 2 rows for A (0, 1) and 4 columns for BC. The columns are written in the special order **00, 01, 11, 10** — notice that from one column to the next only ONE bit changes. (This order is called Gray code.)

	BC = 00	BC = 01	BC = 11	BC = 10
A = 0	m0	m1	m3	m2
A = 1	m4	m5	m7	m6

Which minterm sits in which box (m0 = row 0 of the truth table, etc.)

Step 2 — put a 1 in the boxes of the given minterms, and 0 everywhere else. For F = Σm(0, 1, 4, 5):

	BC = 00	BC = 01	BC = 11	BC = 10
A = 0	1	1	0	0
A = 1	1	1	0	0

F = Σm(0, 1, 4, 5)

Step 3 — circle groups of 1s. Rules: a group must be a rectangle of **1, 2, 4 or 8** boxes (powers of 2); make each group **as big as possible**; use **as few groups as possible**; every 1 must be in at least one group; boxes on opposite edges count as neighbours (the map wraps around like a tube).

Step 4 — read each group. Look at each variable across the boxes of the group: if it **changes** (is 0 in some boxes and 1 in others), it disappears. If it **stays the same**, keep it — as the plain letter if it stays 1, as the letter with a dash (') if it stays 0.

➖ SOLVED EXAMPLE

Simplify F = Σm(0, 1, 4, 5)

1. The four 1s form one 2 × 2 block (columns BC = 00 and 01, both rows). That's a group of 4 — the biggest possible.
2. Check A: it is 0 in the top row and 1 in the bottom row → A changes → A disappears.
3. Check C: it is 0 in column 00 and 1 in column 01 → C changes → C disappears.
4. Check B: it is 0 in both columns (00 and 01) → B stays 0 → keep it as B′.

✅ F = B′ (instead of four long terms!)

📐 FORMULAS

Group of 2 boxes **`removes 1 variable`**

Group of 4 boxes **`removes 2 variables`**

Group of 8 boxes **`removes 3 variables`**

Group of 2^k boxes **`removes k variables`**

Don't-care conditions (X): boxes whose output never matters. You may treat an X as 1 if it helps you make a bigger group, or ignore it otherwise.

⚠ EXAM TRAP

The most common mistake is labelling the columns 00, 01, 10, 11 (normal counting order). Then neighbouring boxes differ by two bits and the grouping goes wrong. Always use 00, 01, 11, 10.

🧠 MEMORY TRICK

Remember the column order as "zero-zero, zero-one, one-one, one-zero" — the second bit goes 0 1 1 0, like stepping up and back down.

⏱ RECAP IN 30 SECONDS

- ✓ Columns/rows in Gray order: 00, 01, 11, 10
- ✓ Groups of 1, 2, 4, 8 — as big and as few as possible; edges wrap
- ✓ Inside a group, a variable that changes disappears
- ✓ A group of 2^k boxes removes k variables

✍️ **Quick quiz**

Q1. In a 4-variable K-map, a group of 4 cells removes how many variables?

(A) 4 (B) 1
(C) 3 (D) 2

Q2. K-map columns for two variables are ordered:

(A) 00, 01, 10, 11 (B) 11, 10, 01, 00
(C) 00, 01, 11, 10 (D) 00, 10, 01, 11

Q3. How many boxes does a 4-variable K-map have?

(A) 8 (B) 4
(C) 16 (D) 32

Answers: Q1 D — 4 = 2^2 → removes 2 variables. · Q2 C — Gray-code order keeps neighbours one bit apart. · Q3 C — 2^4 = 16 input combinations.

LESSON 1.6 · 16 MIN

Circuits that calculate & circuits that remember

In one line: **Combinational circuits only look at the inputs right now; sequential circuits also remember what happened before.**

Gates are joined together to make useful circuits. There are two families, and the difference is **memory**.

> 💡 **THINK OF IT LIKE THIS**
> A calculator's "+" is combinational: type 2 + 3 and it shows 5, every time, whatever you did before. A turnstile counter at a metro station is sequential: its display depends on how many people have passed BEFORE — it remembers.

A. Combinational circuits (no memory).

Half adder — adds two bits. Remember from binary addition: 1 + 1 = 0 carry 1. Look at the pattern: the sum bit is 1 when the inputs are different → that is **XOR**. The carry is 1 only when both are 1 → that is **AND**. So Sum = A ⊕ B and Carry = A · B.

Full adder — adds THREE bits: A, B and the carry coming from the previous column. Sum = A ⊕ B ⊕ Cin. A full adder can be made from two half adders and one OR gate. Many full adders in a row make an n-bit adder.

Multiplexer (MUX) — a data selector. It has many data inputs, ONE output, and some **select lines** that choose which input reaches the output.

> 💡 **THINK OF IT LIKE THIS**
> A MUX is a TV remote: many channels (inputs), one screen (output), and the channel number you press (select lines) decides what you see.

➖ SOLVED EXAMPLE

How many select lines does an 8 : 1 MUX need?

1. The select lines form a binary number that names the input to pass through.
2. With 2 lines you can make $2^2 = 4$ numbers — not enough for 8 inputs.
3. With 3 lines you can make $2^3 = 8$ numbers (000 to 111) — exactly enough.

✅ **3 select lines (rule: 2^n inputs need n select lines)**

Circuit	Inputs → outputs	Think of it as
Multiplexer	Many → 1 (chosen by select lines)	TV remote
Demultiplexer	1 → many	A postman delivering one letter to one of many houses
Decoder	n → 2^n (exactly one output ON)	A 3-to-8 decoder turns a 3-bit number into one of 8 lights
Encoder	2^n → n	The reverse: which of 8 buttons was pressed → 3-bit number

B. Sequential circuits (with memory). The basic memory cell is the **flip-flop**: it stores exactly **one bit**, and changes only when a **clock** signal "ticks" (the clock is a steady on-off pulse that keeps everything in step).

Flip-flop	Inputs	What it does	Remember
SR (Set–Reset)	S, R	S = 1 stores 1; R = 1 stores 0; both 0 → keep	S = R = 1 is NOT allowed (invalid)
JK	J, K	Like SR, but J = K = 1 makes it TOGGLE (flip 0↔1)	It fixes SR's invalid case
D (Data / Delay)	D	Output becomes whatever D is at the clock tick	Used in registers
T (Toggle)	T	T = 1 flips the stored bit, T = 0 keeps it	Used in counters

Counters are flip-flops connected so that the stored number goes up (or down) by 1 on every clock tick. Each flip-flop holds one bit, so n flip-flops can count through **2^n** different states.

➖ SOLVED EXAMPLE

How many flip-flops are needed to count from 0 to 9 (a mod-10 counter)?

1. We need 10 different states.
2. 3 flip-flops give $2^3 = 8$ states — too few.
3. 4 flip-flops give $2^4 = 16$ states — enough (we stop and reset after 9).

✅ **4 flip-flops**

- Ripple (asynchronous) counter: each flip-flop is triggered by the one before it — simple but slow, because the change "ripples" through
- Synchronous counter: all flip-flops use the same clock — faster
- Register: a row of flip-flops holding a whole word (e.g. 8 bits); a shift register moves the bits left or right on each tick
- Race-around condition: a JK flip-flop with J = K = 1 can toggle many times during one long clock pulse — fixed by edge triggering or a master–slave design

⚠ EXAM TRAP

To count up to N states you need the smallest n with $2^n \geq N$ — not N flip-flops. A mod-10 counter needs 4 flip-flops, a mod-12 also needs 4, a mod-20 needs 5.

⏱ RECAP IN 30 SECONDS

- ✓ Combinational = no memory (adders, MUX, decoder); sequential = memory (flip-flops, counters, registers)
- ✓ Half adder: Sum = XOR, Carry = AND
- ✓ MUX with 2^n inputs needs n select lines
- ✓ SR has an invalid state (S = R = 1); JK toggles at J = K = 1
- ✓ n flip-flops → 2^n states

✍️ **Quick quiz**

Q1. A 16 : 1 multiplexer needs how many select lines?

(A) 4 (B) 3
(C) 2 (D) 16

Q2. Which flip-flop has an invalid input combination?

(A) T (B) D
(C) JK (D) SR

Q3. Minimum flip-flops for a mod-12 counter:

(A) 6 (B) 3
(C) 12 (D) 4

Q4. The carry output of a half adder is produced by which gate?

(A) OR (B) NOT
(C) XOR (D) AND

Answers: Q1 A — $2^4 = 16$. · Q2 D — S = R = 1 is not allowed in an SR flip-flop. · Q3 D — $2^4 = 16 \geq 12$, but $2^3 = 8$ is too few. · Q4 D — Carry is 1 only when both bits are 1.

MODULE 2

Programming in C & Python

From “what is a program?” to predicting tricky outputs — the biggest scoring area, explained from zero.

LESSON 2.1 · 12 MIN

What is a program? Algorithms, flowcharts & translators

In one line: **An algorithm is a recipe; a program is that recipe written in a language the computer can follow; a translator turns it into 0s and 1s.**

What is a program? A computer can only follow instructions — it cannot think. A **program** is a list of exact instructions written for the computer, for example “take two numbers, add them, show the answer”.

Before writing a program we first plan the steps. That plan is called an **algorithm**.

> 💡 **THINK OF IT LIKE THIS**
>
> A recipe for making tea is an algorithm: 1) boil water, 2) add tea leaves, 3) add milk and sugar, 4) boil for 2 minutes, 5) strain and serve. Each step is clear, the steps are in order, and the recipe ends. A computer algorithm is the same, only for data.

Property of a good algorithm	Meaning
Input	Takes zero or more inputs (e.g. two numbers)
Output	Gives at least one result
Definiteness	Every step is clear — no confusion about what to do
Finiteness	It must stop after a limited number of steps
Effectiveness	Every step is simple enough to actually be done

Flowchart = an algorithm drawn as a picture with standard shapes joined by arrows. Exams ask which shape means what:

Shape	Used for	Example
Oval (rounded)	Start / End	START, STOP
Parallelogram (slanted box)	Input / Output	Read A, B / Print SUM
Rectangle	Process (a calculation)	SUM = A + B
Diamond	Decision (yes/no question)	Is A > B ?
Arrow	Flow — which step comes next	→

➖ SOLVED EXAMPLE

Algorithm: find the bigger of two numbers

1. Start.
2. Read two numbers A and B.
3. Ask: is A > B? (this is the diamond in a flowchart)
4. If yes → print A. If no → print B.
5. Stop.

✅ **5 clear steps — this is the algorithm; written in C or Python it becomes a program**

Pseudocode = the algorithm written in plain, structured English (IF … THEN … ELSE, WHILE …), not in any real programming language. It's used to plan before coding.

Why do we need a translator? The CPU only understands **machine language** (0s and 1s). We write in languages people can read (C, Python, Java). A **translator** converts our code into machine language. There are three kinds:

Translator	Converts	How
Assembler	Assembly language (MOV, ADD…) → machine code	One instruction at a time
Compiler	High-level language (C, C++) → machine code	The WHOLE program at once, makes an .exe file, then you run it
Interpreter	High-level language (Python, JavaScript) → runs directly	Line by line, stops at the first error

💡 THINK OF IT LIKE THIS

A compiler is like translating a whole book into Hindi first and then reading it — slow to prepare, fast to read again and again. An interpreter is a live translator at a meeting who translates sentence by sentence — no waiting, but slower overall.

	Compiler	Interpreter
Translates	Whole program at once	One line at a time
Creates a separate file?	Yes (object/executable)	No
Errors shown	All together, after compiling	Stops at the first error
Running speed	Faster	Slower
Examples	C, C++	Python, JavaScript, BASIC

After compiling: the **linker** joins your program with library code (like the code for printf) into one executable, and the **loader** copies that executable into memory so it can run.

Three kinds of errors: syntax error — breaking the grammar of the language (missing semicolon) → caught by the compiler; **run-time error** — happens while running (dividing by zero); **logical error** — the program runs but gives the wrong answer (you wrote + instead of −) → nobody catches it except you.

⏱ **RECAP IN 30 SECONDS**

- ✓ Algorithm = clear, finite steps; program = algorithm in a programming language
- ✓ Flowchart: oval start/end, parallelogram input/output, rectangle process, diamond decision
- ✓ Compiler: whole program, faster, C/C++; interpreter: line by line, Python
- ✓ Assembler = assembly → machine code; linker joins files; loader loads into memory
- ✓ Syntax errors are caught by the compiler; logical errors are not

✍ **Quick quiz**

Q1. Which flowchart symbol represents a decision?

(A) Parallelogram (B) Oval
(C) Rectangle (D) Diamond

Q2. Which program combines object files and libraries into one executable?

(A) Linker (B) Interpreter
(C) Loader (D) Assembler

Q3. An algorithm must end after a limited number of steps. This property is:

(A) Effectiveness (B) Definiteness
(C) Finiteness (D) Input

Answers: Q1 D — A diamond has yes/no branches. · Q2 A — The linker links; the loader loads it into memory. · Q3 C — Finite = it stops.

LESSON 2.2 · 15 MIN

C basics: variables, data types & operators

In one line: **A variable is a labelled box in memory; its data type says what kind of value fits in the box; operators do the maths.**

Your first C program, line by line:

```c
#include <stdio.h>      // bring in input/output functions like printf

int main() {            // every C program starts running from main()
    int age = 20;       // make a box called age, put 20 in it
    printf("Age is %d", age);   // print it; %d means "an integer goes here"
    return 0;           // tell the system: finished successfully
}
```

```
Output ▸ Age is 20
```

What is a variable? A named box in memory that stores a value. `int age = 20;` means: make a box named **age**, it can hold an **int** (whole number), put **20** in it. The value can change later (`age = 21;`).

What is a data type? It tells the computer what kind of value goes in the box and how much memory to give it.

Data type	Stores	Typical size	printf code	Example
char	One character	1 byte	%c	char g = 'A';
int	Whole number	4 bytes (2 on old 16-bit compilers)	%d	int n = -15;
float	Decimal number (about 6–7 digits)	4 bytes	%f	float p = 3.14;
double	Decimal number (about 15 digits)	8 bytes	%lf	double d = 3.14159265;

Arithmetic operators: + − * / and **%** (remainder, called modulus). The tricky one is **division of two integers**: C throws away the decimal part.

Expression	Result	Why
7 / 2	3	Both are integers → decimal part dropped
7.0 / 2	3.5	One is a decimal → real division
7 % 2	1	7 = 2 × 3 + 1, remainder 1
17 % 5	2	17 = 5 × 3 + 2
2 % 7	2	7 doesn't fit into 2 even once → remainder is 2

Relational operators compare and give 1 (true) or 0 (false): `< > <= >= == !=`. **Logical operators** combine conditions: `&&` (AND — both true), `||` (OR — at least one true), `!` (NOT). In C, **0 means false and any other number means true**.

Increment and decrement: `i++` and `++i` both add 1 to i. The difference is only WHEN the increase happens if you use the value in the same line:

Written	Name	What happens
++i	Pre-increment	Increase FIRST, then use the new value
i++	Post-increment	Use the OLD value first, increase afterwards

➖ SOLVED EXAMPLE

Trace: int i = 5; int a = i++; int b = ++i;

1. Start: i = 5.
2. a = i++ → post-increment: a gets the OLD value 5, then i becomes 6.
3. b = ++i → pre-increment: i becomes 7 FIRST, then b gets 7.
4. Final values: a = 5, b = 7, i = 7.

✅ printf("%d %d %d", a, b, i) prints 5 7 7

Operator precedence = which operator is done first when there are several. Like BODMAS in maths:

Priority (high → low)	Operators
1	( ) brackets
2	++ -- ! (unary)
3	* / %
4	+ −
5	< <= > >=
6	== !=
7	&&
8	||
9	?: (conditional)
10	= += −= ... (assignment, done last)

➖ SOLVED EXAMPLE

What is 2 + 3 * 4 % 5 ?

1. * and % have the same priority and go left to right: 3 * 4 = 12.
2. 12 % 5 = 2.
3. Then + : 2 + 2 = 4.

✅ 4

C

```c
int x = 0;
if (x = 5)              // = is ASSIGNMENT, not comparison!
    printf("yes");
else
    printf("no");
```

Output ▸ yes

x = 5 stores 5 in x; the value of the expression is 5, which is non-zero → true. To compare you must write x == 5. This is a favourite exam trap.

⚠ EXAM TRAP

= puts a value into a variable; == compares two values. `if (x = 0)` is always false (it stores 0), `if (x = 5)` is always true.

⏱ **RECAP IN 30 SECONDS**

- ✓ Variable = named box; data type = what fits (char 1 B, int 4 B, float 4 B, double 8 B)
- ✓ Integer division drops the decimal: 7 / 2 = 3; % gives the remainder
- ✓ ++i: increase then use; i++: use then increase
- ✓ Precedence: brackets → unary → * / % → + − → comparisons → && → || → =
- ✓ = assigns, == compares; in C, 0 is false and any non-zero value is true

✍ **Quick quiz**

Q1. int i = 3; printf("%d", i++ + 2); prints:

(A) Error
(B) 5
(C) 3
(D) 6

Q2. In C, 17 % 5 equals:

(A) 1
(B) 3
(C) 2
(D) 3.4

Q3. In C, 9 / 2 evaluates to:

(A) 5
(B) 4
(C) 4.5
(D) 4.0

Answers: Q1 B — Post-increment uses the old value 3: 3 + 2 = 5 (i becomes 4 afterwards). · Q2 C — 17 = 5 × 3 + 2. · Q3 B — Both are integers, so the decimal part is dropped.

LESSON 2.3 · 15 MIN

Decisions & loops: if, switch, for, while, do-while

In one line: **if/switch let a program choose; loops let it repeat — and tracing a loop on paper is how you answer output questions.**

Making decisions with if–else. The program checks a condition; if it is true one block runs, otherwise the else block runs.

C

```c
int marks = 72;
if (marks >= 90)      printf("A");
else if (marks >= 60) printf("B");
else                  printf("C");
```

Output ▸ B

Conditions are checked top to bottom; the FIRST true one runs and the rest are skipped.

switch picks one case by matching a value. Each case should end with `break`, otherwise the program "falls through" into the next case.

```c
int day = 2;
switch (day) {
    case 1: printf("Mon ");
    case 2: printf("Tue ");
    case 3: printf("Wed ");
            break;
    default: printf("Other");
}
```

Output ▸ Tue Wed

case 2 has no break, so it falls through and also prints Wed, then stops at break.

Loops repeat a block of code. C has three:

Loop	Checks the condition	Runs at least once?	Best when
for	Before each round	No	You know how many times (e.g. 10 times)
while	Before each round (entry-controlled)	No	Repeat until something happens
do ... while	After each round (exit-controlled)	YES — always once	Menus: show at least once

How a for loop works: `for (start; condition; update)`. Step 1: run start once. Step 2: check condition — if false, leave. Step 3: run the body. Step 4: run update. Go back to step 2.

➖ SOLVED EXAMPLE

Trace: for (i = 1; i <= 10; i = i + 3) printf("%d ", i);

1. Start: i = 1. Check 1 <= 10 → true → print 1. Update: i = 4.
2. Check 4 <= 10 → true → print 4. Update: i = 7.
3. Check 7 <= 10 → true → print 7. Update: i = 10.
4. Check 10 <= 10 → true → print 10. Update: i = 13.
5. Check 13 <= 10 → false → loop ends.

✅ Output: 1 4 7 10 (the loop body ran 4 times)

C

```c
int i;
for (i = 0; i < 3; i++);     // <-- semicolon here!
    printf("Hi");
```

Output ▸ Hi

The ; ends the loop with an EMPTY body. The loop runs 3 times doing nothing; then printf runs once.

C

```c
int n = 10;
do {
    printf("%d", n);
} while (n < 5);
```

Output ▸ 10

The condition is false from the start, but do-while still runs the body once.

break jumps out of the loop immediately. **continue** skips the rest of this round and goes to the next round.

C

```c
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;   // skip 3
    if (i == 5) break;      // stop at 5
    printf("%d ", i);
}
```

Output ▸ 1 2 4

🧠 **MEMORY TRICK**

For every loop question make a tiny table on paper with columns: i | condition true? | output. Fill one row per round. You will never miss an output again.

⚠ **EXAM TRAP**

Count loop runs carefully: for (i = 0; i < n; i++) runs n times; for (i = 0; i <= n; i++) runs n + 1 times; for (i = 1; i <= n; i++) runs n times.

⏱ **RECAP IN 30 SECONDS**

- ✓ if–else if–else: the first true condition wins
- ✓ switch without break falls through to the next case
- ✓ for/while check first (may run 0 times); do-while checks last (runs at least once)
- ✓ break leaves the loop; continue jumps to the next round
- ✓ A ; right after for(...) makes an empty loop

✍️ **Quick quiz**

Q1. Which loop always executes its body at least once?

(A) while
(B) do...while
(C) None
(D) for

Q2. How many times does for (i = 0; i <= 5; i++) run?

(A) 4
(B) 5
(C) 6
(D) Infinite

Q3. A switch case without break causes:

(A) Fall-through into the next case
(B) A compile error
(C) The default case only
(D) An infinite loop

Answers: Q1 B — The condition is checked after the body. · Q2 C — i = 0, 1, 2, 3, 4, 5 → 6 times. · Q3 A — Execution continues into the following case.

LESSON 2.4 · 14 MIN

Functions & recursion

In one line: **A function is a reusable mini-program with a name; recursion is a function that solves a problem by calling a smaller copy of itself.**

Why functions? If you need to calculate an area in 10 places, you write the formula once inside a function and just call it 10 times. Less code, fewer mistakes.

```c
int square(int n) {      // return type, name, parameter
    return n * n;        // send the answer back
}

int main() {
    int r = square(6);   // call it; 6 goes into n
    printf("%d", r);
}
```

Output ▸ 36

Word	Meaning
Parameter	The variable in the function definition (n)
Argument	The actual value passed when calling (6)
Return type	Type of the answer sent back (int); void = nothing returned
Prototype / declaration	Telling the compiler about a function before it's used: int square(int);

Call by value (the C default): the function receives a COPY of the value. Changing the copy does NOT change the original. **Call by reference**: the function receives the ADDRESS (a pointer), so it can change the original.

```c
void change(int x) { x = 100; }

int main() {
    int a = 5;
    change(a);
    printf("%d", a);
}
```

Output ▸ 5

change() modified its own copy x; a is untouched. To really change a, pass &a and use a pointer.

Recursion — a function that calls itself. Every recursive function needs two parts: a **base case** (a small problem answered directly, which stops the calls) and a **recursive case** (solve a smaller version by calling itself).

💡 **THINK OF IT LIKE THIS**

Russian nesting dolls: to reach the smallest doll you open a doll, which contains a smaller doll, which contains a smaller one… until the smallest doll cannot be opened (the base case).

C

```c
int fact(int n) {
    if (n <= 1) return 1;          // base case: 1! = 1
    return n * fact(n - 1);        // recursive case
}
```

Output ▸ `fact(4) = 24`

➖ **SOLVED EXAMPLE**

Trace fact(4) step by step

1. fact(4) cannot answer directly, so it asks: 4 × fact(3). It waits.
2. fact(3) asks 3 × fact(2). It waits.
3. fact(2) asks 2 × fact(1). It waits.
4. fact(1) hits the base case → returns 1.
5. Now the waiting calls finish in reverse: fact(2) = 2 × 1 = 2, fact(3) = 3 × 2 = 6, fact(4) = 4 × 6 = 24.

✅ fact(4) = 24. The waiting calls are kept on the call STACK — last called, first finished.

⚠ **EXAM TRAP**

Without a base case the function calls itself forever, the call stack fills up, and the program crashes with a stack overflow.

Recursion vs loops (iteration): anything done with recursion can be done with a loop. Recursion is shorter and natural for trees and divide-and-conquer, but uses more memory (each call sits on the stack).

⏱ **RECAP IN 30 SECONDS**

- ✓ Function = named reusable block; parameter in definition, argument in the call
- ✓ Call by value passes a copy (original unchanged); call by reference passes the address
- ✓ Recursion needs a base case and a smaller recursive call
- ✓ Pending recursive calls live on the stack; no base case → stack overflow

✍️ **Quick quiz**

Q1. Recursion without a base case results in:

(A) Zero output
(B) Faster code
(C) A compile error
(D) Stack overflow

Q2. In call by value, changes made to a parameter inside the function:

(A) Change the caller's variable
(B) Do not affect the caller's variable
(C) Change only global variables
(D) Cause an error

Q3. Which data structure does the system use to manage function calls?

(A) Stack
(B) Queue
(C) Tree
(D) Array of strings

Answers: Q1 D — Calls keep piling up on the call stack. · Q2 B — The function works on a copy. · Q3 A — The last call made is the first to finish.

LESSON 2.5 · 15 MIN

Arrays, strings & pointers in C

In one line: **An array is a row of boxes under one name; a pointer is a variable that stores the address of a box.**

Array = many values of the same type stored side by side, under one name. `int marks[5];` makes 5 boxes: marks[0] … marks[4]. Counting starts at **0**, so the last box of an array of size 5 is index **4**.

```
index:    0     1     2     3     4
        +-----+-----+-----+-----+-----+
marks:  | 72  | 85  | 60  | 91  | 48  |
        +-----+-----+-----+-----+-----+
address: 2000  2004  2008  2012  2016   (int = 4 bytes each)
```

Elements are stored next to each other in memory

📐 **FORMULAS**

Address of a[i] **base address + i × (size of one element)**

➡ **SOLVED EXAMPLE**

int a[5] starts at address 2000; int is 4 bytes. Where is a[3]?

1. Use the formula: 2000 + 3 × 4.
2. 3 × 4 = 12.
3. 2000 + 12 = 2012.

✅ a[3] is at address 2012

Strings in C are character arrays that end with a special invisible character **'\0'** (null). "hello" has 5 letters but needs **6** bytes. `strlen("hello")` = 5 (doesn't count '\0'); `sizeof` of that array = 6.

What is a pointer? Every box in memory has an address (like a house number). A **pointer** is a variable that stores an address.

> 💡 **THINK OF IT LIKE THIS**
> A variable is a house; its value is the family living inside; a pointer is a slip of paper with the house number written on it. & means "what is the house number of…?"; * means "go to that house and look inside".

C

```c
int x = 10;
int *p = &x;    // p stores the ADDRESS of x
printf("%d", *p);   // *p = value at that address = 10
*p = 20;        // change the value at that address
printf("%d", x);    // x is now 20
```

Output ▸ 10 then 20

Expression	Means
x	Value of x (10)
&x	Address of x (e.g. 5000)
p	The address stored in p (5000)
*p	Value at that address (10)
&p	Address of p itself

Pointers and arrays are closely linked. The name of an array is the address of its first element: `a` means `&a[0]`, and `a[i]` is the same as `*(a + i)`.

Pointer arithmetic: adding 1 to a pointer moves it to the NEXT ELEMENT, not the next byte. If p is an int pointer at 1000 and int is 4 bytes, then p + 1 = 1004 and p + 3 = 1012.

Swap with pointers (call by reference): to really swap two variables, pass their addresses:

```c
void swap(int *a, int *b) {
    int t = *a;  *a = *b;  *b = t;
}
// call: swap(&x, &y);
```

Output ▸ x and y are exchanged

⚠ EXAM TRAP

When an array is passed to a function it becomes a pointer, so sizeof(arr) inside the function gives the size of a pointer (4 or 8), not of the whole array.

⏱ RECAP IN 30 SECONDS

- ✓ Array indexes start at 0; a[i] address = base + i × size
- ✓ C strings end with '\0': "hello" needs 6 bytes, strlen = 5
- ✓ &x = address of x; *p = value at the address stored in p
- ✓ a[i] == *(a + i); p + 1 moves by one element (e.g. 4 bytes for int)
- ✓ Pass addresses (&x) to let a function change the caller's variables

✍ **Quick quiz**

Q1. If int a[5] starts at address 2000 and int is 4 bytes, the address of a[3] is:

(A) 2016 (B) 2003
(C) 2012 (D) 2006

Q2. int x = 7; int *p = &x; What is *p?

(A) The address of p (B) 7
(C) 0 (D) The address of x

Q3. How many bytes does the string "EXAM" occupy in C?

(A) 8 (B) 3
(C) 4 (D) 5

Answers: Q1 C — 2000 + 3 × 4 = 2012. · Q2 B — *p reads the value at the address p holds. · Q3 D — 4 letters + the '\0' terminator.

LESSON 2.6 · 13 MIN

Python basics: variables, types & operators

In one line: Python is like plain English: no semicolons, no data-type declarations, and indentation (spaces) decides which lines belong together.

PYTHON

```
name = "Asha"          # no type needed — Python works it out
age = 21
print("Hello", name, "you are", age)
```

Output ▸ Hello Asha you are 21

Big differences from C: no `;` at the end of lines, no `{ }` — blocks are shown by **indentation** (usually 4 spaces), and you don't write the data type: Python decides it from the value (this is called **dynamic typing**).

PYTHON

```
marks = 75
if marks >= 60:
    print("Pass")       # indented → inside the if
else:
    print("Fail")
print("Done")           # not indented → always runs
```

Output ▸ Pass
Done

Type	Example	type() shows
int	42	<class 'int'>
float	3.5	<class 'float'>
str (text)	'hi' or "hi"	<class 'str'>
bool	True / False	<class 'bool'>
list	[1, 2, 3]	<class 'list'>
tuple	(1, 2, 3)	<class 'tuple'>
dict	{'a': 1}	<class 'dict'>
NoneType	None	<class 'NoneType'>

Division in Python is different from C — this is asked very often:

Operator	Name	Example	Result
/	True division (always a float)	7 / 2	3.5
//	Floor division (round DOWN)	7 // 2	3
//	...with a negative number	-7 // 2	-4 (−3.5 rounded DOWN is −4)
%	Remainder	7 % 2	1
	Power	2 ** 3	8

⚠ EXAM TRAP

-7 // 2 is −4, not −3. Floor means "round towards minus infinity", so −3.5 goes DOWN to −4. Also, 6 / 2 is 3.0 (a float), not 3.

Logical operators are words: `and`, `or`, `not` (instead of &&, ||, !). Comparisons can be chained: `1 < x < 10`.

Input: `input()` ALWAYS returns text (a string). To do maths, convert it: `n = int(input("Enter: "))`.

Loops in Python: `for i in range(...)` and `while condition:`.

range(...)	Gives	Rule
range(5)	0 1 2 3 4	Starts at 0, stops BEFORE 5
range(2, 6)	2 3 4 5	Stops before 6
range(1, 10, 3)	1 4 7	Step of 3
range(5, 0, -1)	5 4 3 2 1	Counting down

PYTHON

```
total = 0
for i in range(1, 5):
    total = total + i
print(total)
```

Output ▸ 10

i takes 1, 2, 3, 4 (5 is excluded): 1 + 2 + 3 + 4 = 10.

⏱ **RECAP IN 30 SECONDS**

- ✓ Indentation makes blocks; no ; or { }; types are decided automatically
- ✓ / always gives a float; // rounds down (−7 // 2 = −4); ** is power
- ✓ and, or, not are the logical operators
- ✓ input() returns a string — use int() for numbers
- ✓ range(a, b) stops BEFORE b

✍ **Quick quiz**

Q1. print(10 / 4) in Python outputs:

(A) 2.0 (B) 3
(C) 2 (D) 2.5

Q2. list(range(2, 8, 2)) is:

(A) [2, 4, 6, 8] (B) [2, 4, 6]
(C) [2, 3, 4, 5, 6, 7] (D) [4, 6, 8]

Q3. The value of 2 ** 3 ** 2 in Python is:

(A) 512 (B) 36
(C) 64 (D) 12

Answers: Q1 D — / is true division. · Q2 B — Start 2, step 2, stop before 8. · Q3 A — ** is right-associative: 3 ** 2 = 9, then 2 ** 9 = 512.

LESSON 2.7 · 16 MIN

Python strings, lists, tuples, dictionaries & errors

In one line: **Know three things — slicing, which types can change (mutable) and how errors are handled — and most Python output questions become easy.**

Indexing: every character of a string (or item of a list) has a position number. Counting from the left starts at **0**; counting from the right starts at **−1**.

```
s = "COMPUTER"

character:   C   O   M   P   U   T   E   R
index:       0   1   2   3   4   5   6   7
from right: -8  -7  -6  -5  -4  -3  -2  -1
```

Always write this table first for any slicing question

Slicing `s[start : stop : step]` takes a part. The character at **start is included**; the one at **stop is NOT included**. Leave start empty = from the beginning; leave stop empty = to the end.

Slice	Result	How to read it
s[1:4]	OMP	positions 1, 2, 3 (stop 4 is excluded)
s[:3]	COM	from the start up to position 2
s[5:]	TER	from position 5 to the end
s[-3:]	TER	last three characters
s[::2]	CMUE	every 2nd character: positions 0, 2, 4, 6
s[::-1]	RETUPMOC	step −1 walks backwards → reversed string

Mutable vs immutable — can you change it after creating it?

Type	Brackets	Can change? (mutable)	Notes
list	[ ]	Yes	append(), insert(), remove(), pop(), sort()
tuple	( )	No	Faster; good for fixed data
str	' ' / " "	No	s[0] = 'X' gives an error
dict	{key: value}	Yes	Keys must be unique and immutable
set	{ }	Yes	No duplicates, no order: {1, 2, 2} → {1, 2}

PYTHON

```
a = [1, 2, 3]
b = a          # b is NOT a copy — it is a second name for the same list
b.append(4)
print(a)
```

Output ▸ `[1, 2, 3, 4]`

Both names point to one list. To make a real copy use b = a.copy() or b = a[:].

PYTHON

```
nums = [10, 20, 30]
nums.append(40)      # add at the end    → [10, 20, 30, 40]
nums.insert(1, 15)   # add at index 1     → [10, 15, 20, 30, 40]
nums.pop()           # remove the last    → [10, 15, 20, 30]
print(len(nums), nums[-1])
```

Output ▸ `4 30`

PYTHON

```
student = {"name": "Ravi", "marks": 82}
student["marks"] = 90        # change a value
student["city"] = "Patna"     # add a new key
print(student["marks"], len(student))
```

Output ▸ 90 3

- len([1, [2, 3], 4]) is 3 — the inner list counts as ONE item
- "ab" * 3 → "ababab"; [0] * 3 → [0, 0, 0]; "ab" + "cd" → "abcd"
- "py" in "python" → True (membership test)
- Useful string methods: upper(), lower(), split(), strip(), replace(), find()

Handling errors (exceptions). When something goes wrong while running (dividing by zero, opening a missing file), Python raises an exception. You can catch it so the program doesn't crash:

PYTHON

```
try:
    x = 10 / 0                 # this raises an error
except ZeroDivisionError:
    print('cannot divide by zero')
else:
    print('no error')          # runs only if NO error happened
finally:
    print('done')              # ALWAYS runs
```

Output ▸ cannot divide by zero
done

Exception	When it happens
ZeroDivisionError	Dividing by 0
ValueError	int("abc") — right type, wrong value
TypeError	"5" + 3 — wrong types together
IndexError	list index out of range
KeyError	dictionary key not found
NameError	using a variable that doesn't exist
FileNotFoundError	opening a file that isn't there

Files: `open("data.txt", "r")` reads, `"w"` writes (erases old content), `"a"` appends. The `with` statement closes the file automatically: `with open("data.txt") as f: text = f.read()`.

🧠 **MEMORY TRICK**

For slicing questions: 1) write the index table, 2) start is IN, stop is OUT, 3) for negative steps, walk right-to-left.

⏱ **RECAP IN 30 SECONDS**

- ✓ Indexes start at 0 from the left and −1 from the right
- ✓ s[start:stop:step]: start included, stop excluded; s[::-1] reverses
- ✓ list and dict can change; tuple and str cannot
- ✓ b = a shares the same list; use a.copy() for a real copy
- ✓ try / except / else / finally — finally always runs

✍️ **Quick quiz**

Q1. print(-7 // 2) in Python outputs:

(A) 3
(B) -3.5
(C) -4
(D) -3

Q2. s = 'PYTHON'; print(s[1:4]) outputs:

(A) YTHO
(B) YTH
(C) PYT
(D) THO

Q3. Which of these is immutable?

(A) dict
(B) list
(C) tuple
(D) set

Q4. int("abc") raises:

(A) ValueError
(B) TypeError
(C) KeyError
(D) NameError

Answers: Q1 C — Floor division rounds down towards −∞: −3.5 → −4. · Q2 B — Positions 1, 2, 3 → Y T H (4 is excluded). · Q3 C — Tuples cannot be changed after creation. · Q4 A — The type (string) is allowed, but this value can't become an int.

MODULE 3

Data structures

Different ways of arranging data, and why each one makes some job fast.

LESSON 3.1 · 14 MIN

What is a data structure? Arrays vs linked lists

In one line: **An array is a row of numbered seats (jump straight to any seat); a linked list is a treasure hunt (each item tells you where the next one is).**

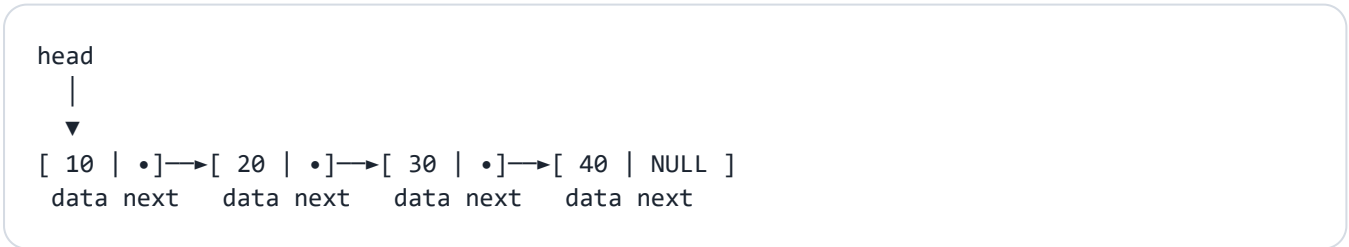
What is a data structure? It is simply a way of **organising data in memory** so that the operations you need (search, insert, delete) are fast. A phone's contact list kept in alphabetical order is a data structure — it makes finding a name quick.

Data structures are of two kinds: **linear** (items in a line — array, linked list, stack, queue) and **non-linear** (items branch out — tree, graph).

1. Array — elements of the same type stored in **one continuous block** of memory, numbered 0, 1, 2, ...

> 💡 **THINK OF IT LIKE THIS**
>
> Cinema seats in one row, numbered 0 to 9. To reach seat 7 you walk straight to it — you don't need to pass the other seats one by one. But if a new person must sit at seat 2, everyone from seat 2 onwards has to shift one place.

2. Linked list — each element (called a **node**) holds the data AND a pointer to the next node. Nodes can be anywhere in memory; the pointers link them in order. The first node is called the **head**; the last node points to **NULL** (nothing).



A singly linked list

> 💡 **THINK OF IT LIKE THIS**
>
> A treasure hunt: each clue tells you where the next clue is hidden. To reach the 4th clue you must follow clues 1, 2 and 3 first. But adding a new clue in the middle is easy — just change where one clue points.

➖ SOLVED EXAMPLE

Insert 15 between 10 and 20 in the linked list above

1. Create a new node with data 15.
2. Make the new node point to where 10 was pointing, i.e. to 20.
3. Make node 10 point to the new node 15.
4. Nothing else moves — only two pointers changed.

✅ 10 → 15 → 20 → 30 → 40, done in constant time O(1) once you are at node 10

Job	Array	Linked list
Go to the i-th element	Instant — O(1)	Walk from the head — O(n)
Insert/delete at the beginning	Shift everything — O(n)	Change the head pointer — O(1)
Memory	One continuous block, fixed size	Scattered nodes, grows as needed, extra memory for pointers
Binary search possible?	Yes (if sorted)	No (no direct access to the middle)

Types of linked list: singly (pointer to next only), **doubly** (pointers to next AND previous — can move both ways), **circular** (last node points back to the first).

Finding the address of an array element (asked often): the computer calculates it, which is why access is instant.

📐 FORMULAS

1-D array **`Address of A[i] = Base + i × size`**

2-D array, row-major **`Address of A[i][j] = Base + (i × number of columns + j) × size`**

2-D array, column-major **`Address of A[i][j] = Base + (j × number of rows + i) × size`**

➖ SOLVED EXAMPLE

int A[4][5] (4 rows, 5 columns), base 1000, int = 4 bytes, row-major. Address of A[2][3]?

1. Row-major means rows are stored one after another: row 0, then row 1, then row 2…
2. Before reaching row 2 we pass 2 full rows = 2 × 5 = 10 elements.
3. Inside row 2 we pass 3 more elements (columns 0, 1, 2).
4. Total elements before A[2][3] = 10 + 3 = 13. Each is 4 bytes → 13 × 4 = 52 bytes.
5. Address = 1000 + 52.

✅ 1052

⚠ EXAM TRAP

Read whether the question says row-major or column-major — the answer differs. And remember indexing starts at 0 unless the question says otherwise.

⏱ RECAP IN 30 SECONDS

- ✓ Array: continuous memory, instant access by index, costly insert/delete
- ✓ Linked list: nodes with pointers, easy insert/delete, slow access
- ✓ Singly, doubly (both directions) and circular linked lists
- ✓ A[i] = Base + i × size; row-major A[i][j] = Base + (i × cols + j) × size

✍ Quick quiz

Q1. Accessing the middle element of a singly linked list of n nodes takes:

(A) $O(n^2)$ (B) O(n)
(C) O(1) (D) O(log n)

Q2. Which list lets you move backwards?

(A) Doubly linked (B) Singly linked
(C) None (D) Array of pointers

Q3. int a[10] starts at 500, int = 2 bytes. Address of a[7]:

(A) 512 (B) 514
(C) 507 (D) 516

Answers: Q1 B — You must walk from the head node by node. · Q2 A — Each node has a prev pointer too. · Q3 B — 500 + 7 × 2 = 514.

LESSON 3.2 · 16 MIN

Stack & queue (and converting expressions)

In one line: Stack = pile of plates (last put on, first taken off). Queue = ticket line (first to come, first served).

Stack — you can only add or remove at ONE end, called the **top**. Adding is **push**, removing is **pop**. The last item pushed is the first popped: **LIFO** (Last In, First Out).

> 💡 **THINK OF IT LIKE THIS**
>
> A pile of plates in a wedding buffet: the plate put on last is the one taken first. You never pull a plate from the middle.

SOLVED EXAMPLE

Push 1, push 2, push 3, pop, push 4, pop, pop — what remains?

1. Push 1 → [1]
2. Push 2 → [1, 2]
3. Push 3 → [1, 2, 3]
4. Pop → removes 3 (the top) → [1, 2]
5. Push 4 → [1, 2, 4]
6. Pop → removes 4 → [1, 2]
7. Pop → removes 2 → [1]

✅ Only 1 remains. Popped in order: 3, 4, 2

Errors: **overflow** = pushing onto a full stack; **underflow** = popping from an empty stack. Stacks are used for function calls, undo (Ctrl+Z), the browser back button, recursion and evaluating expressions.

Queue — add at one end (**rear**), remove from the other end (**front**). Adding is **enqueue**, removing is **dequeue**. **FIFO** (First In, First Out).

> 💡 **THINK OF IT LIKE THIS**
>
> A railway ticket counter line: the first person who joined is served first; new people join at the back.

Kind of queue	What's special
Simple queue	Insert at rear, delete at front
Circular queue	The end wraps around to the start, so freed space is reused: rear = (rear + 1) % size
Deque (double-ended)	Insert and delete at BOTH ends
Priority queue	The item with the highest priority leaves first (usually built with a heap)

Queues are used for printer jobs, CPU scheduling, keyboard buffers and Breadth-First Search.

Expressions — infix, prefix, postfix. We write `A + B` with the operator IN between (**infix**). Computers prefer **postfix** `A B +` (operator AFTER the operands) because it needs no brackets and is easy to evaluate with a stack. **Prefix** is `+ A B` (operator BEFORE).

Converting infix → postfix by hand (the easiest method): 1) put brackets around each operation in the order you would calculate it (BODMAS: * and / before + and −); 2) move each operator to just after its closing bracket; 3) remove the brackets.

➖ **SOLVED EXAMPLE**

Convert A + B * C − D to postfix

1. Which is done first? * → bracket it: A + (B * C) − D
2. Then + and − from left to right: ((A + (B * C)) − D)
3. Move each operator after its own bracket: (B * C) → B C *
4. (A + B C *) → A B C * +
5. (A B C * + − D) → A B C * + D −

✅ A B C * + D −

Evaluating a postfix expression with a stack: read left to right. A number → push it. An operator → pop TWO numbers, apply the operator (second-popped OP first-popped), push the result.

➖ SOLVED EXAMPLE

Evaluate 5 3 2 * + 4 –

1. Read 5 → push. Stack: [5]
2. Read 3 → push. Stack: [5, 3]
3. Read 2 → push. Stack: [5, 3, 2]
4. Read * → pop 2, pop 3 → 3 * 2 = 6 → push. Stack: [5, 6]
5. Read + → pop 6, pop 5 → 5 + 6 = 11 → push. Stack: [11]
6. Read 4 → push. Stack: [11, 4]
7. Read – → pop 4, pop 11 → 11 – 4 = 7 → push. Stack: [7]

✅ 7

⚠ EXAM TRAP

When you pop for an operator, the FIRST value popped is the RIGHT-hand operand. For 8 2 –: pop 2, pop 8 → 8 – 2 = 6, not 2 – 8.

⏱ RECAP IN 30 SECONDS

- ✓ Stack = LIFO, push/pop at the top; overflow/underflow
- ✓ Queue = FIFO, enqueue at rear, dequeue at front; circular queue reuses space
- ✓ Infix A + B, prefix + A B, postfix A B +
- ✓ Infix → postfix: bracket in BODMAS order, move operators after their brackets
- ✓ Postfix evaluation uses a stack; first popped = right operand

✍ Quick quiz

Q1. Push 1, 2, 3 then pop twice. What remains on the stack?

(A) 2 (B) 3
(C) 1 (D) 1 and 2

Q2. The postfix of (A + B) * C is:

(A) A B + C * (B) + A B * C
(C) A + B C * (D) A B C + *

Q3. Breadth-first search uses a:

(A) Hash table (B) Stack
(C) Heap (D) Queue

Answers: Q1 C — The two pops remove 3 then 2. · Q2 A — The bracketed sum first, then multiply. · Q3 D — BFS visits level by level using a queue.

LESSON 3.3 · 16 MIN

Trees, binary search trees & heaps

In one line: **A tree is like a family tree: one root at the top, children below. In a binary search tree, smaller values go left and bigger ones go right.**

Why trees? Lists are slow to search when they are long. A tree splits the data so you can skip half of it at every step.

Words to know: root = top node; **parent/child** = a node and the nodes directly below it; **leaf** = node with no children; **level** = distance from the root (root is level 0); **height** = number of levels below the root on the longest path.

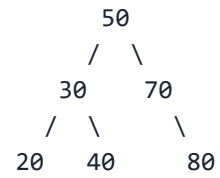
Binary tree = every node has at most 2 children (left and right). **Binary search tree (BST)** = a binary tree with one rule: everything in the LEFT subtree is SMALLER than the node, everything in the RIGHT subtree is BIGGER.

SOLVED EXAMPLE

Build a BST by inserting 50, 30, 70, 20, 40, 80

1. 50 → tree is empty, so 50 becomes the root.
2. 30 → smaller than 50 → goes LEFT of 50.
3. 70 → bigger than 50 → goes RIGHT of 50.
4. 20 → smaller than 50 (go left to 30) → smaller than 30 → LEFT of 30.
5. 40 → smaller than 50 (left to 30) → bigger than 30 → RIGHT of 30.
6. 80 → bigger than 50 (right to 70) → bigger than 70 → RIGHT of 70.

✅ The tree below



The BST built above

Searching a BST for 40: start at 50 → 40 is smaller → go left to 30 → 40 is bigger → go right → found. Each step throws away half the remaining tree, so a balanced BST is searched in O(log n) steps.

Traversal = visiting every node in some order. There are three famous orders, named by WHEN you visit the root:

Traversal	Order	Result for the tree above
Preorder	Root → Left → Right	50 30 20 40 70 80
Inorder	Left → Root → Right	20 30 40 50 70 80 (sorted!)
Postorder	Left → Right → Root	20 40 30 80 70 50
Level order	Level by level, left to right (uses a queue)	50 30 70 20 40 80

➖ **SOLVED EXAMPLE**

How to get the inorder traversal without mistakes

1. Start at the root 50. Inorder says: do the LEFT subtree completely first.
2. Left subtree root is 30 → again do ITS left first → 20 (a leaf, write 20).
3. Back at 30 → write 30 → then its right → 40 (write 40).
4. Left side of 50 is done → write 50.
5. Now the right subtree: 70 has no left → write 70 → its right → write 80.

✅ 20 30 40 50 70 80

🧠 **MEMORY TRICK**

PRE = root first, IN = root in the middle, POST = root last. And the inorder of a BST is always in ascending (sorted) order — a quick way to check your answer.

📐 **FORMULAS**

Maximum nodes at level l (root = level 0) 2^{l}

Maximum nodes in a binary tree of height h $2^{h+1} - 1$

Minimum height for n nodes $\lfloor \log_2 n \rfloor$

Full binary tree with i internal nodes has leaves $i + 1$

Worst case: if you insert already-sorted data (10, 20, 30, 40…) into a BST, every node goes to the right — the tree becomes a straight line and search becomes O(n). **AVL trees** fix this by re-balancing so the left and right heights never differ by more than 1.

Heap — a complete binary tree (filled level by level, left to right) where every parent is ≥ its children (**max-heap**) or ≤ its children (**min-heap**). The biggest (or smallest) item is always at the root — perfect for priority queues and heap sort.

A heap is stored in an **array**: for the element at index i (starting at 0), its children are at **2i + 1** and **2i + 2**, and its parent is at **⌊(i − 1) / 2⌋**.

⚠ EXAM TRAP

Preorder alone (or postorder alone) cannot rebuild a unique binary tree. You need the inorder sequence plus one of them.

⏱ RECAP IN 30 SECONDS

- ✓ BST rule: left < node < right; search is O(log n) if balanced, O(n) if skewed
- ✓ Pre = root first, In = root middle, Post = root last; inorder of a BST is sorted
- ✓ Max nodes with height h = $2^{h+1} - 1$
- ✓ Heap: complete tree, max/min at the root; children of i at 2i+1, 2i+2
- ✓ AVL tree keeps the BST balanced

✍ Quick quiz

Q1. Inorder traversal of a BST gives the keys in:

(A) Level order
(B) Descending order
(C) Random order
(D) Ascending order

Q2. Maximum number of nodes in a binary tree of height 3 (root at height 0):

(A) 15
(B) 16
(C) 7
(D) 8

Q3. Finding the maximum element in a max-heap takes:

(A) O(n log n)
(B) O(n)
(C) O(log n)
(D) O(1)

Answers: Q1 D — Left subtree < root < right subtree. · Q2 A — $2^4 - 1 = 15$. · Q3 D — It is always at the root.

LESSON 3.4 · 15 MIN

Graphs & hashing

In one line: **A graph is dots joined by lines (cities and roads); hashing calculates exactly where a value is stored, so finding it is almost instant.**

Graph = a set of **vertices** (points, V) joined by **edges** (lines, E). Unlike a tree, a graph can have cycles and no single root.

💡 THINK OF IT LIKE THIS

A map of Bihar: cities are vertices, roads are edges. A one-way road is a directed edge; the road length in km is the weight of the edge.

Term	Meaning
Directed / undirected	Edges have a direction (one-way) or not
Weighted	Each edge has a number (distance, cost)
Degree	Number of edges touching a vertex
Path / cycle	A route between vertices / a route that comes back to the start
Connected	There is a path between every pair of vertices
Complete graph	Every vertex is joined to every other vertex

📐 **FORMULAS**

Sum of all degrees (undirected) `2 × E (each edge touches two vertices)`

Edges in a complete graph of n vertices `n(n − 1) / 2`

Edges in a tree of n vertices `n − 1`

➖ **SOLVED EXAMPLE**

Maximum edges in a simple undirected graph with 6 vertices

1. Each vertex can join the other 5 → 6 × 5 = 30.
2. But each edge was counted twice (A–B and B–A), so divide by 2.

✅ 30 / 2 = 15 edges

Storing a graph	How	Space	Good for
Adjacency matrix	V × V grid; 1 if an edge exists	V^2	Dense graphs; checking an edge is instant
Adjacency list	For each vertex, a list of its neighbours	V + E	Sparse graphs (few edges) — saves memory

Visiting every vertex — BFS and DFS. BFS (Breadth-First Search) explores level by level — all neighbours first, then their neighbours — using a **queue**. **DFS (Depth-First Search)** goes as deep as possible along one path, then backtracks — using a **stack** (or recursion). Both take O(V + E) time.

💡 **THINK OF IT LIKE THIS**

BFS is like ripples in a pond — spreading out ring by ring. DFS is like exploring a maze — follow one corridor to the end, then come back and try the next.

BFS finds the **shortest path in an unweighted graph** (fewest edges). DFS is used to detect cycles and for topological sorting.

Hashing — instead of searching, we CALCULATE where a value should be stored using a **hash function**, for example h(key) = key mod 10. To find the key later we calculate the same position — no searching needed.

> 💡 **THINK OF IT LIKE THIS**
> A school puts each student's file in a cupboard numbered by the last digit of their roll number. To find roll 4523 you go straight to cupboard 3.

A **collision** happens when two keys get the same position (23 and 43 both give 3). Two ways to solve it: **chaining** (keep a small linked list at each position) and **open addressing** (look for another empty slot — linear probing tries the next slot, then the next…).

➖ SOLVED EXAMPLE

Insert 23, 43, 13 into a table of size 10 with h(k) = k mod 10, linear probing

1. 23 mod 10 = 3 → slot 3 is empty → store at 3.
2. 43 mod 10 = 3 → slot 3 is taken → try 4 → empty → store at 4.
3. 13 mod 10 = 3 → 3 taken, 4 taken → try 5 → store at 5.

✅ 23 at 3, 43 at 4, 13 at 5

Load factor α = number of keys ÷ table size. A low load factor keeps searching close to O(1). Linear probing causes **clustering** (long runs of filled slots).

⏱ RECAP IN 30 SECONDS

- ✓ Graph = vertices + edges; complete graph has $n(n-1)/2$ edges; a tree has $n-1$
- ✓ Adjacency matrix V^2 space (dense); adjacency list $V + E$ (sparse)
- ✓ BFS = queue, level by level, shortest path in unweighted graphs; DFS = stack, deep first
- ✓ Hashing computes the position; collisions solved by chaining or probing

✍️ **Quick quiz**

Q1. Maximum edges in a simple undirected graph with 6 vertices:

(A) 12 (B) 30
(C) 36 (D) 15

Q2. Which representation suits a sparse graph?

(A) Adjacency list (B) None
(C) Adjacency matrix (D) Incidence matrix

Q3. With h(k) = k mod 7, key 50 goes to slot:

(A) 0 (B) 7
(C) 1 (D) 5

Answers: Q1 D — 6 × 5 / 2 = 15. · Q2 A — It stores only the edges that exist: O(V + E). · Q3 C — 50 = 7 × 7 + 1.

MODULE 4

Algorithms & complexity

How to measure how fast a method is, and the famous searching, sorting and graph algorithms — traced step by step.

LESSON 4.1 · 14 MIN

Time complexity (Big-O) from zero

In one line: **Big-O doesn't measure seconds — it measures how the number of steps GROWS when the input gets bigger.**

Why not measure in seconds? The same program runs faster on a new laptop than on an old one. So instead we count **how many basic steps** the algorithm needs for an input of size **n**, and look at how that number grows as n grows.

💡 **THINK OF IT LIKE THIS**

Finding a name in a 1000-page phone book. Method 1: read every page from the start — up to 1000 page-turns (grows like n). Method 2: open the middle, decide which half the name is in, repeat — about 10 page-turns (grows like log n). For a million pages: a million turns versus about 20!

The rules for working out Big-O: 1) count how many times the main work repeats; 2) **drop constants** (2n and 5n are both O(n)); 3) **keep only the fastest-growing term** ($n^2 + 100n + 7$ is $O(n^2)$), because for big n the n^2 part dwarfs the rest).

Big-O	Name	If n = 1,000 steps ≈	Example
O(1)	Constant	1	Get a[5] from an array; push/pop
O(log n)	Logarithmic	10	Binary search
O(n)	Linear	1,000	Linear search; one loop over the data
O(n log n)	Linearithmic	10,000	Merge sort, heap sort
O(n^2)	Quadratic	1,000,000	Two nested loops; bubble sort
O(2^n)	Exponential	a 302-digit number	Trying every subset

From fastest to slowest

➖ SOLVED EXAMPLE

Find the complexity of each piece of code

1. for (i = 0; i < n; i++) sum++; → the line runs n times → O(n).
2. for (i = 0; i < n; i++) for (j = 0; j < n; j++) sum++; → n rounds × n rounds = n^2 → O(n^2).
3. for (i = 1; i < n; i = i * 2) sum++; → i goes 1, 2, 4, 8 … doubling until it reaches n, about $\log_2$ n times → O(log n).
4. A loop of n, followed (not inside) by another loop of n → n + n = 2n → O(n).

✅ Nested loops MULTIPLY; loops one after another ADD; a loop that doubles or halves gives log n

```c
for (i = 1; i < n; i = i * 2)     // runs log n times
    for (j = 0; j < n; j++)        // runs n times each
        count++;
```

Output ▸ O(n log n)

Best, average and worst case. For linear search the best case is O(1) (the item is first), the worst case is O(n) (the item is last or missing). Exams usually mean the worst case unless they say otherwise.

Symbol	Meaning	Think of it as
O (Big-O)	Upper bound	"at most this much" — worst case
Ω (Omega)	Lower bound	"at least this much" — best case
Θ (Theta)	Tight bound	"exactly this order"

Recurrences describe recursive algorithms. You only need the common ones:

📐 FORMULAS

T(n) = T(n/2) + 1 **O(log n) — binary search**

T(n) = 2T(n/2) + n **O(n log n) — merge sort**

T(n) = T(n − 1) + 1 **O(n)**

T(n) = T(n − 1) + n **O(n^2)**

T(n) = 2T(n − 1) + 1 **O(2^n) — Tower of Hanoi**

Space complexity = extra memory the algorithm needs. Merge sort needs an extra array → O(n) space; bubble sort only swaps in place → O(1) space.

⏱ **RECAP IN 30 SECONDS**

- ✓ Big-O = how the number of steps grows with n; drop constants, keep the biggest term
- ✓ $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
- ✓ Nested loops multiply; doubling/halving loops give log n
- ✓ O = upper bound (worst), Ω = lower bound (best), Θ = tight
- ✓ $T(n) = 2T(n/2) + n \rightarrow O(n \log n)$

✍ **Quick quiz**

Q1. The complexity of $5n^3 + 100n^2 + 7$ is:

(A) $O(1)$ (B) $O(100n^2)$
(C) $O(n^3)$ (D) $O(n^2)$

Q2. $T(n) = 2T(n/2) + n$ solves to:

(A) $O(n^2)$ (B) $O(n \log n)$
(C) $O(n)$ (D) $O(\log n)$

Q3. A loop where i = i * 2 until i ≥ n runs about:

(A) n/2 times (B) n^2 times
(C) n times (D) $\log_2 n$ times

Answers: Q1 C — Keep the fastest-growing term, drop constants. · Q2 B — The merge-sort recurrence. · Q3 D — Doubling reaches n after $\log_2 n$ steps.

LESSON 4.2 · 20 MIN

Searching & sorting — traced step by step

In one line: **Each sorting method is just a different strategy for putting things in order; learn how each one moves the numbers and its speed.**

Linear search — check each item one by one from the start. Works on any list, sorted or not. Worst case O(n).

Binary search — only for a **sorted** list. Look at the middle item: if it's the one, stop; if your target is smaller, keep only the left half; if bigger, keep only the right half. Repeat. Each step throws away half → O(log n).

SOLVED EXAMPLE

Binary search for 23 in [5, 9, 13, 17, 23, 31, 40] (indexes 0–6)

1. low = 0, high = 6 → mid = (0 + 6) / 2 = 3 → a[3] = 17. 23 > 17 → search the right half: low = 4.
2. low = 4, high = 6 → mid = 5 → a[5] = 31. 23 < 31 → search the left half: high = 4.
3. low = 4, high = 4 → mid = 4 → a[4] = 23 → found!

✅ Found at index 4 in 3 comparisons (linear search would need 5)

Bubble sort — compare each pair of neighbours and swap them if they're in the wrong order. After each full pass the biggest remaining number “bubbles up” to the end.

SOLVED EXAMPLE

Bubble sort [5, 1, 4, 2] — pass 1

1. Compare 5 and 1 → wrong order → swap → [1, 5, 4, 2]
2. Compare 5 and 4 → swap → [1, 4, 5, 2]
3. Compare 5 and 2 → swap → [1, 4, 2, 5] — 5 is now in its final place
4. Pass 2 does the same on the first three → [1, 2, 4, 5]

✅ Sorted: [1, 2, 4, 5]

Selection sort — find the smallest item and swap it into position 0; find the next smallest and put it in position 1; and so on. Makes very few swaps (at most n − 1).

Insertion sort — like arranging playing cards in your hand: take the next card and slide it left into its correct place among the cards already sorted. Very fast when the data is already almost sorted.

Merge sort (divide and conquer) — split the list into two halves, sort each half (by splitting again and again until single items), then **merge** the sorted halves together.

SOLVED EXAMPLE

Merge sort [38, 27, 43, 3]

1. Split: [38, 27] and [43, 3]
2. Split again: [38] [27] [43] [3] — single items are already sorted
3. Merge pairs: [27, 38] and [3, 43]
4. Merge the two halves by repeatedly taking the smaller front item: 3, 27, 38, 43

✅ [3, 27, 38, 43] — always O(n log n), but needs extra memory O(n)

Quick sort (divide and conquer) — pick a **pivot**, put everything smaller on its left and bigger on its right, then quick-sort each side. Usually the fastest in practice, but if the pivot is always the smallest or largest (e.g. already sorted data with the first element as pivot) it degrades to $O(n^2)$.

Heap sort — build a max-heap, repeatedly take the root (the largest) and move it to the end. Always O(n log n), no extra memory.

Algorithm	Best	Average	Worst	Extra space	Stable?
Bubble sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting / radix	O(n + k)	O(n + k)	O(n + k)	O(n + k)	Yes

The most-asked table in the whole unit

What does "stable" mean? If two items have equal keys, a stable sort keeps them in their original order. Example: sorting students by marks — two students with 80 stay in roll-number order.

MEMORY TRICK

Stable sorts: "Bubbles Insert Merge Counting" — Bubble, Insertion, Merge, Counting. The unstable ones: Selection, Quick, Heap.

⚠ EXAM TRAP

Binary search needs at most $\lfloor \log_2 n \rfloor + 1$ comparisons. For 1000 sorted items that's 10 — not 500. And it does NOT work on an unsorted list.

⏱ RECAP IN 30 SECONDS

- ✓ Linear search O(n) on any list; binary search O(log n) on a sorted list
- ✓ Bubble swaps neighbours; selection picks the minimum; insertion slides into place
- ✓ Merge sort always O(n log n) but uses O(n) extra memory
- ✓ Quick sort: average O(n log n), worst $O(n^2)$ with bad pivots
- ✓ Stable: bubble, insertion, merge, counting

✍️ **Quick quiz**

Q1. Which sort has O(n log n) time even in the worst case?

(A) Bubble sort
(B) Insertion sort
(C) Quick sort
(D) Merge sort

Q2. Best sort for an array that is already almost sorted:

(A) Quick sort with first pivot
(B) Selection sort
(C) Insertion sort
(D) Heap sort

Q3. Which of these is NOT stable?

(A) Insertion sort
(B) Merge sort
(C) Heap sort
(D) Bubble sort

Q4. Maximum comparisons in binary search on 1024 sorted items:

(A) 11
(B) 512
(C) 1024
(D) 10

Answers: Q1 D — Merge sort always splits evenly. · Q2 C — Very few shifts → close to O(n). · Q3 C — Heap operations can reorder equal keys. · Q4 A — $\lfloor \log_2 1024 \rfloor + 1 = 10 + 1 = 11$.

LESSON 4.3 · 18 MIN

Problem-solving strategies & graph algorithms

In one line: **Greedy grabs the best-looking choice right now; dynamic programming saves answers to small problems so it never solves the same one twice.**

Most algorithms follow one of four strategies:

Strategy	Idea in one line	Famous examples
Divide & conquer	Split the problem, solve the parts, combine	Merge sort, quick sort, binary search
Greedy	Take the best choice at each step and never look back	Kruskal, Prim, Dijkstra, Huffman coding, fractional knapsack
Dynamic programming (DP)	Solve small sub-problems once, store the answers, reuse them	0/1 knapsack, LCS, Floyd–Warshall, Bellman–Ford
Backtracking	Try an option; if it fails, undo and try the next	N-Queens, Sudoku, subset sum

💡 **THINK OF IT LIKE THIS**

Greedy = paying ₹ 70 with the fewest coins by always taking the biggest coin that fits (50, then 20). DP = a student who writes down the answer of every practice question so that when it appears again she just looks it up.

Knapsack example. You can carry 10 kg. In the **fractional** version you may take part of an item (like rice) → greedy works: take items with the highest value per kg first. In the **0/1** version each item must be taken whole or not at all → greedy can fail, so we use DP.

Minimum Spanning Tree (MST) — connect ALL vertices of a weighted graph using the cheapest total edge cost, with no cycles. A spanning tree of V vertices always has **V − 1** edges.

SOLVED EXAMPLE

Kruskal's algorithm on edges A–B 1, B–C 3, A–C 2, C–D 4, B–D 5

1. Sort the edges from cheapest: AB 1, AC 2, BC 3, CD 4, BD 5.
2. Take AB (1) — no cycle.
3. Take AC (2) — no cycle.
4. BC (3)? A, B, C are already connected; adding BC makes a cycle → skip it.
5. Take CD (4). We now have 3 edges = V − 1 for 4 vertices → stop.

✅ MST = AB + AC + CD, total weight 1 + 2 + 4 = 7

Prim's algorithm also builds an MST, but grows ONE tree from a starting vertex, always adding the cheapest edge that reaches a new vertex.

Shortest paths. Dijkstra finds the shortest path from one source to all vertices: it repeatedly picks the closest unvisited vertex and updates its neighbours' distances. It is greedy, so it can go wrong with **negative edge weights**. Bellman–Ford handles negative weights (and detects negative cycles) by relaxing every edge V − 1 times.

SOLVED EXAMPLE

Dijkstra from A: edges A–B 4, A–C 1, C–B 2, B–D 1

1. Start: dist(A) = 0, all others = ∞.
2. Visit A: update B = 4, C = 1.
3. Closest unvisited is C (1). Via C, B = 1 + 2 = 3 < 4 → update B = 3.
4. Next closest is B (3). Via B, D = 3 + 1 = 4.
5. Visit D. Done.

✅ Shortest distances from A: C = 1, B = 3, D = 4

Algorithm	Solves	Negative weights?	Time
Dijkstra	One source → all vertices	No	O((V + E) log V) with a heap
Bellman–Ford	One source → all vertices	Yes, and detects negative cycles	O(V × E)
Floyd–Warshall	All pairs	Yes (no negative cycles)	$O(V^3)$
Kruskal	Minimum spanning tree (sort edges)	—	O(E log E)
Prim	Minimum spanning tree (grow a tree)	—	O(E log V) with a heap

Huffman coding (greedy) compresses data: frequent characters get short codes, rare ones get long codes. **Topological sort** orders tasks so each comes after the ones it depends on — only possible in a graph with no cycles (DAG).

⚠ EXAM TRAP

Dijkstra fails with negative edges — the exam answer for negative weights is Bellman–Ford. For "all pairs" the answer is Floyd–Warshall.

⏱ RECAP IN 30 SECONDS

- ✓ Greedy: best choice now (Kruskal, Prim, Dijkstra, Huffman, fractional knapsack)
- ✓ DP: store sub-problem answers (0/1 knapsack, LCS, Floyd–Warshall, Bellman–Ford)
- ✓ A spanning tree of V vertices has V − 1 edges
- ✓ Kruskal = sort edges, skip cycles; Prim = grow one tree
- ✓ Negative weights → Bellman–Ford; all pairs → Floyd–Warshall

✍️ **Quick quiz**

Q1. Which algorithm finds shortest paths between ALL pairs of vertices?

(A) Kruskal
(B) Dijkstra
(C) Prim
(D) Floyd–Warshall

Q2. 0/1 knapsack is best solved by:

(A) Greedy
(B) Binary search
(C) BFS
(D) Dynamic programming

Q3. A spanning tree of a graph with 10 vertices has how many edges?

(A) 9
(B) 11
(C) 45
(D) 10

Answers: Q1 D — Floyd–Warshall is all-pairs, $O(V^3)$. · Q2 D — Greedy works only for the fractional version. · Q3 A — $V - 1$.

MODULE 5

Operating systems

The manager that shares the CPU, memory and disk between programs — every numerical explained step by step.

LESSON 5.1 · 14 MIN

What an OS does; processes & threads

In one line: **The OS is the manager between you and the hardware; a process is a program that is currently running.**

What is an operating system? When you open WhatsApp, a browser and music at the same time, something must decide who uses the CPU now, where each app sits in memory and who may use the speaker. That "something" is the **operating system (OS)** — Windows, Linux, Android, macOS.

> 💡 **THINK OF IT LIKE THIS**
>
> The OS is like the manager of a busy restaurant: it decides which cook (CPU) handles which order (program), which table (memory) each customer gets, and makes sure no two waiters grab the same plate at once.

Job of the OS	Meaning
Process management	Starting, pausing and stopping programs; sharing the CPU
Memory management	Giving each program memory and taking it back
File management	Creating, storing and protecting files and folders
Device (I/O) management	Controlling printers, disks, keyboard through drivers
Security & protection	Passwords, permissions, keeping programs apart
User interface	Command line (CLI) or graphical (GUI)

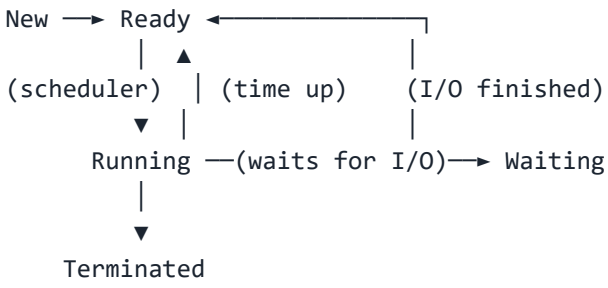
Type of OS	Idea	Example
Batch	Jobs collected and run one after another, no interaction	Old payroll systems
Multiprogramming	Several programs in memory; CPU switches when one waits for I/O	—
Time-sharing / multitasking	CPU switches between programs so fast each feels it runs alone	Windows, Linux
Real-time (RTOS)	Must respond within a strict time limit	Airbag systems, missile control
Distributed	Many computers work together as one	Cloud systems

Kernel = the core part of the OS that always stays in memory and controls the hardware. Programs ask the kernel for services (like reading a file) through **system calls**.

Program vs process. A **program** is a file sitting on the disk (passive). When you run it, it becomes a **process** — a program in action, with its own memory, current position and status.

> 💡 **THINK OF IT LIKE THIS**
> A recipe book on the shelf is a program. Cooking that recipe right now in the kitchen is a process. You can cook the same recipe twice at once — two processes from one program.

Process states — a process moves between these states during its life:



The five-state process model

State	Meaning
New	Being created
Ready	Waiting in line for the CPU
Running	Currently using the CPU
Waiting (blocked)	Waiting for something else — input, disk, printer
Terminated	Finished

The OS keeps information about each process in a **Process Control Block (PCB)**: process ID, state, program counter (where it stopped), CPU register values, memory details, open files, priority. When the CPU switches from one process to another, the OS saves the old PCB and loads the new one — this is a **context switch** (pure overhead, no useful work).

Thread = a lightweight part of a process that can run on its own. A browser process may have one thread loading a page and another playing a video. Threads of the same process **share** code, data and open files, but each has its **own stack, registers and program counter**. Creating a thread is cheaper than creating a process.

> ⚠ **EXAM TRAP**
> A waiting process never goes straight to Running. When its I/O finishes it goes to Ready and waits for the scheduler like everyone else.

> ⏱ **RECAP IN 30 SECONDS**
> ✓ OS = manager of CPU, memory, files, devices and security; kernel is its core
> ✓ Program = passive file; process = program running
> ✓ States: New → Ready ⇄ Running → Waiting → Ready … → Terminated
> ✓ PCB stores everything about a process; context switch = save one PCB, load another
> ✓ Threads share code/data/files but have their own stack and registers

✍️ **Quick quiz**

Q1. Which do threads of the same process NOT share?

(A) Open files
(B) Code
(C) Stack
(D) Global data

Q2. After its I/O completes, a waiting process moves to:

(A) Running
(B) Ready
(C) New
(D) Terminated

Q3. An airbag controller needs which type of OS?

(A) Real-time
(B) Batch
(C) Time-sharing
(D) Distributed

Answers: Q1 C — Each thread has its own stack and registers. · Q2 B — It must wait for the CPU scheduler again. · Q3 A — It must respond within a strict deadline.

LESSON 5.2 · 20 MIN

CPU scheduling — Gantt charts step by step

In one line: **When many processes want the CPU, the scheduler decides the order; draw a timeline (Gantt chart) and every question becomes simple subtraction.**

Why scheduling? Only one process can use a CPU core at a time. The **CPU scheduler** chooses which ready process runs next. Different rules give different waiting times.

Words you need: arrival time = when the process comes; **burst time** = how much CPU time it needs; **completion time** = when it finishes.

📐 **FORMULAS**

Turnaround time (TAT) `Completion time − Arrival time (total time in the system)`

Waiting time (WT) `Turnaround time − Burst time (time spent waiting in line)`

Response time `Time it FIRST gets the CPU − Arrival time`

A Gantt chart is a timeline bar showing which process runs when. We use this example for every method:

Process	Arrival time	Burst time
P1	0	6
P2	0	8
P3	0	7
P4	0	3

1. FCFS (First Come, First Served) — run them in the order they arrived, each to completion. Simple, like a queue at a shop.

SOLVED EXAMPLE

FCFS

1. Order P1, P2, P3, P4. P1 runs 0 → 6, P2 runs 6 → 14, P3 runs 14 → 21, P4 runs 21 → 24.
2. Gantt: | P1 0–6 | P2 6–14 | P3 14–21 | P4 21–24 |
3. Everyone arrived at 0, so waiting time = start time: P1 = 0, P2 = 6, P3 = 14, P4 = 21.
4. Total = 0 + 6 + 14 + 21 = 41. Average = 41 ÷ 4.

✅ Average waiting time = 10.25

Problem with FCFS: the short job P4 (3 units) waits 21 units behind long jobs. This is called the **convoy effect**.

2. SJF (Shortest Job First) — always run the job with the smallest burst time next.

SOLVED EXAMPLE

SJF (non-preemptive)

1. Sort by burst: P4 (3), P1 (6), P3 (7), P2 (8).
2. Gantt: | P4 0–3 | P1 3–9 | P3 9–16 | P2 16–24 |
3. Waiting times = start times: P4 = 0, P1 = 3, P3 = 9, P2 = 16.
4. Total = 28. Average = 28 ÷ 4.

✅ Average waiting time = 7 — the lowest possible

SJF gives the **minimum average waiting time**, but a long job may wait forever if short jobs keep arriving (**starvation**). The preemptive version, **SRTF**, stops the running job if a new job with a shorter remaining time arrives.

3. Round Robin (RR) — every process gets a fixed slice of time called the **time quantum**. If it isn't finished when its slice ends, it goes to the back of the line.

➖ SOLVED EXAMPLE

Round Robin with time quantum = 4

1. P1 runs 0–4 (2 units left), P2 runs 4–8 (4 left), P3 runs 8–12 (3 left), P4 runs 12–15 (needs only 3 → finished).
2. Second round: P1 runs 15–17 (finished), P2 runs 17–21 (finished), P3 runs 21–24 (finished).
3. Completion times: P1 = 17, P2 = 21, P3 = 24, P4 = 15.
4. Waiting = completion − burst (all arrived at 0): P1 = 17 − 6 = 11, P2 = 21 − 8 = 13, P3 = 24 − 7 = 17, P4 = 15 − 3 = 12.
5. Total = 53. Average = 53 ÷ 4.

✅ Average waiting time = 13.25 (but every process gets a turn quickly — fair for many users)

4. Priority scheduling — the process with the highest priority runs first. Low-priority jobs can starve; the fix is **aging** — slowly increasing the priority of jobs that have waited a long time.

Algorithm	Preemptive?	Good point	Problem
FCFS	No	Simple, fair order	Convoy effect; high waiting time
SJF	No (SRTF = yes)	Minimum average waiting time	Starvation; burst time must be known
Round Robin	Yes	Fair; good response time	Too small quantum = too many switches; too large = FCFS
Priority	Either	Important jobs first	Starvation → fixed by aging

⚠ EXAM TRAP

When arrival times are different, a process cannot run before it arrives. Write the arrival times on the timeline first, then fill the Gantt chart.

➖ SOLVED EXAMPLE

Different arrival times — FCFS: P1 arrives 0 (burst 5), P2 arrives 1 (burst 3)

1. P1 runs 0–5. P2 arrived at 1 but must wait until 5, then runs 5–8.
2. P2: turnaround = 8 − 1 = 7; waiting = 7 − 3 = 4.

✅ Waiting time of P2 = 4

⏱ **RECAP IN 30 SECONDS**

- ✓ TAT = completion − arrival; WT = TAT − burst
- ✓ FCFS = simple but convoy effect; SJF = minimum average waiting time but starvation
- ✓ Round Robin = time quantum, fair; huge quantum behaves like FCFS
- ✓ Priority starvation is fixed by aging
- ✓ Always draw the Gantt chart first

✍ **Quick quiz**

Q1. Which algorithm may cause starvation?

(A) FCFS
(B) Round Robin
(C) None of these
(D) SJF

Q2. P1 arrives at 0 with burst 5, P2 at 1 with burst 3; FCFS. Waiting time of P2:

(A) 0
(B) 1
(C) 5
(D) 4

Q3. Aging is a solution to:

(A) Fragmentation
(B) Thrashing
(C) Starvation
(D) Deadlock

Answers: Q1 D — Long jobs can wait forever if short jobs keep coming. · Q2 D — P2 starts at 5; waiting = 5 − 1 = 4. · Q3 C — Waiting processes gradually gain priority.

LESSON 5.3 · 14 MIN

Process synchronisation & semaphores

In one line: **When two processes use the same data, only one may be in the "critical section" at a time — a semaphore is the key that controls entry.**

The problem. Two processes sharing one variable can produce a wrong result if they change it at the same moment. This is called a **race condition** — the result depends on who "wins the race".

➖ **SOLVED EXAMPLE**

A race condition with a bank balance of ₹ 1000

1. Process A wants to add ₹ 100; process B wants to subtract ₹ 200. Correct final balance = ₹ 900.
2. A reads balance = 1000. Before A saves, B also reads 1000.
3. A saves 1000 + 100 = 1100.
4. B saves 1000 − 200 = 800 — overwriting A's update.

✅ **Final balance ₹ 800 instead of ₹ 900 — A's deposit is lost**

The part of the code that touches shared data is the **critical section**. A correct solution must guarantee: **mutual exclusion** (only one process inside at a time), **progress** (if nobody is inside, someone who wants to enter can), and **bounded waiting** (no process waits forever).

> 💡 **THINK OF IT LIKE THIS**
> A single-key washroom at a petrol pump: whoever holds the key goes in, everyone else waits; when they come out they hand the key back. The key is the semaphore.

Semaphore = an integer variable S that can only be changed by two special operations:

Operation	Also called	What it does
wait(S)	P(S), down	If S > 0, decrease S by 1 and enter; if S ≤ 0, wait
signal(S)	V(S), up	Increase S by 1 (and wake up a waiting process)

Binary semaphore takes only 0 or 1 — it works like a lock (a **mutex**). **Counting semaphore** can be any number — it allows up to N processes in at once (e.g. 3 printers → S starts at 3).

➖ SOLVED EXAMPLE

A counting semaphore S = 10. Then 6 P (wait) and 4 V (signal) operations are performed. Final S?

1. Every P decreases S by 1: 10 − 6 = 4.
2. Every V increases S by 1: 4 + 4 = 8.

✅ **S = 8 (formula: initial − P + V)**

Classic problem	What it shows
Producer–consumer (bounded buffer)	One process fills a buffer, another empties it; neither may overflow/underflow it
Readers–writers	Many may read together, but a writer needs the data alone
Dining philosophers	5 philosophers, 5 forks — shows deadlock and starvation

Other tools: **Peterson's solution** (a pure software solution for 2 processes), **monitors** (a language feature that allows only one process inside at a time), **test-and-set** hardware instructions. Constantly looping while waiting is **busy waiting** (a **spinlock**) — it wastes CPU time.

⏱ **RECAP IN 30 SECONDS**

- ✓ Race condition = result depends on timing of shared updates
- ✓ Critical section needs mutual exclusion, progress, bounded waiting
- ✓ wait/P decreases S (blocks if not positive); signal/V increases S
- ✓ Final S = initial − number of P + number of V
- ✓ Binary semaphore = mutex; counting semaphore = N copies of a resource

✍ **Quick quiz**

Q1. S = 7; then 20 P and 15 V operations. Final S?

(A) 12 (B) 2
(C) 22 (D) −2

Q2. The dining philosophers problem illustrates:

(A) Paging (B) Compilation
(C) Disk scheduling (D) Deadlock and starvation in resource sharing

Q3. Only one process may be in the critical section at a time. This requirement is:

(A) Bounded waiting (B) Aging
(C) Mutual exclusion (D) Progress

Answers: Q1 B — 7 − 20 + 15 = 2. · Q2 D — Five philosophers compete for five forks. · Q3 C — Mutual exclusion.

LESSON 5.4 · 18 MIN

Deadlock & Banker's algorithm

In one line: **Deadlock = everyone is holding something and waiting for something another one holds, so nobody can ever move.**

💡 **THINK OF IT LIKE THIS**

Two people on a narrow bridge from opposite ends, each refusing to step back. Or four cars at a crossroad, each waiting for the car on its right. Nobody moves — ever.

Example with computers: process P1 holds the printer and wants the scanner; process P2 holds the scanner and wants the printer. Each waits for the other forever.

Deadlock happens only if ALL FOUR conditions hold at the same time:

Condition	Meaning
Mutual exclusion	A resource can be used by only one process at a time
Hold and wait	A process holds one resource while waiting for another
No preemption	A resource cannot be forcibly taken away
Circular wait	P1 waits for P2, P2 waits for P3 … Pn waits for P1

🧠 **MEMORY TRICK**

"My Hungry Neighbour Cries" — Mutual exclusion, Hold and wait, No preemption, Circular wait.

Way to handle it	Idea
Prevention	Make sure one of the four conditions can never hold (e.g. ask for all resources at once; or number resources and request in increasing order — breaks circular wait)
Avoidance	Check every request: grant it only if the system stays SAFE — Banker's algorithm
Detection & recovery	Let deadlock happen, detect it with a wait-for graph, then kill or roll back a process
Ignore it (ostrich)	Assume it is rare and reboot if it happens — most desktop OSs

Banker's algorithm — like a careful bank manager who lends money only if he can still satisfy every customer in SOME order. A state is **safe** if there is at least one order (a **safe sequence**) in which every process can get what it still needs, finish, and give its resources back.

📐 **FORMULAS**

Need of a process `Max – Allocation (how much more it may still ask for)`

Available `Total resources – sum of all Allocation`

Process	Allocation (A, B)	Max (A, B)	Need = Max − Allocation
P0	2, 1	5, 3	3, 2
P1	3, 1	4, 2	1, 1
P2	2, 1	7, 3	5, 2
P3	1, 0	2, 2	1, 2

Total: A = 10, B = 5. Allocated: A = 2+3+2+1 = 8, B = 1+1+1+0 = 3 → Available = (2, 2)

➖ SOLVED EXAMPLE

Is this state safe? Find a safe sequence.

1. Available = (2, 2). Look for a process whose Need ≤ Available.
2. P0 needs (3, 2): 3 > 2 → can't run yet. P1 needs (1, 1) ≤ (2, 2) → YES. P1 finishes and RETURNS its allocation (3, 1): Available = (2+3, 2+1) = (5, 3).
3. P2 needs (5, 2) ≤ (5, 3) → yes. Returns (2, 1): Available = (7, 4).
4. P3 needs (1, 2) ≤ (7, 4) → yes. Returns (1, 0): Available = (8, 4).
5. Back to P0: needs (3, 2) ≤ (8, 4) → yes. Returns (2, 1): Available = (10, 5) = the total ✓.

✅ **Safe sequence P1 → P2 → P3 → P0, so the system is SAFE**

⚠ EXAM TRAP

When a process finishes you ADD its Allocation (not its Need) back to Available. And an unsafe state is not yet a deadlock — it only means a deadlock is possible.

Quick formula question: n processes each need k units of a resource at most. The minimum number of units that guarantees no deadlock = **n(k − 1) + 1**. Reason: in the worst case each process holds k − 1 units and waits; one more unit lets one of them finish and free its units.

➖ SOLVED EXAMPLE

3 processes each need 2 units. Minimum units to guarantee no deadlock?

1. Worst case: each holds 2 − 1 = 1 unit → 3 units used, all waiting.
2. One extra unit lets one process finish → 3 + 1.

✅ **4 units (3 × (2 − 1) + 1)**

⏱ RECAP IN 30 SECONDS

- ✓ Four conditions together: mutual exclusion, hold & wait, no preemption, circular wait
- ✓ Prevention breaks a condition; avoidance = Banker's; detection uses a wait-for graph
- ✓ Need = Max − Allocation; finishing process returns its Allocation
- ✓ Safe = a safe sequence exists; unsafe ≠ deadlock
- ✓ Minimum units = n(k − 1) + 1

✍️ **Quick quiz**

Q1. 3 processes each need 2 units of a resource. Minimum units that guarantee no deadlock:

(A) 5 (B) 3
(C) 4 (D) 6

Q2. Banker's algorithm is a deadlock ___ technique.

(A) detection (B) recovery
(C) avoidance (D) prevention

Q3. Which is NOT a necessary condition for deadlock?

(A) Mutual exclusion (B) Circular wait
(C) Preemption (D) Hold and wait

Answers: Q1 C — $n(k-1)+1 = 3\times1+1 = 4$. · Q2 C — It avoids unsafe states. · Q3 C — The condition is NO preemption.

LESSON 5.5 · 22 MIN

Memory management, virtual memory & disk scheduling

In one line: **Paging cuts programs into equal pages so they fit anywhere in memory, and virtual memory loads only the pages actually needed.**

The problem. RAM is limited, and many programs want it at once. The OS must decide where each program goes, and what to do when they don't all fit.

Fragmentation = wasted memory. Internal fragmentation: a program gets a block bigger than it needs; the leftover inside the block is wasted. **External fragmentation**: there is enough free memory in total, but it's in small scattered pieces, so a big program can't fit. Fix: **compaction** (slide programs together) or **paging**.

> 💡 **THINK OF IT LIKE THIS**
> Internal: renting a 3-BHK flat for 1 person — rooms inside are wasted. External: a cinema with 20 empty seats scattered one-by-one, so a family of 5 can't sit together.

Paging. Cut the program (logical memory) into equal pieces called **pages**, and RAM (physical memory) into pieces of the same size called **frames**. Any page can go into any free frame. A **page table** remembers which page is in which frame. No external fragmentation (any free frame works), only a little internal fragmentation in the last page.

Logical address = page number + offset. If the page size is 2^k bytes, the last k bits of the address are the **offset** (position inside the page) and the remaining bits are the **page number**.

SOLVED EXAMPLE

32-bit logical address, page size 4 KB. How many entries in the page table?

1. 4 KB = 4 × 1024 = 4096 = 2^{12} bytes → offset needs 12 bits.
2. Page number bits = 32 − 12 = 20.
3. Number of pages = 2^{20}.

✅ 2^{20} = 1,048,576 page-table entries

Segmentation cuts the program into logical, variable-sized parts — code, data, stack — as the programmer sees them. It suffers from external fragmentation.

	Paging	Segmentation
Piece size	Fixed (pages)	Variable (segments)
Fragmentation	Internal	External
Seen by programmer?	No	Yes

TLB (Translation Lookaside Buffer) — looking up the page table in memory every time is slow, so a small fast cache keeps recent page→frame entries. A **TLB hit** saves one memory access.

FORMULAS

Effective access time (EAT)

`h × (t + m) + (1 − h) × (t + 2m) where h = TLB hit ratio, t = TLB time, m = memory time`

SOLVED EXAMPLE

TLB = 20 ns, memory = 100 ns, hit ratio 80%

1. Hit: check TLB (20) + read data (100) = 120 ns.
2. Miss: check TLB (20) + read page table (100) + read data (100) = 220 ns.
3. EAT = 0.8 × 120 + 0.2 × 220 = 96 + 44.

✅ EAT = 140 ns

Virtual memory lets a program run even if it doesn't fully fit in RAM: only the pages being used are kept in RAM; the rest wait on disk. Pages are loaded only when needed — **demand paging**. If a needed page isn't in RAM, a **page fault** happens and the OS brings it from disk (slow).

Page replacement — when RAM is full and a new page is needed, which page do we throw out?

Algorithm	Throws out...
FIFO	The page that came in FIRST (oldest)
LRU (Least Recently Used)	The page not USED for the longest time
Optimal	The page that will not be needed for the longest time in the FUTURE (best possible, but needs to know the future — used only for comparison)

SOLVED EXAMPLE

Reference string 7, 0, 1, 2, 0, 3, 0, 4 with 3 frames — FIFO

1. 7 → fault, frames [7]. 0 → fault [7, 0]. 1 → fault [7, 0, 1].
2. 2 → fault; oldest is 7 → replace → [2, 0, 1].
3. 0 → already there → HIT.
4. 3 → fault; oldest now is 0 → replace → [2, 3, 1].
5. 0 → fault; oldest is 1 → replace → [2, 3, 0].
6. 4 → fault; oldest is 2 → replace → [4, 3, 0].

✅ FIFO: 7 page faults, 1 hit

SOLVED EXAMPLE

Same string with LRU

1. 7, 0, 1 → 3 faults [7, 0, 1].
2. 2 → fault; least recently used is 7 → [2, 0, 1].
3. 0 → hit (0 is now the most recently used).
4. 3 → fault; least recently used is 1 → [2, 0, 3].
5. 0 → hit.
6. 4 → fault; least recently used is 2 → [4, 0, 3].

✅ LRU: 6 page faults (Optimal also gives 6 here)

Belady's anomaly — with FIFO, giving MORE frames can sometimes cause MORE page faults. It never happens with LRU or Optimal.

Thrashing — when processes have too few frames, they page-fault constantly; the CPU spends almost all its time swapping pages instead of doing work. Fix: give processes enough frames (working-set model) or run fewer processes.

Disk scheduling — the disk head moves across cylinders to read requests; we want to reduce total head movement.

Algorithm	Idea
FCFS	Serve requests in arrival order
SSTF	Go to the NEAREST request next (can starve far ones)
SCAN (elevator)	Move one way to the end serving requests, then reverse
C-SCAN	Move one way, jump back to the start, repeat
LOOK / C-LOOK	Like SCAN / C-SCAN but turn at the last request, not the disk end

➖ SOLVED EXAMPLE

SSTF: head at 50; requests 82, 170, 43, 140, 24, 16, 190

1. From 50 the nearest is 43 (7 away). From 43 → 24 (19). From 24 → 16 (8).
2. From 16 the nearest left is 82 (66). Then 140 (58), 170 (30), 190 (20).
3. Total = 7 + 19 + 8 + 66 + 58 + 30 + 20.

✅ **208 cylinders (FCFS would need 642)**

⏱ RECAP IN 30 SECONDS

- ✓ Internal fragmentation = waste inside a block; external = scattered free holes
- ✓ Paging: fixed pages/frames, page table; offset bits = $\log_2$(page size)
- ✓ EAT = $h(t + m) + (1 - h)(t + 2m)$
- ✓ FIFO = oldest out; LRU = least recently used out; Optimal = farthest future use
- ✓ Belady's anomaly only with FIFO; thrashing = too much paging
- ✓ SSTF = nearest request first

✍️ **Quick quiz**

Q1. Belady's anomaly occurs in:

(A) FIFO
(B) LRU
(C) LFU
(D) Optimal

Q2. Page size 1 KB, 16-bit logical address. Bits for the offset:

(A) 10
(B) 1
(C) 6
(D) 16

Q3. Paging suffers from which fragmentation?

(A) Internal
(B) External
(C) Both equally
(D) None

Answers: Q1 A — FIFO is not a stack algorithm. · Q2 A — 1 KB = 2^{10} bytes → 10 offset bits. · Q3 A — The last page of a process may be partly empty.

MODULE 6

DBMS & SQL

Tables, keys, queries, normal forms and transactions — explained with one small school database.

LESSON 6.1 · 16 MIN

What is a database? Tables, keys & the ER model

In one line: **A database is an organised collection of data; a DBMS is the software that stores it, finds it and protects it for many users at once.**

Why not just use files (like Excel sheets)? Imagine the school office, the library and the hostel each keep their own file of students. The same student's address is typed three times (**redundancy**); if it changes in one file but not the others, the data disagrees (**inconsistency**); two clerks editing at once can overwrite each other. A **DBMS** (Database Management System — MySQL, Oracle, PostgreSQL) solves all of this by keeping one shared, controlled copy.

Advantage of a DBMS	Meaning
Less redundancy	Each fact stored once
Consistency & integrity	Rules (constraints) keep data correct
Sharing & concurrency	Many users at once, safely
Security	Each user sees only what they are allowed to
Backup & recovery	Data survives crashes

The relational model stores data in **tables**. In exam language:

Exam word	Everyday word
Relation	Table
Tuple	Row (one record)
Attribute	Column (one field)
Degree	Number of COLUMNS
Cardinality	Number of ROWS
Domain	Set of allowed values for a column (e.g. marks 0–100)

roll	name	dept	marks
1	Asha	CS	82
2	Ravi	IT	67
3	Meena	CS	91
4	Sohan	EC	58
5	Kiran	IT	75

Table student — degree 4 (columns), cardinality 5 (rows). We'll use it in every lesson.

> 🧠 **MEMORY TRICK**
> Degree → D → coluMns? No: remember "Cardinality = Count of rows". Degree is the other one (columns).

Keys — columns that identify rows. Build them up step by step:

Key	Definition	In the student table
Super key	ANY set of columns that uniquely identifies every row	{roll}, {roll, name}, {roll, dept, marks}…
Candidate key	A super key with NO unnecessary column (minimal)	{roll} (and {name} only if names are guaranteed unique)
Primary key	The candidate key we choose; must be unique and never NULL	roll
Alternate key	Candidate keys not chosen as primary	—
Composite key	A key made of two or more columns	{student_id, course_id} in an enrolment table
Foreign key	A column that refers to the primary key of ANOTHER table	dept in student → dept_id in a department table

A foreign key enforces **referential integrity**: you can't give a student a department that doesn't exist in the department table.

Three levels (three-schema architecture): external (what each user sees — views), **conceptual/logical** (all tables and their relationships), **internal/physical** (how data is actually stored on disk). **Data independence** means changing one level doesn't force changes to the level above.

ER (Entity–Relationship) model — a diagram used to design a database before creating tables:

ER symbol	Means	Example
Rectangle	Entity (a thing)	Student, Course
Ellipse	Attribute (property)	name, age
Underlined ellipse	Key attribute	roll
Double ellipse	Multivalued attribute	phone numbers
Dashed ellipse	Derived attribute	age (from date of birth)
Diamond	Relationship	Student ENROLS IN Course
Double rectangle	Weak entity (no key of its own)	Dependent of an employee

Cardinality of relationships: 1:1 (one principal — one school), 1:N (one department — many students), M:N (many students — many courses).

⚠ EXAM TRAP

Degree = number of columns; cardinality = number of rows. A table with 5 columns and 100 rows has degree 5 and cardinality 100 — exams swap them to trick you.

⏱ RECAP IN 30 SECONDS

- ✓ DBMS removes redundancy and inconsistency, allows safe sharing, security, recovery
- ✓ Relation = table, tuple = row, attribute = column; degree = columns, cardinality = rows
- ✓ Super key ⊇ candidate key (minimal) → primary key (chosen, not NULL)
- ✓ Foreign key refers to another table's primary key (referential integrity)
- ✓ ER: rectangle entity, ellipse attribute, diamond relationship

✍️ **Quick quiz**

Q1. A minimal super key is called a:

(A) Foreign key
(B) Composite key
(C) Alternate key
(D) Candidate key

Q2. A table has 6 columns and 40 rows. Its degree is:

(A) 46
(B) 6
(C) 40
(D) 240

Q3. In an ER diagram a relationship is drawn as a:

(A) Rectangle
(B) Diamond
(C) Ellipse
(D) Double ellipse

Answers: Q1 D — Remove any column and it stops being unique. · Q2 B — Degree counts columns. · Q3 B — Diamond = relationship.

LESSON 6.2 · 16 MIN

SQL from zero: creating tables & asking questions

In one line: **SQL reads almost like English: SELECT the columns you want FROM a table WHERE a condition is true.**

SQL (Structured Query Language) is the language for talking to a relational database. Its commands fall into four groups:

Group	Purpose	Commands
DDL — Data Definition	Create/change the STRUCTURE	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML — Data Manipulation	Work with the DATA (rows)	SELECT, INSERT, UPDATE, DELETE
DCL — Data Control	Permissions	GRANT, REVOKE
TCL — Transaction Control	Save or undo changes	COMMIT, ROLLBACK, SAVEPOINT

```sql
CREATE TABLE student (
    roll   INT PRIMARY KEY,
    name   VARCHAR(30) NOT NULL,
    dept   CHAR(2),
    marks  INT CHECK (marks BETWEEN 0 AND 100)
);

INSERT INTO student VALUES (1, 'Asha', 'CS', 82);
```

```
Output ▸ Table created; 1 row inserted
```

PRIMARY KEY, NOT NULL, CHECK, UNIQUE, DEFAULT and FOREIGN KEY are called constraints — rules the DBMS enforces.

Reading data with SELECT. Using our student table (Asha CS 82, Ravi IT 67, Meena CS 91, Sohan EC 58, Kiran IT 75):

```sql
SELECT name, marks
FROM student
WHERE dept = 'CS'
ORDER BY marks DESC;
```

```
Output ▸ Meena 91
Asha  82
```

FROM picks the table, WHERE keeps only CS rows, SELECT keeps two columns, ORDER BY … DESC sorts highest first.

Condition	Meaning	Matches
marks > 70	Greater than 70	Asha, Meena, Kiran
marks BETWEEN 60 AND 80	60 to 80, both ends included	Ravi (67), Kiran (75)
dept IN ('IT', 'EC')	Any value in the list	Ravi, Sohan, Kiran
name LIKE 'A%'	Starts with A (% = any characters)	Asha
name LIKE '_a%'	Second letter is a (_ = exactly one character)	Ravi
dept IS NULL	Value is missing	— (never write = NULL)

```sql
UPDATE student SET marks = 70 WHERE roll = 4;    -- change Sohan's marks
DELETE FROM student WHERE marks < 60;            -- remove rows
```

Output ▸ 1 row updated; 0 rows deleted (Sohan now has 70)

	DELETE	TRUNCATE	DROP
Removes	Selected rows (WHERE allowed)	All rows	The whole table (structure too)
Type	DML	DDL	DDL
Can ROLLBACK?	Yes (before COMMIT)	No	No

Relational algebra is the maths behind SQL. You only need to match the symbols:

Operation	Symbol	Does	SQL equivalent
Selection	σ (sigma)	Picks ROWS that satisfy a condition	WHERE
Projection	π (pi)	Picks COLUMNS	SELECT col1, col2
Union / Intersection / Difference	∪ / ∩ / −	Combine two compatible tables	UNION / INTERSECT / EXCEPT
Cartesian product	×	Every row with every row	FROM A, B
Join	⋈	Product + matching condition	JOIN … ON
Rename	ρ (rho)	Rename a table/column	AS

🧠 **MEMORY TRICK**

σ = Select rows (Sigma, Select, rowS); π = Pick columns.

⏱ **RECAP IN 30 SECONDS**

- ✓ DDL = structure (CREATE, ALTER, DROP, TRUNCATE); DML = data (SELECT, INSERT, UPDATE, DELETE); DCL = GRANT/REVOKE; TCL = COMMIT/ROLLBACK
- ✓ SELECT … FROM … WHERE … ORDER BY …
- ✓ LIKE: % any characters, _ one character; BETWEEN includes both ends; use IS NULL
- ✓ DELETE can be rolled back; TRUNCATE and DROP cannot
- ✓ σ selects rows, π projects columns

✍️ **Quick quiz**

Q1. TRUNCATE belongs to:

(A) TCL
(B) DCL
(C) DML
(D) DDL

Q2. Using the student table, SELECT COUNT(*) FROM student WHERE marks > 70; returns:

(A) 3
(B) 2
(C) 5
(D) 4

Q3. Which relational algebra operation selects rows?

(A) σ
(B) π
(C) ρ
(D) ×

Answers: Q1 D — It works on the table as a whole and auto-commits. · Q2 A — 82, 91 and 75. · Q3 A — σ = selection (rows), π = projection (columns).

LESSON 6.3 · 18 MIN

Joins, GROUP BY, HAVING & subqueries

In one line: **A join glues two tables together on a matching column; GROUP BY folds many rows into groups so you can count, sum or average them.**

Why joins? Data is split across tables to avoid repetition. To answer "which department does each employee work in, by name?" we must combine (join) two tables.

emp	dept_id
Anil	10
Bina	20
Chetan	30

Table emp

dept_id	dname
10	Sales
20	HR
40	IT

Table dept — note: no dept 30, and nobody works in 40

INNER JOIN — only rows that have a match in BOTH tables:

```sql
SELECT e.emp, d.dname
FROM emp e INNER JOIN dept d ON e.dept_id = d.dept_id;
```

```
Output ▸ Anil   Sales
Bina   HR
```

Chetan (30) has no matching department and IT (40) has no employee — both disappear.

LEFT (OUTER) JOIN — ALL rows of the left table, plus matches from the right; where there is no match, the right side is NULL:

```sql
SELECT e.emp, d.dname
FROM emp e LEFT JOIN dept d ON e.dept_id = d.dept_id;
```

```
Output ▸ Anil    Sales
Bina    HR
Chetan  NULL
```

Join	Keeps	Rows here
INNER	Only matching rows	2
LEFT OUTER	All of the left + matches	3 (Chetan–NULL)
RIGHT OUTER	All of the right + matches	3 (NULL–IT)
FULL OUTER	Everything from both sides	4
CROSS	Every left row with every right row	3 × 3 = 9
SELF JOIN	A table joined with itself	e.g. employee → manager

🧠 **MEMORY TRICK**

Draw two overlapping circles (Venn diagram): INNER = only the overlap; LEFT = the whole left circle; RIGHT = the whole right circle; FULL = both circles completely.

Aggregate functions turn many rows into one number: **COUNT, SUM, AVG, MIN, MAX**. **GROUP BY** makes one group per value so you get one result per group.

➖ SOLVED EXAMPLE

Average marks per department (student table)

1. GROUP BY dept makes groups: CS = {82, 91}, IT = {67, 75}, EC = {58}.
2. AVG of each group: CS = (82 + 91) / 2 = 86.5, IT = (67 + 75) / 2 = 71, EC = 58.

✅ CS 86.5 · IT 71 · EC 58

HAVING filters the GROUPS (after grouping); **WHERE** filters the ROWS (before grouping).

SQL

```
SELECT dept, AVG(marks) AS avg_marks
FROM student
GROUP BY dept
HAVING COUNT(*) > 1;
```

```
Output ▸ CS 86.5
IT 71.0
```

EC has only one student, so HAVING removes that group.

Order SQL actually works in	Step
1	FROM (and JOIN)
2	WHERE — filter rows
3	GROUP BY — make groups
4	HAVING — filter groups
5	SELECT — choose columns/aggregates
6	ORDER BY — sort

⚠ EXAM TRAP

WHERE AVG(marks) > 70 is an ERROR — aggregates can't be used in WHERE because grouping hasn't happened yet. Use HAVING AVG(marks) > 70.

COUNT(*) **counts all rows; COUNT(column) **counts only rows where that column is NOT NULL;** COUNT(DISTINCT column)** counts different values.

Subquery — a query inside another query. Example: students who scored above the class average:

```sql
SELECT name FROM student
WHERE marks > (SELECT AVG(marks) FROM student);
```

```
Output ▸ Asha
Meena
Kiran
```

Average = (82 + 67 + 91 + 58 + 75) / 5 = 373 / 5 = 74.6; above it: 82, 91, 75.

⏱ RECAP IN 30 SECONDS

- ✓ INNER = matches only; LEFT/RIGHT keep one whole side with NULLs; FULL keeps both; CROSS = m × n rows
- ✓ GROUP BY makes groups; aggregates: COUNT, SUM, AVG, MIN, MAX
- ✓ WHERE filters rows before grouping; HAVING filters groups after
- ✓ COUNT(*) counts rows; COUNT(col) skips NULLs
- ✓ A subquery is a query inside another query

✍ **Quick quiz**

Q1. Table A has 4 rows and B has 5 rows. A CROSS JOIN B returns:

(A) 5 (B) 4
(C) 9 (D) 20

Q2. With the emp and dept tables above, a LEFT JOIN returns how many rows?

(A) 3 (B) 2
(C) 9 (D) 4

Q3. COUNT(column) differs from COUNT(*) because it:

(A) Counts columns (B) Ignores NULLs
(C) Counts distinct values (D) Is faster

Answers: Q1 D — Every row of A with every row of B: 4 × 5. · Q2 A — All 3 employees; Chetan gets NULL. · Q3 B — COUNT(*) counts every row.

LESSON 6.4 · 22 MIN

Functional dependencies & normalisation (1NF → BCNF)

In one line: **Normalisation splits a messy table into smaller ones so each fact is stored only once — removing update, insert and delete problems.**

Why normalise? Look at this badly designed table:

roll	name	course	teacher	teacher_phone
1	Asha	DBMS	Mr. Rao	98xxx01
1	Asha	OS	Ms. Sen	98xxx02
2	Ravi	DBMS	Mr. Rao	98xxx01

Problem (anomaly)	In this table
Update anomaly	Mr. Rao's phone is stored twice; change one copy and forget the other → data disagrees
Insert anomaly	You can't add a new teacher until some student takes their course
Delete anomaly	Ms. Sen appears only in Asha's OS row; if Asha drops OS and we delete that row, Ms. Sen's phone number is lost too

Functional dependency (FD) X → Y means: if you know X, Y is fixed. “roll → name” (a roll number always has the same name). “teacher → teacher_phone”. The left side is the **determinant**.

Armstrong's axioms — rules for deriving new FDs:

Rule	Statement	Example
Reflexivity	If Y is part of X, then X → Y	{roll, name} → name
Augmentation	If X → Y, then XZ → YZ	roll → name gives {roll, dept} → {name, dept}
Transitivity	If X → Y and Y → Z, then X → Z	course → teacher, teacher → phone ⇒ course → phone
Union (derived)	X → Y and X → Z ⇒ X → YZ	
Decomposition (derived)	X → YZ ⇒ X → Y and X → Z	

Finding a key with closure. The closure X^+ is everything X can determine. If X^+ contains all columns, X is a super key.

➖ SOLVED EXAMPLE

R(A, B, C, D) with A → B, B → C, C → D. Find the candidate key.

1. Start with A^+ = {A}.
2. A → B, so add B: {A, B}. B → C, add C: {A, B, C}. C → D, add D: {A, B, C, D}.
3. A^+ contains every column, and A alone is minimal.

✅ Candidate key = A

Prime attribute = a column that is part of some candidate key. **Non-prime** = not part of any candidate key.

Normal form	Rule in plain words	What it removes
1NF	Every cell holds ONE value (no lists like "DBMS, OS" in one cell)	Repeating groups
2NF	1NF + no non-key column depends on only PART of a composite key	Partial dependency
3NF	2NF + no non-key column depends on another non-key column	Transitive dependency
BCNF	For EVERY FD X → Y, X must be a super key	Any FD whose left side isn't a key
4NF / 5NF	No multivalued / join dependencies	Rarely asked in detail

➖ SOLVED EXAMPLE

Normalise Order(order_id, product_id, qty, product_name, customer_id, customer_city) — key (order_id, product_id)

1. 1NF: assume every cell has one value ✓.
2. 2NF check — partial dependencies: product_name depends only on product_id (part of the key) ✗; customer_id depends only on order_id ✗. Move them out: Product(product_id, product_name), Orders(order_id, customer_id, customer_city), keep OrderLine(order_id, product_id, qty).
3. 3NF check — transitive dependencies: in Orders, order_id → customer_id → customer_city (a non-key column decides another non-key column) ✗. Split: Orders(order_id, customer_id) and Customer(customer_id, customer_city).
4. Now every non-key column depends on the whole key and nothing but the key.

✅ 4 tables in 3NF: OrderLine, Product, Orders, Customer

🧠 **MEMORY TRICK**

"The key (1NF), the whole key (2NF), and nothing but the key (3NF)".

3NF vs BCNF. 3NF allows X → Y when X is a super key OR Y is a prime attribute. BCNF is stricter: X must ALWAYS be a super key. So every BCNF table is in 3NF, but not every 3NF table is in BCNF.

⚠ **EXAM TRAP**

Decomposing into BCNF is always lossless but may lose some dependencies; 3NF decomposition can always be both lossless AND dependency-preserving. And a table with a single-column key is automatically in 2NF (no "part" of the key exists).

⏱ **RECAP IN 30 SECONDS**

- ✓ Anomalies (update, insert, delete) come from storing a fact many times
- ✓ X → Y: knowing X fixes Y; Armstrong: reflexivity, augmentation, transitivity
- ✓ Closure X^+ containing all columns ⇒ X is a super key
- ✓ 1NF atomic; 2NF no partial; 3NF no transitive; BCNF every determinant is a super key
- ✓ BCNF ⊂ 3NF ⊂ 2NF ⊂ 1NF

✍ **Quick quiz**

Q1. A partial dependency violates:

(A) BCNF only
(B) 2NF
(C) 3NF
(D) 1NF

Q2. If A → B and B → C, A → C follows by:

(A) Augmentation
(B) Transitivity
(C) Reflexivity
(D) Union

Q3. Which statement is correct?

(A) Every BCNF relation is in 3NF
(B) 2NF is stricter than 3NF
(C) Every 3NF relation is in BCNF
(D) BCNF always preserves dependencies

Answers: Q1 B — 2NF requires full dependency on the whole key. · Q2 B — Transitivity. · Q3 A — BCNF is the stricter form.

LESSON 6.5 · 18 MIN

Transactions, ACID, concurrency & indexing

In one line: **A transaction is all-or-nothing: in a money transfer, either both the debit and the credit happen, or neither does.**

Transaction = a group of database operations that must be treated as ONE unit. Example: transfer ₹ 500 from A to B = (1) subtract 500 from A, (2) add 500 to B. If the power fails after step 1, money

would vanish — unless the database undoes step 1.

ACID property	Plain meaning	Bank example	Ensured by
Atomicity	All steps or none	Debit without credit never stays	Rollback / undo log
Consistency	Rules stay true before and after	Total money of A + B unchanged	Constraints + correct programs
Isolation	Running together = as if run one by one	Two transfers don't mix up balances	Concurrency control (locks)
Durability	Once committed, it survives crashes	Transfer confirmed → stays done after a restart	Logs / recovery

Problems when transactions run at the same time:

Problem	What happens
Lost update	Two transactions update the same row; one overwrites the other
Dirty read	T2 reads a value T1 changed but hasn't committed; T1 then rolls back → T2 used fake data
Non-repeatable read	T1 reads a row twice and gets different values because T2 changed it in between
Phantom read	T1 runs the same query twice and NEW rows appear (T2 inserted them)

Isolation levels — how strictly the DBMS separates transactions. Stricter = safer but slower:

Isolation level	Dirty read	Non-repeatable read	Phantom
Read uncommitted	possible	possible	possible
Read committed	prevented	possible	possible
Repeatable read	prevented	prevented	possible
Serializable	prevented	prevented	prevented

> 🧠 **MEMORY TRICK**
> Each level up the ladder blocks one more problem: dirty → non-repeatable → phantom.

Locks. A **shared (S) lock** is for reading — many transactions can hold it together. An **exclusive (X) lock** is for writing — no one else may hold any lock on that item. **Two-Phase Locking (2PL):** a

transaction first only ACQUIRES locks (growing phase), then only RELEASES them (shrinking phase). 2PL guarantees serializability but can cause deadlock.

Serializability test: draw a precedence graph (an arrow T1 → T2 if T1 does a conflicting operation on an item before T2). **No cycle = conflict-serializable.**

Indexing — like the index at the back of a book: instead of reading every page, you look up the word and jump to the page. A database index lets the DBMS find rows without scanning the whole table.

Index type	Meaning
Primary / clustered	Rows are physically stored in key order — only one per table
Secondary / non-clustered	A separate sorted list pointing to rows — many allowed
Dense	An index entry for every row
Sparse	An entry for only some rows (e.g. one per block)

B-tree and B+ tree — balanced search trees used for indexes. In a **B+ tree** all actual data pointers are in the **leaf nodes**, and the leaves are linked to each other — so range queries (marks between 60 and 80) are very fast. Most databases use B+ trees.

> ⚠ **EXAM TRAP**
> ROLLBACK cannot undo a transaction that has already been COMMITted. And don't confuse isolation (about concurrent users) with durability (about surviving crashes).

> ⏱ **RECAP IN 30 SECONDS**
> ✓ ACID: Atomicity (all or nothing), Consistency, Isolation, Durability
> ✓ Problems: lost update, dirty read, non-repeatable read, phantom
> ✓ Read uncommitted → read committed → repeatable read → serializable
> ✓ Shared lock = read (many), exclusive lock = write (one); 2PL = grow then shrink
> ✓ B+ tree: data pointers only in linked leaves → fast range queries

✍️ **Quick quiz**

Q1. The isolation level that prevents phantom reads is:

(A) Repeatable read
(B) Read committed
(C) Read uncommitted
(D) Serializable

Q2. Which ACID property is ensured by concurrency control?

(A) Durability
(B) Atomicity
(C) Consistency
(D) Isolation

Q3. In B+ trees, data pointers are stored:

(A) In all nodes
(B) In internal nodes only
(C) Only in leaf nodes
(D) Only in the root

Answers: Q1 D — Only serializable blocks phantoms (in the SQL standard). · Q2 D — Locks keep concurrent transactions isolated. · Q3 C — Internal nodes only guide the search.

MODULE 7

Computer networks & security

How a message travels from your phone to a server and back — explained like sending a parcel by post.

LESSON 7.1 · 14 MIN

Network basics: types, shapes, devices & cables

In one line: **A network is two or more computers connected so they can share data; topology is the shape of the connections; devices direct the traffic.**

What is a computer network? Computers connected (by cable or wireless) so they can share files, printers and the internet. Your home Wi-Fi with a phone, laptop and smart TV is a small network; the **Internet** is a network of millions of networks.

Type	Full form	Covers	Example
PAN	Personal Area Network	A few metres	Phone ↔ Bluetooth earbuds
LAN	Local Area Network	A room, building or campus	School computer lab
MAN	Metropolitan Area Network	A city	Cable TV network of a city
WAN	Wide Area Network	Countries / the world	The Internet, bank networks

Direction of data flow (transmission mode): simplex — one way only (keyboard → computer, TV broadcast); **half-duplex** — both ways but one at a time (walkie-talkie: "over"); **full-duplex** — both ways at the same time (phone call).

Topology = the shape of the connections.

Topology	Shape	Plus	Minus
Bus	All devices on one main cable	Cheap, little cable	If the main cable breaks, the whole network stops
Star	All devices connect to a central hub/switch	Easy to add devices; one cable fault affects only one device	If the centre fails, everything stops
Ring	Each device connects to two neighbours in a circle	Orderly (token passing)	One break can stop the ring
Mesh	Every device connects to every other	Very reliable — many routes	Lots of cable: n(n − 1)/2 links
Tree	Stars connected in a hierarchy	Scalable	Depends on the top link

➖ SOLVED EXAMPLE

How many cables for a full mesh of 8 computers?

1. Each computer connects to the other 7 → 8 × 7 = 56.
2. Each cable joins two computers, so it was counted twice → divide by 2.

✅ 28 cables (n(n − 1)/2)

Device	Job	Works with
Repeater	Boosts a weak signal so it travels further	Signals (layer 1)
Hub	Sends incoming data out to ALL ports (not smart)	Signals (layer 1)
Switch	Sends data only to the port of the right device	MAC addresses (layer 2)
Bridge	Joins two LAN segments	MAC addresses (layer 2)
Router	Connects DIFFERENT networks and picks the best route	IP addresses (layer 3)
Gateway	Connects networks that use different protocols	All layers
Modem	Converts digital ↔ analog (modulator–demodulator)	Phone/cable lines

💡 THINK OF IT LIKE THIS

A hub is a person shouting a message to everyone in the room. A switch is a receptionist who walks the message to the right desk. A router is the post office that decides which city the parcel should go to.

Cables and wireless (transmission media). Guided (wired): **twisted pair** (ordinary LAN cable), **coaxial** (cable TV), **optical fibre** (light signals — fastest, longest distance, not affected by electrical interference). Unguided (wireless): radio waves (Wi-Fi), microwaves, infrared (TV remote), satellite.

> ⚠ **EXAM TRAP**
> A hub forms one collision domain for all its ports; a switch gives every port its own collision domain. A router also separates broadcast domains.

> ⏱ **RECAP IN 30 SECONDS**
> ✓ PAN < LAN < MAN < WAN; the Internet is the biggest WAN
> ✓ Simplex one way, half-duplex one at a time, full-duplex both together
> ✓ Star is the most common topology; full mesh needs n(n − 1)/2 links
> ✓ Hub/repeater layer 1, switch/bridge layer 2 (MAC), router layer 3 (IP)
> ✓ Optical fibre is fastest and immune to electrical interference

✍ **Quick quiz**

Q1. Links needed for a full mesh of 8 computers:

(A) 8 (B) 56
(C) 28 (D) 16

Q2. Which device uses IP addresses to forward data between networks?

(A) Switch (B) Hub
(C) Repeater (D) Router

Q3. A walkie-talkie is an example of:

(A) Simplex (B) Full-duplex
(C) Half-duplex (D) Broadcast only

Answers: Q1 C — 8 × 7 / 2 = 28. · Q2 D — Routers work at layer 3. · Q3 C — Both can talk, but only one at a time.

LESSON 7.2 · 16 MIN

The OSI & TCP/IP models (the 7 layers)

In one line: **Sending data is split into 7 small jobs (layers); each layer does one job and hands the data to the layer below, like departments in a post office.**

Why layers? Sending a message involves many jobs: formatting, encrypting, splitting into pieces, addressing, finding a route, turning bits into electrical signals. The **OSI model** splits these into 7 layers so each can be designed separately.

💡 **THINK OF IT LIKE THIS**

Sending a letter: you write it (application), translate it to a language the receiver reads (presentation), say "this is part of our ongoing conversation" (session), number the pages so they can be put back in order (transport), write the city address (network), the local postman takes it to the next post office (data link), and the truck physically carries it (physical). The receiver's side does the same steps in reverse.

#	Layer	Its job in plain words	Data is called	Examples
7	Application	Services you use directly	Data	HTTP, FTP, SMTP, DNS, Telnet
6	Presentation	Translate format, encrypt, compress	Data	SSL/TLS, JPEG, ASCII
5	Session	Start, manage and end a conversation	Data	RPC, NetBIOS
4	Transport	Split into segments, deliver end-to-end, use port numbers, resend lost pieces	Segment	TCP, UDP
3	Network	Logical (IP) addressing and choosing the route	Packet	IP, ICMP, routers
2	Data link	Deliver to the next device using MAC addresses, detect errors	Frame	Ethernet, switches
1	Physical	Send raw bits as electrical/light/radio signals	Bit	Cables, hubs, repeaters

🧠 **MEMORY TRICK**

Top to bottom (7 → 1): "All People Seem To Need Data Processing". Bottom to top (1 → 7): "Please Do Not Throw Sausage Pizza Away".

Encapsulation: as data goes DOWN the layers, each layer adds its own header (like putting the letter in an envelope, then in a bag, then in a box). The receiver removes them in reverse order going UP.

The TCP/IP model is what the Internet actually uses. It has 4 layers that combine some OSI layers:

TCP/IP layer	Equals these OSI layers
Application	Application + Presentation + Session (7, 6, 5)
Transport	Transport (4)
Internet	Network (3)
Network access / Link	Data link + Physical (2, 1)

TCP vs UDP — the two transport-layer protocols:

	TCP	UDP
Connection	Connection-oriented — sets up a connection first with a 3-way handshake (SYN → SYN-ACK → ACK)	Connectionless — just sends
Reliable?	Yes: acknowledgements, resends lost data, keeps order	No guarantee
Speed	Slower (more checking)	Faster, less overhead
Like...	A registered post with acknowledgement	A postcard
Used for	Web (HTTP), e-mail, file transfer, SSH	Live video, voice calls, online games, DNS queries

⏱ **RECAP IN 30 SECONDS**

- ✓ OSI 7 → 1: Application, Presentation, Session, Transport, Network, Data link, Physical
- ✓ Data units: segment (4), packet (3), frame (2), bit (1)
- ✓ Encryption/compression = presentation layer; routing/IP = network layer
- ✓ TCP/IP has 4 layers: application, transport, internet, link
- ✓ TCP = reliable, connection-oriented (3-way handshake); UDP = fast, connectionless

✍️ **Quick quiz**

Q1. Encryption and compression are done at which OSI layer?

(A) Presentation (B) Session
(C) Transport (D) Application

Q2. The data unit at the transport layer is a:

(A) Segment (B) Packet
(C) Bit (D) Frame

Q3. Which protocol is connectionless?

(A) FTP (B) SMTP
(C) TCP (D) UDP

Answers: Q1 A — Presentation = translation, encryption, compression. · Q2 A — Segment (TCP) / datagram (UDP). · Q3 D — UDP has no handshake.

LESSON 7.3 · 22 MIN

IP addresses & subnetting — fully worked

In one line: **An IPv4 address is 32 bits split into a network part (which street) and a host part (which house); the subnet mask tells you where the split is.**

What is an IP address? A unique number for each device on a network, like a postal address. **IPv4** uses **32 bits**, written as 4 numbers (each 0–255) separated by dots: **192.168.1.10**. Each of the 4 numbers is called an **octet** (8 bits).

Network part vs host part. In 192.168.1.10 with mask 255.255.255.0, the first three octets (192.168.1) identify the **network** (the street) and the last octet (10) identifies the **host** (the house on that street).

Classes (old system, still asked): decided by the first octet.

Class	First octet	Default mask	Network / host split	Use
A	1–126	255.0.0.0 (/8)	8 bits / 24 bits	Huge networks
B	128–191	255.255.0.0 (/16)	16 / 16	Medium networks
C	192–223	255.255.255.0 (/24)	24 / 8	Small networks
D	224–239	—	—	Multicast
E	240–255	—	—	Experimental

127.x.x.x is reserved for loopback — 127.0.0.1 means “this computer itself”

Private addresses (used inside homes/offices, not on the public Internet): 10.0.0.0 – 10.255.255.255, 172.16.0.0 – 172.31.255.255, 192.168.0.0 – 192.168.255.255.

What does "/24" mean? It's the number of 1-bits at the start of the mask. /24 = 24 ones = 11111111.11111111.11111111.00000000 = 255.255.255.0. The remaining **32 − 24 = 8 bits** are for hosts.

Why subtract 2 for hosts? In every network the first address (all host bits 0) is the **network address** and the last (all host bits 1) is the **broadcast address**. Neither can be given to a device.

📐 **FORMULAS**

Host bits `32 - prefix`

Usable hosts per subnet `2^(host bits) - 2`

Number of subnets `2^(bits borrowed)`

Block size (jump between subnets) `256 - (last non-255 number of the mask)`

What is subnetting? Splitting one big network into smaller networks (subnets) by **borrowing** some host bits for the network part — like dividing a big colony into separate lanes.

➖ **SOLVED EXAMPLE**

Divide 192.168.1.0/24 into /26 subnets

1. Borrowed bits = 26 − 24 = 2 → number of subnets = 2^2 = 4.
2. Host bits = 32 − 26 = 6 → usable hosts = 2^6 − 2 = 64 − 2 = 62 per subnet.
3. Mask: 26 ones → the last octet is 11000000 = 128 + 64 = 192 → mask 255.255.255.192.
4. Block size = 256 − 192 = 64 → subnets start at .0, .64, .128, .192.
5. Subnet 1: network 192.168.1.0, usable hosts .1 to .62, broadcast .63.
6. Subnet 2: network .64, hosts .65 to .126, broadcast .127 — and so on.

✅ 4 subnets, each with 62 usable hosts

Prefix	Mask (last octet)	Host bits	Usable hosts
/24	0	8	254
/25	128	7	126
/26	192	6	62
/27	224	5	30
/28	240	4	14
/29	248	3	6
/30	252	2	2

Learn this table — it answers most subnetting questions instantly

MEMORY TRICK

The last-octet mask values add the next bit each time: 128, +64 = 192, +32 = 224, +16 = 240, +8 = 248, +4 = 252.

SOLVED EXAMPLE

Which subnet does 192.168.1.100/27 belong to?

1. /27 → block size = 256 − 224 = 32.
2. Subnets start at 0, 32, 64, 96, 128 …
3. 100 lies between 96 and 127.

✅ **Network 192.168.1.96, broadcast 192.168.1.127, usable .97 – .126**

IPv6 uses **128 bits**, written as 8 groups of hexadecimal (e.g. 2001:0db8::1) — enough addresses for every device on Earth many times over. **MAC address** = 48-bit hardware address burned into the network card (e.g. 00:1A:2B:3C:4D:5E).

⚠ EXAM TRAP

Subtract 2 for usable HOSTS, but never subtract anything for the number of SUBNETS. And a /30 network has exactly 2 usable hosts — used for router-to-router links.

⏱ **RECAP IN 30 SECONDS**

- ✓ IPv4 = 32 bits in 4 octets; IPv6 = 128 bits; MAC = 48 bits
- ✓ Class A 1–126, B 128–191, C 192–223; 127 = loopback
- ✓ Hosts = 2^(32 − prefix) − 2; subnets = 2^(borrowed bits)
- ✓ Block size = 256 − mask octet; subnets start at multiples of it
- ✓ Private: 10/8, 172.16–31, 192.168/16

✍ **Quick quiz**

Q1. Usable hosts in a /28 network:

(A) 14 (B) 16
(C) 28 (D) 30

Q2. The subnet mask for /27 is:

(A) 255.255.255.240 (B) 255.255.255.192
(C) 255.255.255.128 (D) 255.255.255.224

Q3. 172.20.5.9 is a:

(A) Class C private address (B) Multicast address
(C) Class B private address (D) Class A public address

Answers: Q1 A — 4 host bits → 16 − 2 = 14. · Q2 D — 128 + 64 + 32 = 224. · Q3 C — 172.16–172.31 is the Class B private range.

LESSON 7.4 · 14 MIN

Protocols, port numbers, ARP & DNS

In one line: **A protocol is an agreed set of rules for communication; a port number tells which program on the computer should receive the data.**

Protocol = a set of rules both sides agree on, like the rule that in a phone call you say “hello” first. Each Internet service has its own protocol.

Port number = which program on a computer the data is for. The IP address finds the building; the port finds the flat. Web servers listen on port 80/443, e-mail on 25, and so on. Ports are 16-bit numbers: 0–65535; 0–1023 are “well-known” ports.

Protocol	Port	What it does
FTP	20 (data), 21 (control)	File transfer
SSH	22	Secure remote login
Telnet	23	Remote login (not encrypted)
SMTP	25	SENDS e-mail
DNS	53	Converts names (google.com) to IP addresses
DHCP	67 / 68	Gives devices an IP address automatically
HTTP	80	Web pages
POP3	110	Downloads e-mail to your device
IMAP	143	Reads e-mail kept on the server
SNMP	161	Network management
HTTPS	443	Secure web (HTTP + TLS encryption)

> 🧠 **MEMORY TRICK**
> "SMTP Sends Mail To People" — SMTP sends; POP and IMAP receive.

DNS (Domain Name System) — the phonebook of the Internet. You type `bpsc.bihar.gov.in`; DNS finds its IP address so your computer can connect. Lookup goes root server → top-level domain server (.in) → authoritative server for that domain.

DHCP — when your phone joins Wi-Fi, the router's DHCP server automatically gives it an IP address, mask, gateway and DNS server. No manual setup needed.

ARP (Address Resolution Protocol) — inside a LAN, data is delivered using MAC addresses. If a computer knows the IP address 192.168.1.5 but not its MAC address, it broadcasts "Who has 192.168.1.5?" and that device replies with its MAC. So **ARP: IP → MAC**. **RARP** does the reverse: MAC → IP.

ICMP carries error and diagnostic messages; the `ping` command (is the other computer reachable?) and `traceroute` use it. **NAT** lets many devices with private IPs share one public IP address.

⏱ **RECAP IN 30 SECONDS**

✓ Protocol = agreed rules; port = which program receives the data

✓ HTTP 80, HTTPS 443, FTP 20/21, SSH 22, Telnet 23, SMTP 25, DNS 53, POP3 110, IMAP 143

✓ DNS: name → IP; DHCP: automatic IP; ARP: IP → MAC; RARP: MAC → IP

✓ ping uses ICMP; NAT shares one public IP

✍ **Quick quiz**

Q1. Which protocol finds a MAC address from an IP address?

(A) RARP
(B) DNS
(C) ARP
(D) DHCP

Q2. The default port for SMTP is:

(A) 143
(B) 25
(C) 110
(D) 21

Q3. ping uses which protocol?

(A) ICMP
(B) ARP
(C) TCP
(D) UDP

Answers: Q1 C — Address Resolution Protocol. · Q2 B — SMTP 25, POP3 110, IMAP 143. · Q3 A — Echo request/reply messages.

LESSON 7.5 · 20 MIN

Error control, who-may-talk rules, flow control & network security

In one line: **The data link layer checks every frame for errors and decides who may send; security makes sure only the right person can read and trust the data.**

1. Detecting errors. Noise can flip a bit (a 0 becomes a 1). The sender adds extra check bits so the receiver can notice.

Method	How it works
Parity bit	Add one bit so the number of 1s is even (even parity) — detects any single-bit error
Checksum	Add up the data blocks and send the total; receiver re-adds and compares
CRC (cyclic redundancy check)	Divide the data by a fixed binary number and send the remainder — the strongest, used in Ethernet
Hamming code	Extra bits at positions 1, 2, 4, 8 … — can CORRECT a single-bit error, not just detect it

➖ SOLVED EXAMPLE

Even parity for the data 1011001

1. Count the 1s: 1 + 0 + 1 + 1 + 0 + 0 + 1 = 4.
2. 4 is already even, so the parity bit is 0.
3. If one bit flips on the way, the receiver counts an odd number of 1s → error detected.

✅ Sent: 1011001 + parity 0

📐 FORMULAS

To DETECT up to d errors `Hamming distance ≥ d + 1`

To CORRECT up to d errors `Hamming distance ≥ 2d + 1`

2. Who may talk? (media access control). On a shared cable, if two computers send at once, the signals collide.

	CSMA/CD	CSMA/CA
Full form	Carrier Sense Multiple Access with Collision DETECTION	... with Collision AVOIDANCE
Used in	Wired Ethernet (IEEE 802.3)	Wi-Fi (IEEE 802.11)
How	Listen; if quiet, send; if a collision is detected, stop, send a jam signal, wait a random time, try again	Listen; wait a random time even if quiet; may reserve the channel with RTS/CTS before sending
Why	On a wire you can detect a collision while sending	Wireless devices can't hear a collision while they transmit

💡 THINK OF IT LIKE THIS

CSMA/CD = people in a meeting: listen before speaking; if two start together, both stop and wait a random moment. CSMA/CA = raising your hand and waiting to be allowed to speak.

3. Flow control — making sure a fast sender doesn't drown a slow receiver. **Stop-and-wait**: send one frame, wait for its acknowledgement (ACK), then send the next — simple but slow. **Sliding window**: send several frames (a "window") before waiting for ACKs — much faster.

Sliding-window protocol	If a frame is lost	Maximum sender window (m-bit sequence numbers)
Go-Back-N	Resend that frame AND all frames after it	$2^m - 1$
Selective Repeat	Resend ONLY the lost frame	2^{m-1}

SOLVED EXAMPLE

3-bit sequence numbers — window sizes

1. Sequence numbers 0 to 7 ($2^3 = 8$ numbers).
2. Go-Back-N maximum window = $2^3 - 1 = 7$.
3. Selective Repeat maximum window = $2^{3-1} = 4$.

Go-Back-N 7, Selective Repeat 4

4. Network security. The three goals (**CIA triad**): **Confidentiality** (only the right person can read it), **Integrity** (nobody changed it), **Availability** (it works when needed).

Encryption scrambles data with a **key** so only someone with the right key can read it.

	Symmetric	Asymmetric (public-key)
Keys	ONE shared secret key for locking and unlocking	A key PAIR: a public key (shared with everyone) and a private key (kept secret)
Speed	Fast	Slow
Problem	How do you share the secret key safely?	Solves key sharing
Examples	DES, 3DES, AES	RSA, Diffie–Hellman, ECC

How to use the two keys — the most-asked question:

Goal	Sender uses	Receiver uses
Secret message to Ravi (confidentiality)	RAVI's PUBLIC key to encrypt	Ravi's PRIVATE key to decrypt
Digital signature (prove it's from Asha, not changed)	ASHA's PRIVATE key to sign	Asha's PUBLIC key to verify

💡 **THINK OF IT LIKE THIS**

Ravi's public key is like an open padlock he hands out to everyone: anyone can lock a box with it, but only Ravi has the key to open it. A signature is the reverse: only Asha can stamp with her private seal, but everyone can check the stamp.

- Hash function (SHA-256, MD5): makes a fixed-size fingerprint of data — any change gives a totally different hash (used for integrity and passwords)
- Firewall: filters network traffic by rules; IDS detects intrusions; VPN creates an encrypted tunnel over the Internet
- Malware: virus (attaches to files, needs you to run it), worm (spreads by itself), Trojan (looks useful but is harmful), ransomware (locks files for money), spyware
- Attacks: phishing (fake e-mails/sites to steal passwords), DoS/DDoS (flooding a server), man-in-the-middle, SQL injection

⚠ **EXAM TRAP**

Signing uses the SENDER's private key; secrecy uses the RECEIVER's public key. Mixing these up is the most common mistake.

⏱ **RECAP IN 30 SECONDS**

- ✓ Parity/checksum/CRC detect errors; Hamming code corrects single-bit errors
- ✓ CSMA/CD = wired Ethernet; CSMA/CA = Wi-Fi
- ✓ Go-Back-N window $2^m - 1$ (resend from the lost frame); Selective Repeat 2^{m-1} (resend only the lost one)
- ✓ Symmetric = one key (AES); asymmetric = public + private (RSA)
- ✓ Secrecy: receiver's public key; signature: sender's private key

✍️ **Quick quiz**

Q1. Wi-Fi networks use:

(A) CSMA/CA
(B) Token passing
(C) CSMA/CD
(D) ALOHA only

Q2. A digital signature is created with the:

(A) Shared secret key
(B) Sender's public key
(C) Sender's private key
(D) Receiver's public key

Q3. With a 3-bit sequence number, the maximum Go-Back-N sender window is:

(A) 8
(B) 4
(C) 3
(D) 7

Answers: Q1 A — Collisions can't be detected while transmitting wirelessly. · Q2 C — Only the sender has it, so it proves who sent the message. · Q3 D — $2^3 - 1 = 7$.

MODULE 8

Computer organisation & architecture

What happens inside the CPU and memory when a program runs — explained like a kitchen.

LESSON 8.1 · 16 MIN

Inside the CPU: parts, registers & the instruction cycle

In one line: **The CPU repeats one simple loop billions of times a second: fetch an instruction from memory, decode it, execute it.**

The basic parts of a computer: input devices (keyboard, mouse), **output** devices (monitor, printer), **memory** (stores programs and data), and the **CPU (Central Processing Unit)** — the "brain" that actually runs instructions.

The CPU itself has three parts:

Part	Job
ALU (Arithmetic Logic Unit)	Does the calculations (+ − × ÷) and comparisons/logic (AND, OR, >)
CU (Control Unit)	Reads each instruction, works out what it means, and tells the other parts what to do — the manager
Registers	Tiny, super-fast storage boxes INSIDE the CPU for the values being worked on right now

💡 **THINK OF IT LIKE THIS**

The CPU is a chef in a kitchen. The control unit reads the next line of the recipe and decides what to do; the ALU is the chef's hands doing the chopping and mixing; registers are the few things the chef is holding right now; main memory (RAM) is the fridge — bigger but a walk away.

Important registers — exams ask what each one holds:

Register	Holds
PC — Program Counter	The ADDRESS of the NEXT instruction to fetch
IR — Instruction Register	The instruction being executed RIGHT NOW
MAR — Memory Address Register	The address the CPU wants to read from / write to
MDR (MBR) — Memory Data/Buffer Register	The data just read from memory, or about to be written
Accumulator (AC)	The result of the ALU's latest calculation
Flag / Status register	Single bits telling what happened: Zero, Carry, Sign, Overflow

The instruction cycle (fetch – decode – execute) — this is what a CPU does all day:

➖ **SOLVED EXAMPLE**

One trip round the instruction cycle

1. FETCH: copy the address in the PC into the MAR (MAR ← PC).
2. Read memory at that address; the instruction arrives in the MDR (MDR ← Memory[MAR]).
3. Copy it into the IR (IR ← MDR), and add 1 to the PC so it points to the next instruction (PC ← PC + 1).
4. DECODE: the control unit looks at the IR and works out what operation it is (e.g. ADD) and where the data is.
5. EXECUTE: the ALU does the work (or data is moved, or the PC changes for a jump). Then the cycle starts again.

✅ Fetch → Decode → Execute → (repeat)

Buses — the roads that connect the CPU, memory and devices: the **address bus** carries addresses (one-way: CPU → memory), the **data bus** carries data (both ways), the **control bus** carries signals like read/write.

📐 **FORMULAS**

An address bus with n lines can address 2^n memory locations

➖ SOLVED EXAMPLE

How many locations can a 16-bit address bus reach?

1. n = 16.
2. 2^{16} = 65,536.

✅ **65,536 locations (64 K)**

	Von Neumann	Harvard
Memory	ONE memory for both program and data	SEPARATE memories for program and data
Speed	Slower (one shared path — the "von Neumann bottleneck")	Faster (fetch instruction and data together)
Used in	Most PCs	Microcontrollers, DSPs

	RISC	CISC
Full form	Reduced Instruction Set Computer	Complex Instruction Set Computer
Instructions	Few, simple, fixed length, one per clock cycle	Many, complex, variable length
Registers	Many	Fewer
Example	ARM (phones)	Intel x86 (PCs)

Generation	Technology	Example
1st (1940–56)	Vacuum tubes	ENIAC, UNIVAC
2nd (1956–63)	Transistors	IBM 1401
3rd (1964–71)	Integrated circuits (ICs)	IBM 360
4th (1971–now)	Microprocessors (VLSI)	Personal computers
5th (now/future)	AI, ULSI, parallel processing	AI systems

⚠ EXAM TRAP

The PC holds an ADDRESS (of the next instruction), not the instruction itself — that's the IR.

⏱ **RECAP IN 30 SECONDS**

- ✓ CPU = ALU (calculates) + CU (controls) + registers (fast storage)
- ✓ PC = address of next instruction; IR = current instruction; MAR = address; MDR = data
- ✓ Instruction cycle: fetch → decode → execute
- ✓ n address lines → 2^n locations
- ✓ Von Neumann = one memory; Harvard = separate; RISC = simple instructions

✍ **Quick quiz**

Q1. A 16-bit address bus can address how many locations?

(A) 65,536 (B) 256
(C) 16 (D) 1,024

Q2. Which register holds the instruction currently being executed?

(A) MDR (B) IR
(C) PC (D) MAR

Q3. Integrated circuits belong to which computer generation?

(A) Second (B) Third
(C) Fourth (D) First

Answers: Q1 A — 2^{16} = 65,536 (64 K). · Q2 B — Instruction Register. · Q3 B — Tubes → transistors → ICs → microprocessors.

LESSON 8.2 · 14 MIN

Instructions & addressing modes

In one line: **Every instruction says WHAT to do (the opcode) and WHERE the data is; the addressing mode is the rule for finding that data.**

An instruction has two parts: the **opcode** (what to do — ADD, MOV, JUMP) and the **operand / address field** (what to do it to). With n bits for the opcode, the CPU can have 2^n different instructions.

Addressing mode = the rule for finding the actual data (the operand). Think of it as different ways of telling a friend where the sweets are:

Mode	Where is the data?	Example	Like saying…
Immediate	Right inside the instruction	MOV A, #5 (put 5 in A)	"Here are the sweets, in my hand."
Register	In a CPU register	MOV A, B	"They're in your pocket."
Direct	At the memory address written in the instruction	MOV A, [2000]	"They're in house no. 2000."
Indirect	At the address that is STORED at the given address	MOV A, [[2000]]	"House no. 2000 has a note telling you the real house."
Register indirect	At the address held in a register	MOV A, [R1]	"The house number is written in your pocket diary."
Indexed / base	At (base address + index) — great for arrays	MOV A, [R1 + 4]	"Fourth house after house R1."
Relative	At (PC + offset) — used for jumps/branches	JMP +8	"Eight houses ahead of where you're standing."
Implied	Obvious from the instruction itself	CMA (complement the accumulator)	No address needed

SOLVED EXAMPLE

How many memory reads to get the operand?

1. Immediate: the data is in the instruction → 0 extra memory reads.
2. Direct: read memory once at the given address → 1.
3. Indirect: read the address first, then read the data at that address → 2.

✅ Immediate 0, direct 1, indirect 2 — immediate is fastest, indirect is slowest

Instruction formats by number of addresses: 3-address (ADD R1, R2, R3 → R1 = R2 + R3), 2-address (ADD R1, R2 → R1 = R1 + R2), 1-address (uses the accumulator: ADD X → AC = AC + X), 0-address (stack machines: PUSH, PUSH, ADD — operands come from the stack).

MEMORY TRICK

Immediate = "I have it right here." Direct = "It's at this house." Indirect = "This house has a note with the real house."

⏱ **RECAP IN 30 SECONDS**

- ✓ Instruction = opcode (what) + operand/address (where); n opcode bits → 2^n instructions
- ✓ Immediate: data in the instruction; direct: address given; indirect: address of the address
- ✓ Indexed for arrays; relative (PC + offset) for branches
- ✓ Memory reads for the operand: immediate 0, direct 1, indirect 2

✍ **Quick quiz**

Q1. Which mode needs two memory accesses to fetch the operand?

(A) Immediate
(B) Indirect
(C) Direct
(D) Register

Q2. Relative addressing is mainly used for:

(A) Constants
(B) Branch/jump instructions
(C) I/O
(D) Arrays

Q3. MOV A, #25 uses which addressing mode?

(A) Indirect
(B) Direct
(C) Register
(D) Immediate

Answers: Q1 B — One to read the address, one to read the data. · Q2 B — Target = PC + offset. · Q3 D — # means the value 25 itself is in the instruction.

LESSON 8.3 · 20 MIN

Memory hierarchy, cache, pipelining & I/O

In one line: **Keep what you use most in the smallest, fastest memory — that's why computers have registers, cache, RAM and disk at different speeds.**

The memory hierarchy. Fast memory is expensive and small; cheap memory is big but slow. So computers use layers:

```
Registers        ← fastest, smallest, costliest (inside the CPU)
Cache (L1, L2, L3)
Main memory (RAM)
Secondary storage (SSD / hard disk)
Tape / offline backup ← slowest, biggest, cheapest per bit
```

Going down: bigger, slower, cheaper

💡 **THINK OF IT LIKE THIS**

Your desk (registers/cache) holds the few books you're using now; the bookshelf in your room (RAM) holds more; the library across town (disk) holds everything but takes a trip.

Why does cache work? Locality of reference. Programs tend to reuse the same data soon (**temporal locality** — a loop counter) and use data stored next to what they just used (**spatial locality** — the next element of an array). So keeping recently used data and its neighbours in cache pays off.

Hit and miss. If the data is found in cache it's a **hit**; if not, a **miss** (go to RAM). **Hit ratio** h = hits ÷ total accesses.

📐 **FORMULAS**

Average access time (simple model) `h × cache time + (1 − h) × memory time`

➖ **SOLVED EXAMPLE**

Cache = 10 ns, memory = 100 ns, hit ratio = 0.9

1. 90% of accesses are hits: 0.9 × 10 = 9 ns.
2. 10% are misses: 0.1 × 100 = 10 ns.
3. Add them: 9 + 10.

✅ Average access time = 19 ns (instead of 100 ns without cache)

⚠ **EXAM TRAP**

Some questions use the “hierarchical” model, where a miss costs cache time + memory time: average = 10 + 0.1 × 100 = 20 ns. Read the question for “simultaneous” or “hierarchical” access.

- Cache mapping: direct (each memory block has exactly one possible cache line), fully associative (any line), set-associative (any line within one set)
- Write-through: every write also goes to RAM immediately; write-back: RAM is updated only when the line is replaced (uses a “dirty bit”)
- SRAM (made of flip-flops) — fast, used for cache; DRAM (capacitors) — must be refreshed regularly, used for main memory
- ROM types: PROM (write once), EPROM (erase with UV light), EEPROM (erase electrically), flash memory (pen drives, SSDs)

Pipelining — overlapping the steps of different instructions, like a factory assembly line. While instruction 1 is executing, instruction 2 is being decoded and instruction 3 is being fetched.

💡 **THINK OF IT LIKE THIS**

Washing clothes: wash, dry, fold. Without a pipeline you finish load 1 completely before starting load 2. With a pipeline, while load 1 is drying, load 2 is already washing.

📐 **FORMULAS**

Pipeline time for n instructions, k stages `(k + n − 1) cycles`

Speed-up `n·k / (k + n − 1) → approaches k for large n`

➖ **SOLVED EXAMPLE**

4-stage pipeline, 10 instructions

1. Without pipeline: 10 × 4 = 40 cycles.
2. With pipeline: k + n − 1 = 4 + 10 − 1 = 13 cycles.
3. Speed-up = 40 / 13 ≈ 3.08.

✅ 13 cycles, about 3× faster

Pipeline hazard	Cause	Fix
Structural	Two instructions need the same hardware at the same time	Add hardware, or stall (wait)
Data	An instruction needs a result that isn't ready yet	Forwarding (bypassing), stalls
Control	A branch/jump — the next instruction isn't known yet	Branch prediction, delayed branch

Input/output (I/O) techniques: programmed I/O — the CPU keeps checking whether the device is ready (busy-waiting, wastes time); **interrupt-driven I/O** — the device sends an **interrupt** when ready, the CPU pauses, runs an Interrupt Service Routine (ISR), then resumes; **DMA (Direct Memory Access)** — a DMA controller moves a whole block of data between the device and memory without the CPU copying each word.

⏱ **RECAP IN 30 SECONDS**

- ✓ Registers → cache → RAM → disk: faster/smaller/costlier at the top
- ✓ Cache works because of temporal and spatial locality
- ✓ Average time = h × cache + (1 − h) × memory
- ✓ Pipeline: k + n − 1 cycles; hazards: structural, data, control
- ✓ DMA moves blocks without the CPU; interrupts avoid busy-waiting

✍️ **Quick quiz**

Q1. Which memory must be refreshed periodically?

(A) ROM
(B) Cache
(C) SRAM
(D) DRAM

Q2. A branch instruction in a pipeline causes a:

(A) No hazard
(B) Data hazard
(C) Structural hazard
(D) Control hazard

Q3. A 5-stage pipeline runs 20 instructions in how many cycles?

(A) 24
(B) 25
(C) 100
(D) 20

Answers: Q1 D — DRAM stores bits as charge on capacitors that leaks away. · Q2 D — The next address is unknown until the branch is resolved. · Q3 A — $k + n - 1 = 5 + 20 - 1 = 24$.

MODULE 9

OOP with C++

Classes, objects, inheritance and polymorphism — from zero to the exact output questions exams ask.

LESSON 9.1 · 16 MIN

Classes, objects & the four pillars of OOP

In one line: **A class is a blueprint; an object is a real thing built from it — and OOP organises programs around such objects.**

Why object-oriented programming (OOP)? In C we write functions and pass data around. In big programs that gets messy. OOP groups the **data** and the **functions that work on it** together into one unit called an **object** — like real-world things that have properties and can do things.

> 💡 **THINK OF IT LIKE THIS**
>
> A class is the architect's drawing of a house: it says every house has rooms, a door and a colour. An object is an actual house built from that drawing. From one drawing you can build many houses, each with its own colour.

CPP

```cpp
class Student {            // the blueprint
public:
    string name;           // data (attributes)
    int marks;
    void show() {          // function (method)
        cout << name << " " << marks;
    }
};

int main() {
    Student s1;            // an object made from the class
    s1.name = "Asha";
    s1.marks = 82;
    s1.show();
}
```

Output ▸ Asha 82

The four pillars of OOP:

Pillar	Plain meaning	Everyday example	In C++
Encapsulation	Wrap data + functions together and hide the inside	A capsule hides the medicine inside	private data, public functions
Abstraction	Show WHAT it does, hide HOW	You drive a car without knowing how the engine works	Abstract classes, interfaces
Inheritance	A new class reuses an existing class	A child inherits features from parents	class Car : public Vehicle
Polymorphism	One name, many forms	The word "run" — a person runs, a program runs, a tap runs	Function overloading, virtual functions

> 🧠 **MEMORY TRICK**
> "A PIE" — Abstraction, Polymorphism, Inheritance, Encapsulation.

Access specifiers — who can see a member:

Specifier	Same class	Derived (child) class	Outside the class
private	✓	✗	✗
protected	✓	✓	✗
public	✓	✓	✓

In a class, members are private by default; in a struct, they are public by default

Inheritance types: single (A → B), multilevel (A → B → C), multiple (C inherits from both A and B — allowed in C++, not for classes in Java), hierarchical (one parent, many children), hybrid (a mix). The **diamond problem** in multiple inheritance (two copies of the same grandparent) is solved with **virtual base classes**.

- Friend function: not a member, but allowed to access private members
- Static member: one copy shared by ALL objects of the class (e.g. a counter of how many students exist)
- this pointer: inside a member function, points to the object that called it
- Inline function: the compiler pastes the code at the call site to save call time

⏱ **RECAP IN 30 SECONDS**

- ✓ Class = blueprint; object = an instance built from it
- ✓ Encapsulation (hide data), Abstraction (show what, hide how), Inheritance (reuse), Polymorphism (many forms)
- ✓ private → only the class; protected → class + children; public → everyone
- ✓ class members private by default; struct members public by default
- ✓ C++ allows multiple inheritance; Java doesn't (for classes)

✍️ **Quick quiz**

Q1. Hiding internal details and exposing only an interface is:

(A) Recursion
(B) Abstraction
(C) Overloading
(D) Inheritance

Q2. Default access of class members in C++ is:

(A) public
(B) private
(C) protected
(D) friend

Q3. Which language allows multiple inheritance of classes?

(A) Java
(B) None
(C) C++
(D) C#

Answers: Q1 B — Abstraction shows what, hides how. · Q2 B — struct members default to public. · Q3 C — Java and C# allow it only through interfaces.

LESSON 9.2 · 15 MIN

Constructors, destructors, overloading & overriding

In one line: **A constructor sets up a new object automatically; a destructor cleans up when it's destroyed — and the order matters in inheritance.**

Constructor — a special function that runs AUTOMATICALLY when an object is created, to give it starting values. Rules: same name as the class, **no return type** (not even void).

CPP

```cpp
class Box {
public:
    int len;
    Box()        { len = 1; }   // default constructor
    Box(int l)   { len = l; }   // parameterised constructor
};

Box a;        // len = 1
Box b(5);     // len = 5
```

Output ▸ `a.len = 1, b.len = 5`

Type of constructor	Meaning
Default	Takes no arguments
Parameterised	Takes arguments to set values
Copy	Makes a new object as a copy of an existing one: Box c(b);

Constructors CAN be overloaded (several with different parameters)

Destructor — runs automatically when an object is destroyed (goes out of scope or is deleted), to free memory or close files. Written `~ClassName()`, takes **no parameters**, so it **cannot be overloaded** — one per class.

Order in inheritance: when you create a child object, the PARENT's constructor runs FIRST (the foundation before the floors), then the child's. Destructors run in the REVERSE order (child first, then parent — like dismantling from the top).

CPP

```cpp
class A { public: A(){ cout << "A "; }  ~A(){ cout << "~A "; } };
class B : public A { public: B(){ cout << "B "; } ~B(){ cout << "~B "; } };

int main() { B obj; }    // obj is created, then destroyed at the end of main
```

Output ▸ `A B ~B ~A`

Overloading vs overriding — very often confused:

	Function overloading	Function overriding
What	Same function name, DIFFERENT parameters	Child class REDEFINES a parent's function with the SAME name and parameters
Where	In the same class/scope	Between parent and child classes
Decided	At compile time (static binding)	At run time when the function is virtual (dynamic binding)
Polymorphism type	Compile-time	Run-time
Example	add(int, int) and add(double, double)	Animal::sound() and Dog::sound()

CPP

```
int    add(int a, int b)          { return a + b; }
double add(double a, double b)    { return a + b; }
int    add(int a, int b, int c)   { return a + b + c; }
// add(2, 3) → 5,  add(2.5, 1.5) → 4.0,  add(1, 2, 3) → 6
```

Output ▸ The compiler picks the version by the arguments

> ⚠ **EXAM TRAP**
> Changing ONLY the return type is not overloading — int f(int) and double f(int) together give a compile error.

Operator overloading lets operators work with your own classes, e.g. `+` adding two Complex numbers. Operators that can NOT be overloaded: `::` (scope), `.` (member access), `.*`, `?:` (conditional), `sizeof`.

> ⏱ **RECAP IN 30 SECONDS**
> ✓ Constructor: same name as class, no return type, runs on creation, can be overloaded
> ✓ Destructor: ~Name(), no parameters, can't be overloaded
> ✓ Construction: parent → child; destruction: child → parent
> ✓ Overloading = same name, different parameters (compile time); overriding = same signature in child (run time)
> ✓ Can't overload: :: . .* ?: sizeof

✍️ **Quick quiz**

Q1. In C++, which operator cannot be overloaded?

(A) ::
(B) []
(C) ==
(D) +

Q2. Derived class D inherits base B. Destructors run in the order:

(A) Only D
(B) D then B
(C) B then D
(D) Only B

Q3. A constructor's return type is:

(A) int
(B) void
(C) none — it has no return type
(D) the class type

Answers: Q1 A — Scope resolution, dot, .*, ?: and sizeof cannot be overloaded. · Q2 B — Reverse of construction. · Q3 C — Constructors never declare a return type.

LESSON 9.3 · 16 MIN

Virtual functions, abstract classes & virtual destructors

In one line: **virtual means: decide which function to run at run time, by the REAL type of the object — not by the type of the pointer.**

A base-class pointer can point to a child object. `Shape *s = new Circle();` is allowed because a Circle IS a Shape. The question is: when we call `s->draw()`, which draw() runs — Shape's or Circle's?

CPP

```
class Shape { public: void draw() { cout << "Shape"; } };
class Circle : public Shape { public: void draw() { cout << "Circle"; } };

Shape *s = new Circle();
s->draw();
```

Output ▸ Shape

Without virtual, C++ looks only at the POINTER's type (Shape*) — decided at compile time (static binding).

CPP

```cpp
class Shape { public: virtual void draw() { cout << "Shape"; } };
class Circle : public Shape { public: void draw() override { cout << "Circle"; } };

Shape *s = new Circle();
s->draw();
```

Output ▸ Circle

With virtual, C++ looks at the real OBJECT (a Circle) — decided at run time (dynamic binding). This is run-time polymorphism.

> 💡 **THINK OF IT LIKE THIS**
> A TV remote (the pointer) says “Power ON”. Without virtual, it would always switch on a generic TV. With virtual, it switches on whatever brand of TV is actually in front of it.

How does C++ do it? Every class with virtual functions gets a hidden table of function addresses — the **vtable**. Every object of that class carries a hidden pointer to its class's vtable — the **vptr**. A virtual call follows the vptr to find the right function.

Pure virtual function & abstract class. Sometimes the parent can't sensibly define a function at all — what's the area of a general “Shape”? Mark it **pure virtual** with `= 0`:

CPP

```cpp
class Shape {
public:
    virtual double area() = 0;   // pure virtual: no body here
};

class Square : public Shape {
    double side = 2;
public:
    double area() override { return side * side; }
};

// Shape s;            // ERROR: cannot create an object of an abstract class
Shape *p = new Square(); // OK: pointer to the abstract class
cout << p->area();
```

Output ▸ 4

- A class with at least ONE pure virtual function is an ABSTRACT class
- You cannot create objects of an abstract class, but you can have pointers/references to it

- A child must override every pure virtual function, otherwise the child is abstract too
- Constructors can NOT be virtual; destructors CAN (and should be, in a base class)

Why a virtual destructor? If you delete a child object through a parent pointer and the parent's destructor is NOT virtual, only the parent's destructor runs — the child's clean-up is skipped (memory leak / undefined behaviour).

CPP

```
class Base    { public: ~Base()    { cout << "~Base "; } };
class Derived : public Base { public: ~Derived() { cout << "~Derived "; } };

Base *p = new Derived();
delete p;
```

Output ▸ ~Base

The Derived part is never cleaned up. Write virtual ~Base() and the output becomes "~Derived ~Base" — both run, child first.

🧠 **MEMORY TRICK**

Rule of thumb: if a class has ANY virtual function, give it a virtual destructor.

⏱ **RECAP IN 30 SECONDS**

- ✓ Without virtual: pointer type decides (compile time). With virtual: object type decides (run time)
- ✓ vtable per class, vptr per object
- ✓ Pure virtual = 0 → abstract class → no objects, only pointers/references
- ✓ Constructors can't be virtual; base destructors should be virtual

✍️ **Quick quiz**

Q1. A class with a pure virtual function is called:

(A) Sealed class
(B) Friend class
(C) Virtual class
(D) Abstract class

Q2. Runtime polymorphism in C++ requires:

(A) Function overloading
(B) Virtual functions called through base pointers/references
(C) Inline functions
(D) Templates

Q3. Why make a base-class destructor virtual?

(A) To make objects smaller
(B) So delete through a base pointer also runs the derived destructor
(C) To allow constructor overloading
(D) It is required by the compiler

Answers: Q1 D — It cannot be instantiated. · Q2 B — Dynamic binding through the vtable. · Q3 B — Otherwise only the base destructor runs.

MODULE 10

Software engineering

How real software is planned, built, tested and measured — explained like building a house.

LESSON 10.1 · 15 MIN

SDLC & software development models

In one line: **Every model follows the same six steps — plan, design, build, test, deliver, maintain — they only differ in how often and in what order you go round them.**

What is software engineering? Writing a 50-line program is easy. Building banking software with 10 lakh lines, 100 programmers and a deadline needs a disciplined process — that process is software engineering.

SDLC (Software Development Life Cycle) — the stages every software project goes through:

Stage	Question it answers	House-building comparison
1. Requirements (analysis)	WHAT should the software do?	How many rooms does the family want?
2. Design	HOW will it be built (modules, database, screens)?	The architect's drawing
3. Coding (implementation)	Write the program	Building the walls
4. Testing	Does it work correctly?	Inspection before handing over
5. Deployment	Install it for users	Family moves in
6. Maintenance	Fix bugs, add features, adapt to changes	Repairs and new rooms over the years

Maintenance usually costs the most over a product's whole life

The **SRS (Software Requirements Specification)** document lists **functional requirements** (what the system does — "students can download admit cards") and **non-functional requirements** (how well — speed, security, usability). A **feasibility study** first checks whether the project is technically, economically and operationally possible.

Development models — different ways of moving through the stages:

Model	How it works	Use it when
Waterfall	Each stage once, in strict order; no going back	Requirements are clear and won't change
V-model	Each development stage has a matching testing stage (verification & validation)	Safety-critical systems (medical, aviation)
Prototype	Build a quick rough model, let users try it, then build the real thing	Users aren't sure what they want
Incremental	Deliver the software in working pieces, adding features each time	Early partial delivery is useful
Spiral (Boehm)	Repeated loops; every loop includes RISK ANALYSIS	Large, expensive, high-risk projects
RAD	Very fast development using reusable components	Tight deadlines, modular system
Agile (Scrum, XP)	Short cycles (sprints of 1–4 weeks), working software each sprint, constant customer feedback	Requirements keep changing

> 🧠 **MEMORY TRICK**
> Spiral = Risk. Prototype = unclear requirements. Waterfall = fixed requirements. Agile = changing requirements. V-model = testing for every stage.

Design tools: DFD (Data Flow Diagram) shows how data moves through the system — level 0 is called the **context diagram** (the whole system as one bubble). **UML** diagrams (use-case, class, sequence, activity) are used for object-oriented design.

> ⏱ **RECAP IN 30 SECONDS**
> ✓ SDLC: requirements → design → coding → testing → deployment → maintenance
> ✓ SRS = functional + non-functional requirements
> ✓ Waterfall fixed; prototype unclear; spiral risk; agile changing; V-model testing per stage
> ✓ DFD level 0 = context diagram

✍️ **Quick quiz**

Q1. Which model explicitly includes risk analysis in each loop?

(A) V-model
(B) RAD
(C) Spiral
(D) Waterfall

Q2. When requirements are unclear at the start, use the:

(A) V-model
(B) Prototype model
(C) Waterfall model
(D) Big-bang model

Q3. Which SDLC phase usually costs the most over a product's life?

(A) Coding
(B) Maintenance
(C) Design
(D) Testing

Answers: Q1 C — Boehm's spiral model is risk-driven. · Q2 B — A prototype helps users discover what they want. · Q3 B — Software is fixed and changed for years after release.

LESSON 10.2 · 18 MIN

Testing, cohesion & coupling, and metrics

In one line: **Testing finds mistakes before users do; good design has modules that each do one job well (high cohesion) and depend on each other little (low coupling).**

Two ways of testing:

	Black-box testing	White-box testing
Looks at	Only inputs and outputs — not the code	The code inside — every path and branch
Done by	Testers, users	Developers
Techniques	Equivalence partitioning, boundary value analysis	Statement, branch and path coverage; cyclomatic complexity
Like	Checking a TV by pressing buttons	Opening the TV and checking the circuit

SOLVED EXAMPLE

Boundary value analysis: a form accepts age 18 to 60

1. Errors usually happen at the edges, so test values at and around the boundaries.
2. Test 17 (just below, should reject), 18 (lowest valid), 19, 59, 60 (highest valid), 61 (just above, should reject).

✅ **Test values: 17, 18, 19, 59, 60, 61**

Level of testing	What is tested
Unit testing	One function/module alone (by the developer)
Integration testing	Modules joined together — do they work with each other?
System testing	The whole system against the requirements
Acceptance testing	The customer checks it meets their needs; alpha (at the developer's site) and beta (real users at their own site)
Regression testing	Re-testing after a change to make sure nothing old broke

Verification vs validation: verification = "Are we building the product RIGHT?" (reviews, inspections against the design); validation = "Are we building the RIGHT product?" (testing with users against their needs).

Cyclomatic complexity (McCabe) measures how complicated a program's logic is = the number of independent paths through the code = the minimum number of test cases needed for full path coverage.

📐 **FORMULAS**

V(G) — from the flow graph

`E − N + 2P (E = edges, N = nodes, P = connected parts, usually 1)`

V(G) — from the code **`number of decision points (if, while, for, case) + 1`**

V(G) — from the picture **`number of enclosed regions + 1 (outer region counted)`**

➖ **SOLVED EXAMPLE**

A flow graph has 9 edges and 7 nodes (one program). Find V(G).

1. E = 9, N = 7, P = 1.
2. V(G) = 9 − 7 + 2 × 1 = 4.

✅ V(G) = 4 → at least 4 test cases to cover every independent path

➖ **SOLVED EXAMPLE**

Code with two if statements and one while loop

1. Decision points = 2 (ifs) + 1 (while) = 3.
2. V(G) = 3 + 1.

✅ V(G) = 4

Cohesion = how closely the parts INSIDE one module belong together. **Coupling** = how much one module DEPENDS ON another. Good design = **high cohesion, low coupling**.

> 💡 **THINK OF IT LIKE THIS**
> A good kitchen drawer holds only cutlery (high cohesion). If changing the fridge forced you to rebuild the cupboards, they'd be tightly coupled — bad. You want each room to work on its own (low coupling).

Cohesion: best → worst	Coupling: best → worst
Functional (one task)	Data (only simple data passed)
Sequential	Stamp (a whole record passed)
Communicational	Control (one module controls another's flow)
Procedural	External
Temporal	Common (shared global data)
Logical	Content (one module changes another's internals)
Coincidental (random)	—

> 🧠 **MEMORY TRICK**
> Worst cohesion = Coincidental; worst coupling = Content — both start with C, both are what you must avoid.

Estimating the project: COCOMO estimates effort (person-months) from the size in KLOC (thousand lines of code), in three modes — organic (small, simple), semi-detached, embedded (complex, strict). **Function point analysis** measures size from inputs, outputs, enquiries, files and interfaces instead of lines of code.

> ⏱ **RECAP IN 30 SECONDS**
> ✓ Black-box = inputs/outputs (BVA, equivalence); white-box = code paths
> ✓ Unit → integration → system → acceptance (alpha, beta); regression after changes
> ✓ Verification = built right; validation = right product
> ✓ V(G) = E − N + 2P = decisions + 1
> ✓ High cohesion (functional best), low coupling (data best)

✍️ **Quick quiz**

Q1. Cyclomatic complexity of a graph with 10 edges, 8 nodes, 1 component:

(A) 4
(B) 3
(C) 18
(D) 2

Q2. The best type of cohesion is:

(A) Temporal
(B) Functional
(C) Logical
(D) Coincidental

Q3. Boundary value analysis is a ___ technique.

(A) integration
(B) white-box
(C) black-box
(D) regression

Answers: Q1 A — 10 − 8 + 2 = 4. · Q2 B — Every part contributes to one task. · Q3 C — It uses only the specification, not the code.

MODULE 11

Theory of computation & compilers

Which machines can recognise which patterns, and how a compiler turns your code into machine code.

LESSON 11.1 · 18 MIN

Automata & the Chomsky hierarchy

In one line: **An automaton is an imaginary machine that reads a string and says YES or NO; machines with more memory can recognise more complicated patterns.**

Three basic words. An **alphabet** is a set of symbols, e.g. {0, 1}. A **string** is a sequence of those symbols, e.g. 0110. A **language** is a set of strings, e.g. "all binary strings ending in 1".

What is an automaton? A simple imaginary machine that reads a string one symbol at a time, moves between **states**, and at the end says **accept** (the string is in the language) or **reject**.

> 💡 **THINK OF IT LIKE THIS**
>
> A turnstile at a metro gate is a finite automaton: two states (Locked, Unlocked). Insert a token → Unlocked. Push through → Locked again. It remembers nothing except which state it is in.

DFA (Deterministic Finite Automaton) — for each state and each input symbol there is exactly ONE next state. **NFA (Non-deterministic)** — there may be several possible next states (or none). Surprisingly, DFAs and NFAs are **equally powerful**: any NFA can be converted into a DFA (the DFA may need up to 2^n states).

SOLVED EXAMPLE

Build a DFA that accepts binary strings ending in 1

1. We only need to remember one thing: was the LAST symbol a 1?
2. State q0 = "last symbol was 0, or nothing read yet" (start state, not accepting).
3. State q1 = "last symbol was 1" (accepting).
4. From ANY state: reading 1 → go to q1; reading 0 → go to q0.
5. Test 0101: q0 →0→ q0 →1→ q1 →0→ q0 →1→ q1 → ends in q1 → ACCEPT ✓. Test 110: ends in q0 → REJECT ✓.

✅ 2 states are enough

Regular expressions describe the same languages that finite automata accept: `a*` = zero or more a's, `a+` = one or more, `a|b` = a or b, `(ab)*` = ab repeated. Example: `(0|1)*1` = binary strings ending in 1.

The limit of finite automata: they have no memory except their states, so they cannot COUNT without limit. The language $a^n b^n$ (n a's followed by exactly n b's, like aabb) needs counting — a finite automaton can't do it. A **pushdown automaton (PDA)** adds a **stack**: push for each a, pop for each b, accept if the stack is empty.

The Chomsky hierarchy — four levels of languages, each recognised by a more powerful machine:

Type	Language / grammar	Machine	Memory	Example
Type 3	Regular	Finite automaton (DFA/NFA)	None — only states	Strings ending in 1
Type 2	Context-free	Pushdown automaton	One stack	$a^n b^n$, balanced brackets
Type 1	Context-sensitive	Linear bounded automaton	Tape as long as the input	$a^n b^n c^n$
Type 0	Recursively enumerable (unrestricted)	Turing machine	Infinite tape	Anything computable

> 🧠 **MEMORY TRICK**
> Each level contains the one below: Regular ⊂ Context-free ⊂ Context-sensitive ⊂ Recursively enumerable. More memory → more power.

Turing machine — the most powerful model: a head reading and writing on an infinite tape. Anything a real computer can compute, a Turing machine can too. Some problems are **undecidable** — no algorithm can solve them for every input. The famous one is the **halting problem**: deciding whether ANY given program will eventually stop.

> ⏱ **RECAP IN 30 SECONDS**
> - ✓ Alphabet → strings → language; an automaton accepts or rejects strings
> - ✓ DFA and NFA are equally powerful (subset construction)
> - ✓ Regular → FA; context-free → PDA (stack); context-sensitive → LBA; recursively enumerable → Turing machine
> - ✓ $a^n b^n$ is context-free but not regular
> - ✓ The halting problem is undecidable

✍️ **Quick quiz**

Q1. Which machine recognises context-free languages?

(A) Mealy machine
(B) Pushdown automaton
(C) Finite automaton
(D) Only a Turing machine

Q2. The language $\{a^n b^n \mid n \geq 0\}$ is:

(A) Finite
(B) Not context-free
(C) Regular
(D) Context-free but not regular

Q3. The halting problem is:

(A) Solvable by a DFA
(B) Regular
(C) Undecidable
(D) Decidable

Answers: Q1 B — The stack handles nesting and counting. · Q2 D — It needs counting → a stack, not just states. · Q3 C — Turing proved it undecidable.

LESSON 11.2 · 16 MIN

How a compiler works: the six phases

In one line: **A compiler is an assembly line: characters → tokens → grammar tree → meaning check → intermediate code → better code → machine code.**

A compiler translates a high-level program into machine code in stages, called **phases**. Follow the line `sum = a + 10;` through each one:

Phase	What it does	For sum = a + 10;
1. Lexical analysis (scanner)	Groups characters into TOKENS; removes spaces and comments	sum | = | a | + | 10 | ; → 6 tokens
2. Syntax analysis (parser)	Checks the GRAMMAR; builds a parse/syntax tree	assignment → identifier = expression ;
3. Semantic analysis	Checks MEANING: are variables declared? do types match?	Are sum and a declared? Is int + int okay?
4. Intermediate code generation	Machine-independent simple code	t1 = a + 10 ; sum = t1 (three-address code)
5. Code optimisation	Make it faster/smaller	sum = a + 10 (temporary t1 removed)
6. Code generation	Target machine/assembly code	MOV R1, a ; ADD R1, #10 ; MOV sum, R1

The symbol table (names, types, scopes) and the error handler are used by every phase

MEMORY TRICK

"Little Students Should Interest Our Classes" — Lexical, Syntax, Semantic, Intermediate code, Optimisation, Code generation.

SOLVED EXAMPLE

How many tokens in: int x = a + 10;

1. Split at every meaningful unit: int | x | = | a | + | 10 | ;
2. Keywords (int), identifiers (x, a), operators (=, +), constants (10) and punctuation (;) are all tokens.

✅ 7 tokens

Which error is caught where?

Error	Phase that catches it
Illegal character like @ in a variable name	Lexical
Missing semicolon or bracket	Syntax
Using an undeclared variable; adding a string to an int	Semantic

- Front end (phases 1–4) depends on the SOURCE language; back end (5–6) depends on the TARGET machine
- Parsers: top-down (recursive descent, LL(1)) and bottom-up (shift-reduce, LR, SLR, LALR, CLR)
- Tools: Lex/Flex generates the lexical analyser; Yacc/Bison generates an LALR parser
- Optimisations: constant folding (2 * 3 → 6), dead-code elimination, common sub-expression elimination, loop-invariant code motion
- Cross-compiler: runs on one machine, produces code for another (e.g. PC → mobile chip)

RECAP IN 30 SECONDS

✓ Lexical (tokens) → Syntax (grammar tree) → Semantic (meaning/types) → Intermediate code → Optimisation → Code generation

✓ Symbol table is shared by all phases

✓ Missing ; = syntax error; undeclared variable = semantic error

✓ Lex → scanner; Yacc → LALR parser

✍️ **Quick quiz**

Q1. Which phase removes comments and whitespace?

(A) Lexical analysis
(B) Syntax analysis
(C) Semantic analysis
(D) Code generation

Q2. Type checking happens in:

(A) Code optimisation
(B) Lexical analysis
(C) Semantic analysis
(D) Syntax analysis

Q3. YACC generates which kind of parser?

(A) LALR
(B) Operator precedence only
(C) Recursive descent
(D) LL(1)

Answers: Q1 A — The scanner ignores them while making tokens. · Q2 C — Meaning, not just grammar. · Q3 A — Yet Another Compiler-Compiler → LALR(1).

MODULE 12

Web, emerging trends & cyber law

How websites work, today's big technologies and India's cyber law — easy marks once you've read it once.

LESSON 12.1 · 16 MIN

How the web works: HTML, CSS, JavaScript & HTTP

In one line: **HTML is the skeleton of a web page, CSS its clothes, JavaScript its muscles — and HTTP is the postman that carries pages from the server to your browser.**

What happens when you open a website? 1) You type `www.example.com`. 2) DNS finds the server's IP address. 3) Your browser (the **client**) sends an **HTTP request** to the **web server**. 4) The server sends back an **HTTP response** with the page (HTML, CSS, JavaScript, images). 5) The browser draws the page. This is the **client–server model**.

URL parts: `https://www.example.com:443/results/2026?roll=12#top` → protocol (https) · domain (www.example.com) · port (443) · path (/results/2026) · query (?roll=12) · fragment (#top).

HTML (HyperText Markup Language) describes the STRUCTURE of a page using **tags** in angle brackets. Most tags come in pairs: `<p>` opens, `</p>` closes. HTML is a markup language, NOT a programming language.

HTML

```
<!DOCTYPE html>
<html>
  <head><title>My Page</title></head>
  <body>
    <h1>Welcome</h1>
    <p>This is a <b>paragraph</b>.</p>
    <a href="https://bpsc.bihar.gov.in">BPSC site</a>
    <img src="logo.png" alt="Logo">
  </body>
</html>
```

Output ▸ A heading, a paragraph with bold text, a link and an image

Tag	Purpose
<h1> … <h6>	Headings (h1 biggest, h6 smallest)
<p>	Paragraph
<a href="…">	Hyperlink (anchor)
<img src="…" alt="…">	Image (alt = text shown if the image can't load)
/ /	Bulleted list / numbered list / list item
<table> <tr> <th> <td>	Table / row / header cell / data cell
<form> <input> <button>	Forms for user input (method GET or POST)
 , <hr>, <img>, <input>	Empty (void) tags — no closing tag

CSS (Cascading Style Sheets) controls the LOOK — colours, fonts, spacing, layout. Example: `h1 { color: blue; font-size: 24px; }` . Ways to add CSS: **inline** (style="…" on a tag), **internal** (<style> in the page), **external** (a separate .css file). Priority when they conflict: **inline > internal > external**. Selectors: element (`p`), class (`.note`), id (`#header`).

JavaScript makes pages interactive (form checks, pop-ups, live updates). It runs in the BROWSER (client side). Languages like PHP, Python (Django), Java (JSP), ASP.NET and Node.js run on the SERVER. The **DOM (Document Object Model)** is the page represented as a tree that JavaScript can read and change.

XML (eXtensible Markup Language) — you invent your own tags; used to STORE and TRANSPORT data, not to display it. A **DTD** or **XSD** defines which tags are allowed.

HTTP is the protocol of the web. It is **stateless** — each request is independent, so websites use **cookies** and **sessions** to remember you. **HTTPS** = HTTP + TLS encryption (port 443).

HTTP method	Use	Data sent
GET	Fetch a page	In the URL (visible, limited length, can be bookmarked)
POST	Submit a form	In the request body (not in the URL — use for passwords)
PUT / DELETE	Update / delete a resource	—

Status code	Meaning
200	OK — success
301 / 302	Moved permanently / temporarily (redirect)
400	Bad request
401 / 403	Not logged in / forbidden
404	Not found
500	Internal server error
503	Service unavailable (server overloaded/down)

1xx info · 2xx success · 3xx redirect · 4xx your (client) mistake · 5xx server's mistake

⏱ RECAP IN 30 SECONDS

- ✓ Browser (client) sends an HTTP request; the web server sends a response
- ✓ HTML = structure (tags), CSS = style (inline > internal > external), JavaScript = behaviour (client side)
- ✓ XML = custom tags for data; DOM = page as a tree
- ✓ GET puts data in the URL; POST in the body
- ✓ 200 OK, 301 moved, 404 not found, 500 server error

✍ **Quick quiz**

Q1. Which HTTP status code means the server crashed while handling the request?

(A) 500 (B) 200
(C) 404 (D) 301

Q2. Which CSS has the highest priority?

(A) External (B) Internal
(C) Browser default (D) Inline

Q3. Which runs on the client side?

(A) PHP (B) JavaScript (in the browser)
(C) JSP (D) ASP.NET

Answers: Q1 A — 5xx = server error. · Q2 D — Inline styles override internal and external ones. · Q3 B — The others run on the server.

LESSON 12.2 · 16 MIN

Emerging technologies, cyber security & the IT Act

In one line: **For each new technology learn one definition, one example and one Indian fact — that's exactly how exams ask it.**

Emerging technologies — what each one really means:

Technology	In plain words	Example
Artificial Intelligence (AI)	Machines doing tasks that need human-like intelligence	Chatbots, face unlock, crop-disease apps
Machine Learning (ML)	Part of AI: the computer learns patterns from data instead of being told rules	Spam filters learning from marked e-mails
Deep Learning	ML using many-layered neural networks	Speech recognition, image recognition
Cloud computing	Using computing (storage, servers, software) over the Internet, pay as you use	Google Drive, DigiLocker
Internet of Things (IoT)	Everyday objects with sensors connected to the Internet	Smart meters, fitness bands, smart farming sensors
Big Data	Data too large/fast/varied for normal tools — the 5 V's: Volume, Velocity, Variety, Veracity, Value	Hadoop, Spark
Blockchain	A shared record book (ledger) copied on many computers; blocks are chained so past entries can't be secretly changed	Cryptocurrency, land records
AR / VR	AR adds digital objects to the real view; VR replaces it with a virtual world	AR: Pokémon Go; VR: headsets
Edge computing	Processing data near where it is produced instead of a far cloud	Smart traffic cameras

Cloud service models — how much the provider manages for you:

Model	You get	Example
IaaS (Infrastructure)	Virtual machines, storage — you install everything else	AWS EC2
PaaS (Platform)	A ready platform to build and run your apps	Google App Engine
SaaS (Software)	Ready-to-use software in the browser	Gmail, Google Docs

Cloud deployment: public, private, hybrid, community

> **MEMORY TRICK**
> "I Paint Sky" — IaaS → PaaS → SaaS: from most control (you manage more) to least.

Cyber security basics. Threats: **virus** (attaches to files and spreads when you run them), **worm** (spreads by itself across networks), **Trojan horse** (looks useful but is harmful), **ransomware** (locks

your files and demands money), **spyware**, **phishing** (fake messages/sites to steal passwords or OTPs), **DoS/DDoS** (flooding a site so it crashes). Protection: strong passwords, two-factor authentication, updates, antivirus, firewalls, never sharing OTPs.

India's cyber law — Information Technology Act, 2000 (amended in 2008). It gives legal recognition to electronic records and digital signatures, and defines cyber offences:

Section	About
43	Damage to a computer system (unauthorised access, viruses) — compensation
66	Computer-related offences (hacking)
66C	Identity theft (misusing someone's password/digital signature)
66D	Cheating by impersonation using a computer
66E	Violation of privacy (capturing/publishing private images)
66F	Cyber terrorism
67	Publishing obscene material electronically

> ⚠ **EXAM TRAP**
> Section 66A (offensive messages) was struck down by the Supreme Court in 2015 (Shreya Singhal case) — it is not a valid offence today.

- CERT-In (Indian Computer Emergency Response Team) handles cyber incidents in India
- Digital Personal Data Protection Act, 2023 governs how personal data is collected and used
- Intellectual property: copyright (software, books, music), patent (inventions), trademark (brand names, logos)
- Open-source software (FOSS) — Linux, LibreOffice, Python, Firefox — free to use, study and modify (licences such as GPL)
- Societal impact topics: digital footprint, netiquette, plagiarism, cyber bullying, e-waste (dispose through authorised recyclers)

⏱ **RECAP IN 30 SECONDS**

- ✓ AI ⊃ ML ⊃ deep learning; IoT = connected devices; blockchain = tamper-proof shared ledger
- ✓ IaaS → PaaS → SaaS (Gmail is SaaS)
- ✓ Virus needs a host; worm spreads itself; Trojan pretends to be useful
- ✓ IT Act 2000 (amended 2008): 43 damage, 66 hacking, 66C identity theft, 66F cyber terrorism, 67 obscenity
- ✓ 66A struck down in 2015

✍ **Quick quiz**

Q1. Gmail is an example of:

(A) SaaS
(B) IaaS
(C) PaaS
(D) On-premise software

Q2. India's IT Act was passed in:

(A) 2008
(B) 2000
(C) 2015
(D) 1998

Q3. Identity theft is punishable under IT Act Section:

(A) 43
(B) 66F
(C) 67
(D) 66C

Answers: Q1 A — Ready-to-use software over the Internet. · Q2 B — Amended in 2008. · Q3 D — 66C — identity theft; 66F — cyber terrorism.

© EdQity — Computer Science course. Original lessons; confirm topics against your exam's official syllabus.